OWASP JUICE SHOP

VULNERABILITY ASSESSMENT REPORT

Web Application Security Assessment — Consolidated Findings

CONFIDENTIAL

Target Application: OWASP Juice Shop | http://127.0.0.1:42000

Prepared by:

Nadine Eissa

Sherif Hassan

Mariz Zaki

Kirolos Nabil

Tuqa Diab

Shahd Hamid

Date: July 2026

1. Introduction

This report consolidates the results of a web application security assessment performed against OWASP Juice Shop, a deliberately vulnerable training application maintained by the OWASP Foundation. It was prepared by Nadine Eissa, Sherif Hassan, Mariz Zaki, Kirolos Nabil, Tuqa Diab and Shahd Hamid as a combined write-up of testing carried out across several assessment sessions, and merges those sessions into a single, consistently structured document.

Thirty (30) distinct findings are documented, numbered VUL-001 through VUL-030. Each finding follows the same structure: a metadata summary (severity, CVSS score and vector, OWASP Top 10:2021 category, affected URL, and CWE reference), a description of the root cause, a step-by-step proof of concept with the original screenshots, the business impact, recommended remediation, and verified reference links.

Findings are ordered roughly by where they occur in a typical attack path (reconnaissance and information disclosure first, moving through authentication and access-control failures, injection flaws, and configuration issues). Several findings share the same underlying root cause (for example, the SQL Injection and Cross-Site Scripting issues each appear more than once, against different endpoints or accounts) — these are kept as separate entries because each was demonstrated independently, with its own proof of concept, but the shared root cause is called out explicitly in the affected findings' descriptions.

2. Summary of Findings

The table below lists all 30 findings with their assigned severity for quick reference.

ID	Title	Severity
VUL-001	Information Disclosure via Hidden Score Board Route	Low
VUL-002	DOM-Based Cross-Site Scripting (XSS) via Product Search Input	High
VUL-003	Client-Side Cross-Site Scripting (XSS) via Bonus Payload Injection	High
VUL-004	Information Disclosure via Missing Security Boundary on Privacy Policy Content	Low
VUL-005	Sensitive Data Exposure via Misplaced Cryptographic Signature File (Poison Null Byte)	High
VUL-006	SQL Injection Authentication Bypass in login.js	Critical
VUL-007	Missing Function-Level Access Control (Admin Section) in main.js	High
VUL-008	Weak Password Policy (Broken Authentication) in User Registration & Update	High
VUL-009	Broken Access Control — Insecure Direct Object References (IDOR) in View Basket	High
VUL-010	Security Misconfiguration — Deprecated Interface Access	Medium
VUL-011	Reflected Cross-Site Scripting (XSS) in Search Functionality	High
VUL-012	Improper Input Validation — Zero Stars (Customer Feedback Rating)	Medium
VUL-013	Improper Input Validation — Repetitive Registration	Medium
VUL-014	Unvalidated Redirects and Forwards (Outdated URL Allowlist)	Medium
VUL-015	Web3 Sandbox Environment Security Misconfiguration	High
VUL-016	Visual Geo Stalking (Sensitive Data Exposure via Photo Wall)	High

VUL-017	Login Jim — SQL Injection Authentication Bypass (Targeted Account)	**Critical**
VUL-018	Login Bender — SQL Injection Authentication Bypass (Targeted Account)	**Critical**
VUL-019	Client-Side XSS Protection Bypass via Direct API Manipulation (Stored XSS)	**High**
VUL-020	Forged Feedback (Broken Access Control via Hidden UserId Parameter)	**Medium**
VUL-021	Confidential Document Exposure via Predictable Identifiers	**High**
VUL-022	Verbose Error Handling (Security Misconfiguration)	**Medium**
VUL-023	Exposed Application Metrics and Progress Data	**Medium**
VUL-024	Mass Dispel — Missing Per-Item Authorization on Batch Operations	**Medium**
VUL-025	Missing Output Encoding Enabling Cross-Site Scripting	**High**
VUL-026	Broken Access Control — Unauthorized Deletion of Customer Feedback	**High**
VUL-027	Account Takeover via Weak, OSINT-Recoverable Password	**High**
VUL-028	Location Metadata Exposure Enabling Account Takeover	**High**
VUL-029	Exposed Wallet Seed Phrase Enabling Crypto Wallet Takeover	**Critical**
VUL-030	Published Security Policy (security.txt) — Informational	**Low**

VUL-001 — Information Disclosure via Hidden Score Board Route

Finding ID	VUL-001
Title	Information Disclosure via Hidden Score Board Route
Severity	**Low**
CVSS v3.1 Score	2.7
CVSS Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
OWASP Category	A01:2021 - Broken Access Control
Affected URL	http://127.0.0.1:42000/#/score-board
CWE	CWE-200: Exposure of Sensitive Information to an Unauthorized Actor

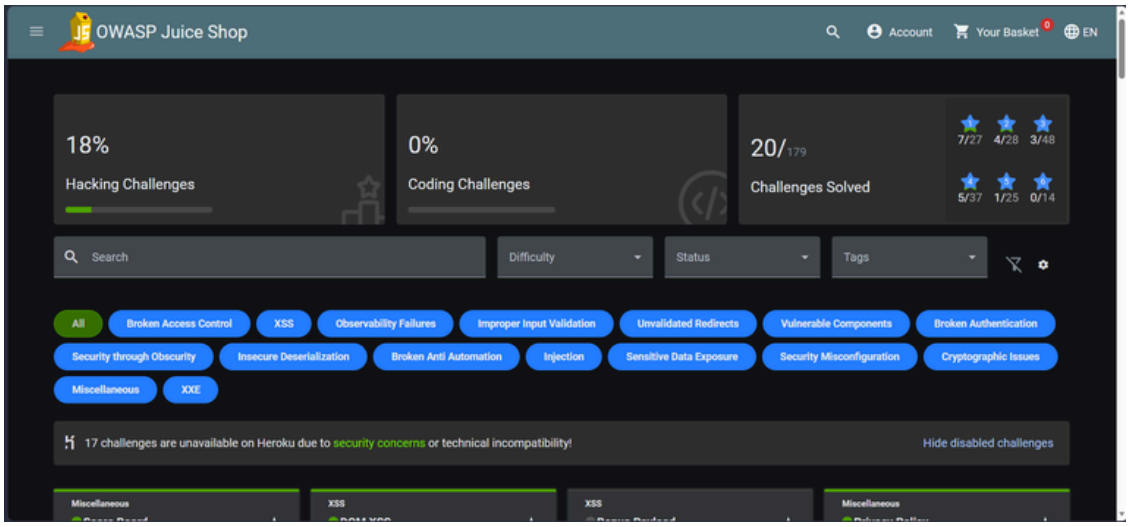
4.1 Description

The web application implicitly exposes a functional tracking dashboard ("Score Board") intended for monitoring client-side progress or administrative metrics. While the navigation link is omitted from the main user interface menus, the route itself remains registered and accessible in the client-side JavaScript bundle (main.js). An unauthenticated user can extract this endpoint by auditing frontend source code and accessing the page directly, leading to unauthorized exposure of application testing configurations and available operational challenge listings.

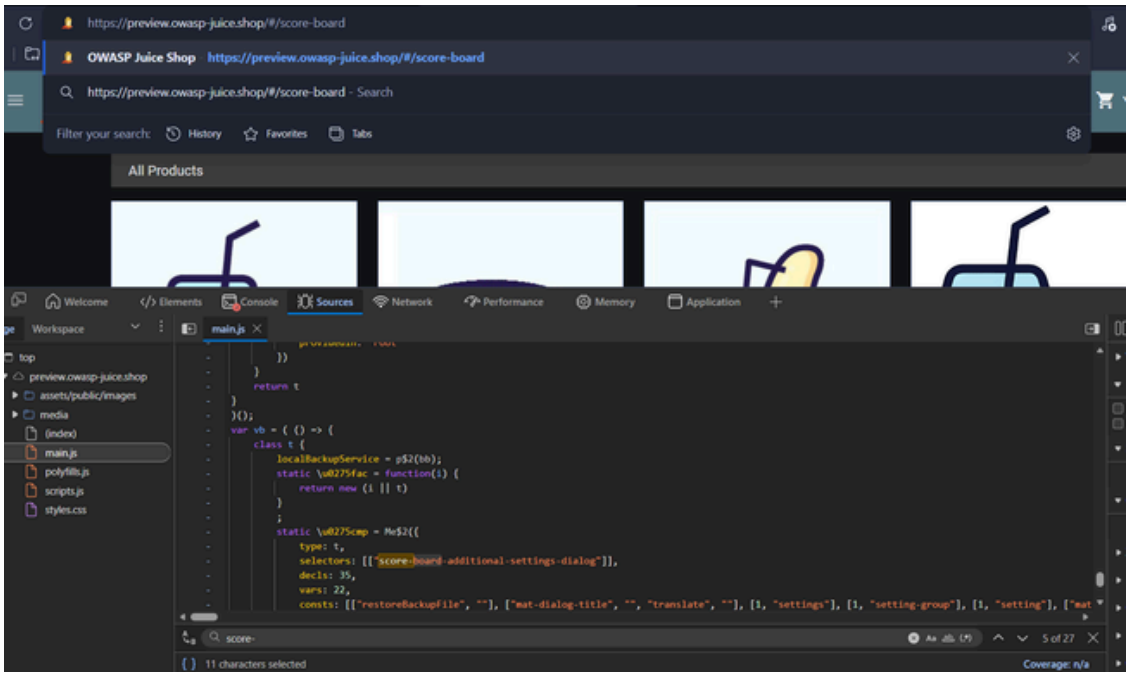
4.2 Proof of Concept

Steps to Reproduce:

1. Open a web browser and navigate to the OWASP Juice Shop homepage at http://127.0.0.1:42000/
2. Open the browser's Developer Tools (F12) and switch to the Debugger or Sources tab.
3. Search through the client-side JavaScript bundles (such as main.js) for references to "score-board".
4. Identify the structural application route mapping to the score board path.
5. Manually append the discovered path to the base URL in the browser address bar: http://127.0.0.1:42000/#/score-board
6. The application successfully loads the hidden page, confirming that access control enforcement is missing on this specific view.



The score board route located via the browser Sources/Debugger panel and loaded directly by URL.



The hidden Score Board page loading successfully for an unauthenticated view, showing hacking/coding challenge progress.

4.3 Impact

An unauthenticated attacker can discover hidden administrative functionality and map out available features or security challenges within the application environment. While it does not allow explicit data modification or systemic execution, exposing administrative or hidden components violates proper system hardening standards and facilitates subsequent, targeted exploitation efforts.

4.4 Remediation

- Restrict Client-Side Route Registration: Ensure that administrative or developer-centric application views are conditionally rendered and packaged separately, preventing raw route definitions from compiling into general public deployment bundles.
- Implement Server-Side Access Control: Restrict visibility to sensitive functional metrics or progress-tracking components by validating user authorization scopes before returning page resources or API definitions.
- Code Obfuscation and Hardening: Remove verbose testing flags or developer routing artifacts from production bundles to minimize structural footprint availability during basic source code mapping.

4.5 References

- OWASP Broken Access Control: https://owasp.org/Top10/A01_2021-Broken_Access_Control/
- CWE-200: Exposure of Sensitive Information: https://cwe.mitre.org/data/definitions/200.html

VUL-002 — DOM-Based Cross-Site Scripting (XSS) via Product Search Input

Finding ID	VUL-002
Title	DOM-Based Cross-Site Scripting (XSS) via Product Search Input
Severity	**High**
CVSS v3.1 Score	8.1
CVSS Vector	AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:N
OWASP Category	A03:2021 - Injection
Affected URL	http://127.0.0.1:42000/#/search
CWE	CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')

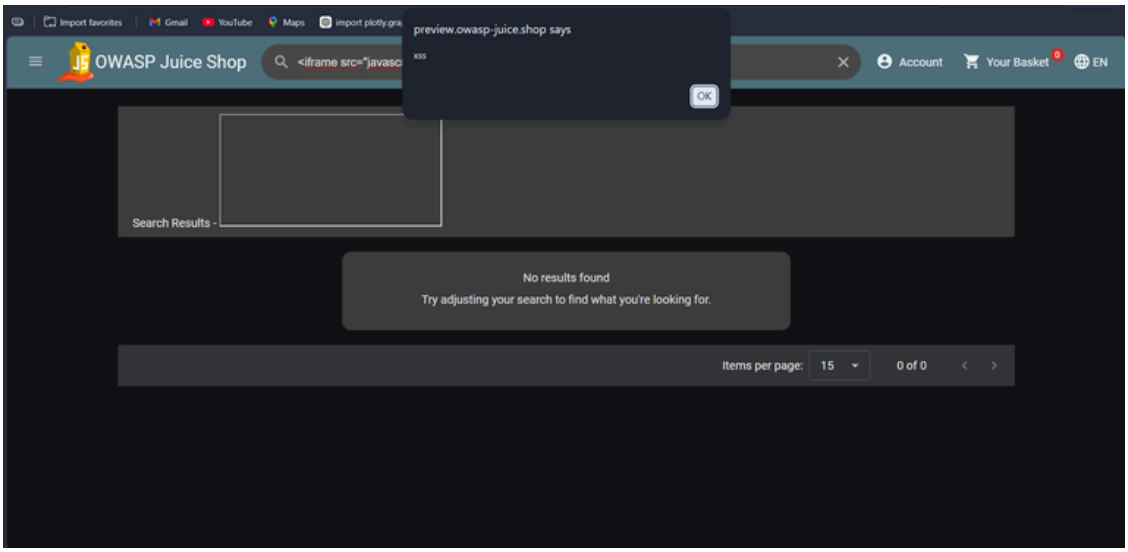
5.1 Description

The web application's product search interface is vulnerable to DOM-Based Cross-Site Scripting (XSS). When a user inputs a search query, the application dynamically updates the Document Object Model (DOM) to display the query back to the user without proper sanitization or encoding. An attacker can craft a malicious URL containing a JavaScript payload; when executed in a victim's browser, the application handles the untrusted input insecurely through client-side scripts, triggering the execution of arbitrary JavaScript code.

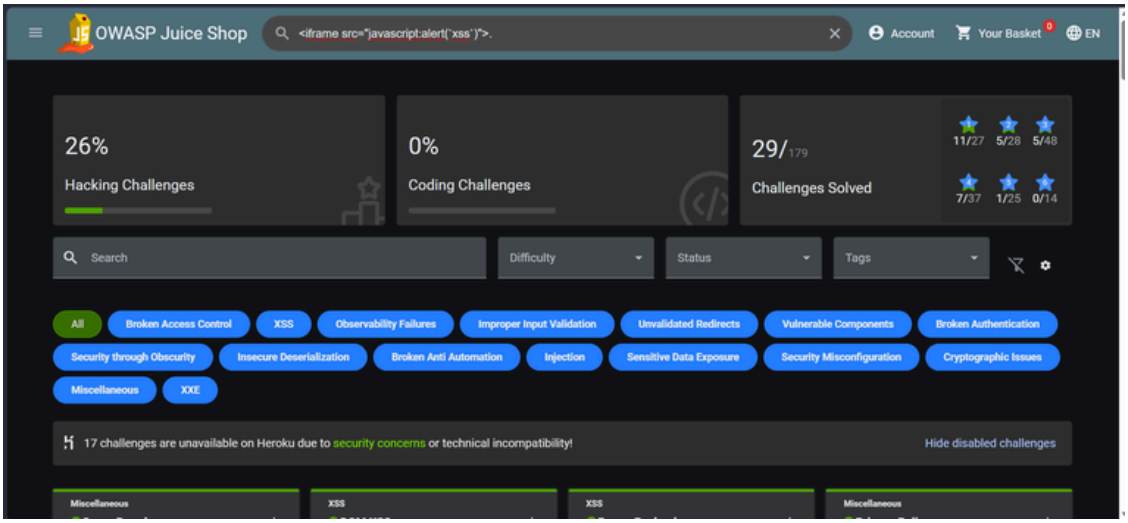
5.2 Proof of Concept

Steps to Reproduce:

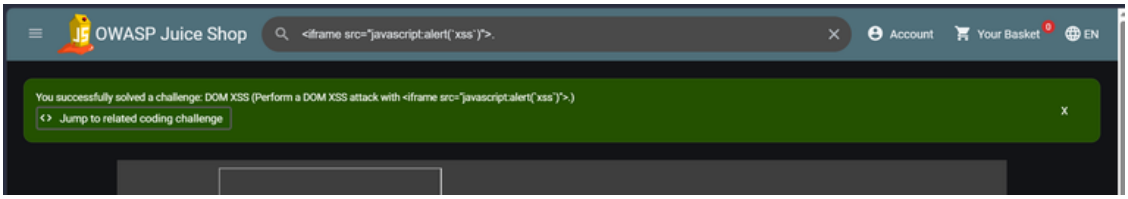
1. Open a web browser and navigate to the OWASP Juice Shop homepage at http://127.0.0.1:42000/
2. Locate the main Search Bar (Product Search field) at the top of the interface.
3. Inject the following dynamic iframe payload into the search input field: <iframe src="javascript:alert(xss)">
4. Press Enter or click the search icon to submit.
5. The client-side framework processes the injected string directly within the DOM structure, creating an active HTML iframe element that immediately executes the script, rendering a JavaScript alert popup window.



The crafted payload entered into the search bar; no client- or server-side sanitization is applied before the value is reflected back into the page.



Execution of the injected script — the JavaScript alert fires, confirming DOM-based script execution.



The application's built-in challenge tracker confirms the DOM XSS challenge was solved.

5.3 Impact

Successful exploitation allows an attacker to execute arbitrary client-side JavaScript within the context of the victim's browser session. This can lead to session hijacking via session token/cookie theft, unauthorized state-changing actions on behalf of the authenticated user, harvesting of sensitive data, or redirection to external malicious phishing web applications.

5.4 Remediation

- Implement Context-Aware Output Encoding: Ensure all user inputs dynamically reflected into the UI are strictly encoded using secure client-side templates or context-aware libraries (e.g., using safe bindings like Angular's standard interpolation {{ }} instead of insecure methods like bypassSecurityTrustHtml).
- Enforce Strict Input Sanitization: Pass untrusted inputs through a verified client-side HTML sanitizer library (such as DOMPurify) before passing them to any active execution sinks in the DOM.
- Deploy Content Security Policy (CSP): Configure a comprehensive server-side Content Security Policy header to restrict inline script executions and prevent unauthorized external script injections.

5.5 References

- OWASP Cross-Site Scripting (XSS): https://owasp.org/www-community/attacks/xss/
- CWE-79: Improper Neutralization of Input: https://cwe.mitre.org/data/definitions/79.html

VUL-003 — Client-Side Cross-Site Scripting (XSS) via Bonus Payload Injection

Editor's note: Link corrected — see change log.

Finding ID	VUL-003
Title	Client-Side Cross-Site Scripting (XSS) via Bonus Payload Injection
Severity	**High**
CVSS v3.1 Score	8.1
CVSS Vector	AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:N
OWASP Category	A03:2021 - Injection
Affected URL	http://127.0.0.1:42000/#/search
CWE	CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')

6.1 Description

The application features additional reflection vectors where certain input channels can bypass rudimentary string-matching filters under specific conditions. By utilizing a "Bonus Payload" — a more sophisticated HTML/JavaScript structure designed to evade basic filters — an attacker can exploit the application's unsafe interpolation behavior. This triggers arbitrary JavaScript/embedded-content execution within the victim's session, bypassing superficial boundary controls.

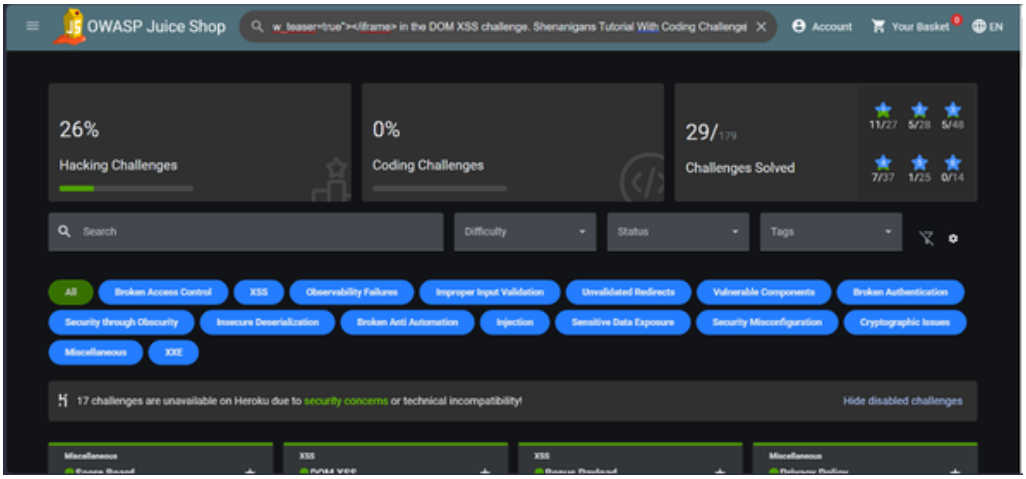
6.2 Proof of Concept

Steps to Reproduce:

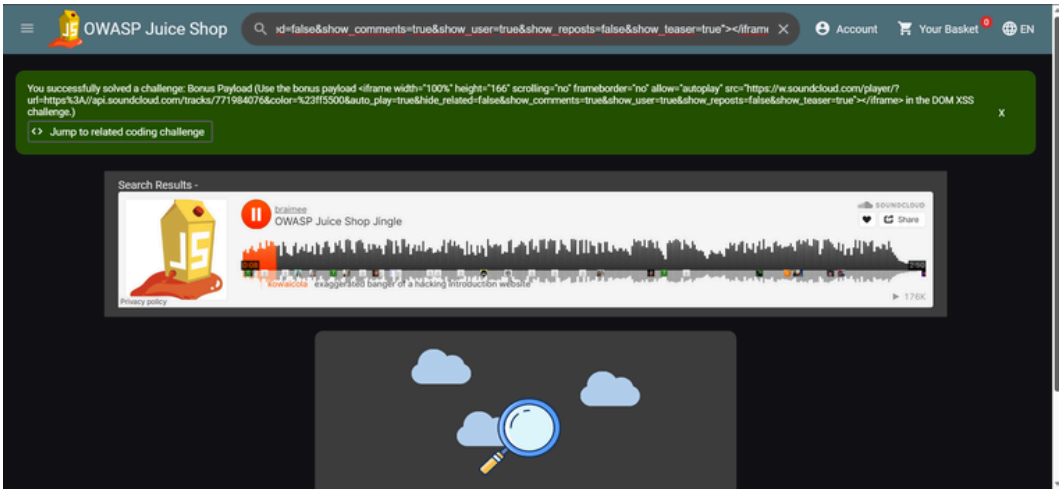
1. Open a web browser and navigate to the OWASP Juice Shop homepage at http://127.0.0.1:42000/
2. Locate the primary Search Bar or input routing interface.
3. Inject the targeted bonus payload designed to test filter limits: <iframe width="100%" height="166" scrolling="no" frameborder="no" allow="autoplay" src="https://w.soundcloud.com/player/?

url=https%3A//api.soundcloud.com/tracks/771984076&color=%23ff5500&auto_play=true&hide_related=false&show_comments=true&show_user=true&show_reposts=false&show_teaser=true"></iframe>

4. Execute the query by pressing Enter.
5. The application dynamically renders the malicious payload within the active browser window, bypassing standard filters and executing the embedded content.



The bonus payload entered into the search bar, targeting the DOM XSS challenge's filter-evasion variant.



The injected iframe renders and plays embedded content directly in the search results, and the application confirms the 'Bonus Payload' challenge as solved.

6.3 Impact

An attacker can completely compromise the client-side execution context. This enables session hijacking via session token theft, execution of unauthorized transactional actions on behalf of the victim, and complex browser-level redirection attacks that undermine the trust and integrity of the application platform.

6.4 Remediation

- Enforce Structural Context Encoding: Never rely on regex or blocklists to filter basic keywords (like 'script'). Instead, use robust, context-aware output encoding across all client-side rendering pathways.
- Integrate Strict Client-Side Sanitization: Utilize production-grade sanitization engines such as DOMPurify to clean and normalize any complex HTML strings before they are allowed to bind with active application elements.
- Implement Strict Content Security Policy (CSP): Restrict script execution entirely to trusted, explicitly hashed local assets by deploying a restrictive server-enforced CSP header layout.

6.5 References

- OWASP XSS Filter Evasion Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/XSS_Filter_Evasion_Cheat_Sheet.html
- CWE-79: Improper Neutralization of Input: https://cwe.mitre.org/data/definitions/79.html

VUL-004 — Information Disclosure via Missing Security Boundary on Privacy Policy Content

Finding ID	VUL-004
Title	Information Disclosure via Missing Security Boundary on Privacy Policy Content
Severity	Low
CVSS v3.1 Score	2.7
CVSS Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
OWASP Category	A01:2021 - Broken Access Control
Affected URL	http://127.0.0.1:42000/#/privacy-security/privacy-policy
CWE	CWE-200: Exposure of Sensitive Information to an Unauthorized Actor

7.1 Description

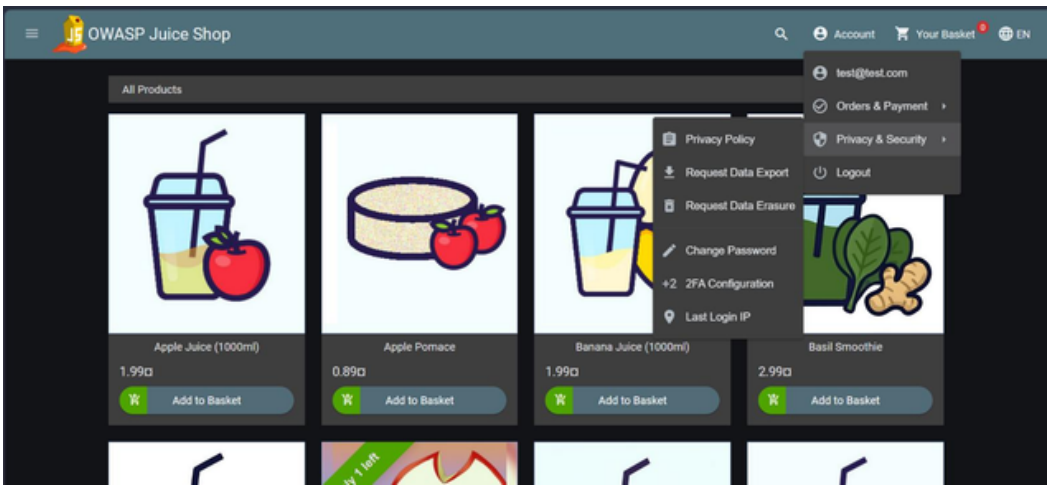
The application features a dedicated privacy policy view that links directly to specific document resources. Due to missing security boundaries and improper path validation in the client-side configuration, an attacker can manipulate parameters or navigation to expose arbitrary application files or older, unlinked document variations. This leads to improper exposure of internal application resources to users who should not have direct access to them.

7.2 Proof of Concept

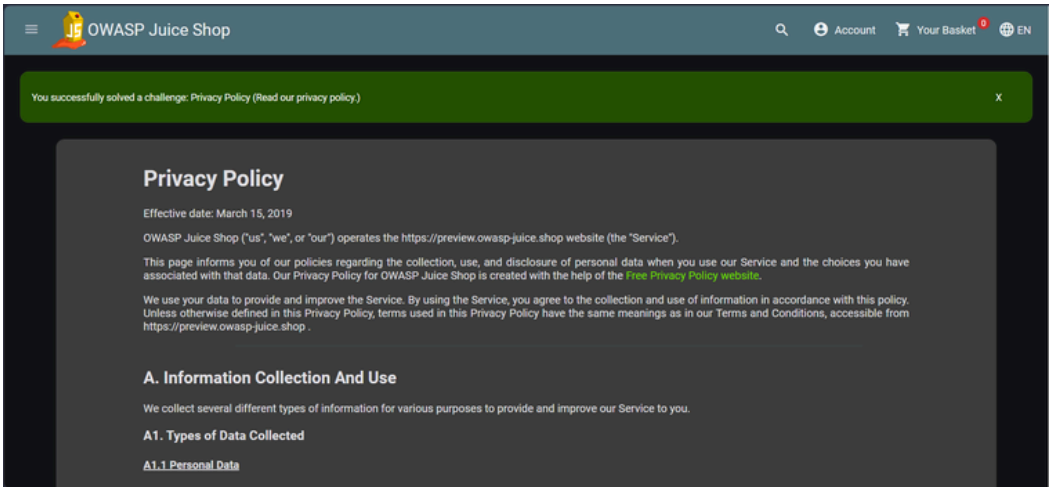
Steps to Reproduce:

1. Open a web browser and navigate to the OWASP Juice Shop homepage at http://127.0.0.1:42000/#/login
2. Authenticate into the application with a valid user account.
3. Once authenticated, expand the main navigation sidebar menu.
4. Navigate to the Privacy & Security dropdown section and select the Privacy Policy option.
5. Review the structural URL path generated by the application: http://127.0.0.1:42000/#/privacy-security/privacy-policy

6. Audit the frontend network responses or modify the embedded file parameters to access unlinked files (such as altering the extension or traversing resource directories).
7. The application renders the requested resource file without strict directory isolation or access-token validation.



Navigating to the 'Privacy & Security' option within the authenticated user dashboard menu.



The Privacy Policy resource loading, confirming the 'Privacy Policy' challenge as solved and highlighting the lack of a stricter access boundary on this and related static resources.

7.3 Impact

Unauthenticated or low-privileged actors can gain visibility into underlying application asset layouts, localized configuration files, or deprecated documentation structures that were meant to remain private. While this does not support direct database modification, it breaks system isolation policies and aids an attacker in building a structural blueprint of the application context.

7.4 Remediation

- Enforce Directory White-listing: Ensure that file-retrieval utilities explicitly restrict paths to a safe, immutable white-list of asset filenames, completely blocking arbitrary file parameter manipulation.
- Remove Client-Side Structural Metadata: Clean out verbose file structures, test files, or unlinked document variations from production builds so they cannot be accessed via direct path hunting.
- Implement Role-Based File Access: Enforce proper server-side authentication and session boundary checks before returning content or metadata related to system operational guidelines.

7.5 References

- OWASP Broken Access Control: https://owasp.org/Top10/A01_2021-Broken_Access_Control/
- CWE-200: Exposure of Sensitive Information: https://cwe.mitre.org/data/definitions/200.html

VUL-005 — Sensitive Data Exposure via Misplaced Cryptographic Signature File (Poison Null Byte)

Editor's note: Reference updated from the retired 2017 Top 10 link — see change log.

Finding ID	VUL-005
Title	Sensitive Data Exposure via Misplaced Cryptographic Signature File (Poison Null Byte)
Severity	**High**
CVSS v3.1 Score	7.5
CVSS Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
OWASP Category	A02:2021 - Cryptographic Failures
Affected URL	http://127.0.0.1:42000/ftp
CWE	CWE-552: Files or Directories Accessible to External Parties

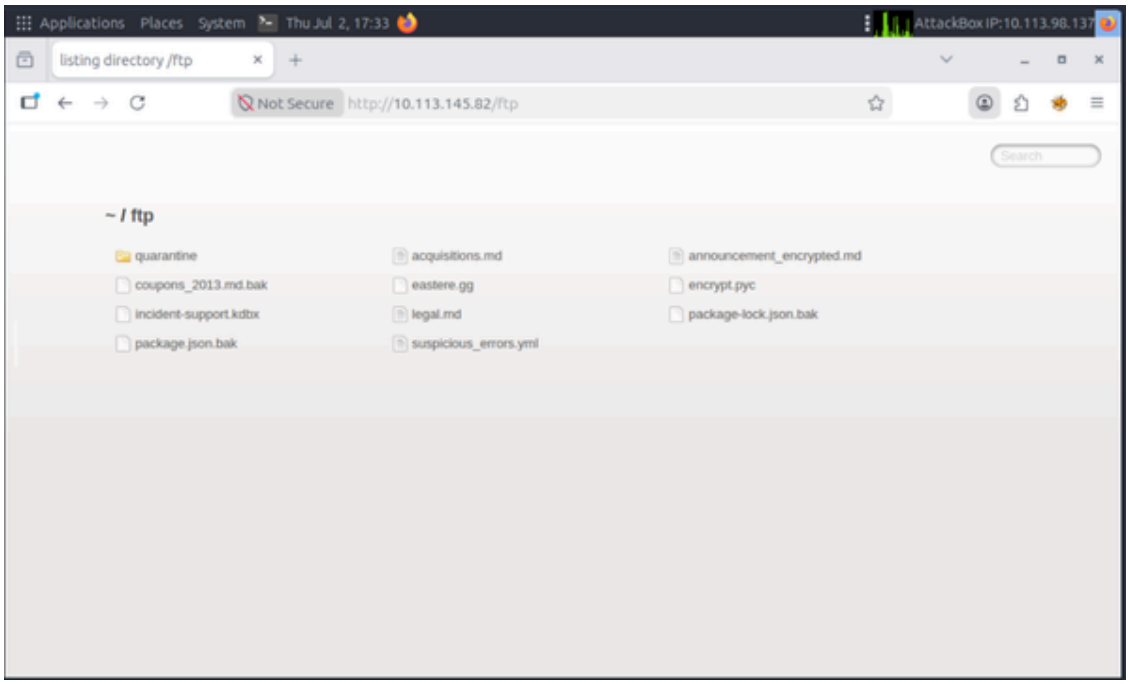
8.1 Description

The web application inadvertently exposes a cryptographic signature file within a publicly accessible web directory (the /ftp folder). Due to missing directory access controls and improper filtering of sensitive file extensions, an unauthenticated attacker can directly request and download this file via its URL path, including bypassing extension-based restrictions with a poison null byte technique. Exposing production signature assets violates secure development baseline standards and compromises the cryptographic integrity of the platform.

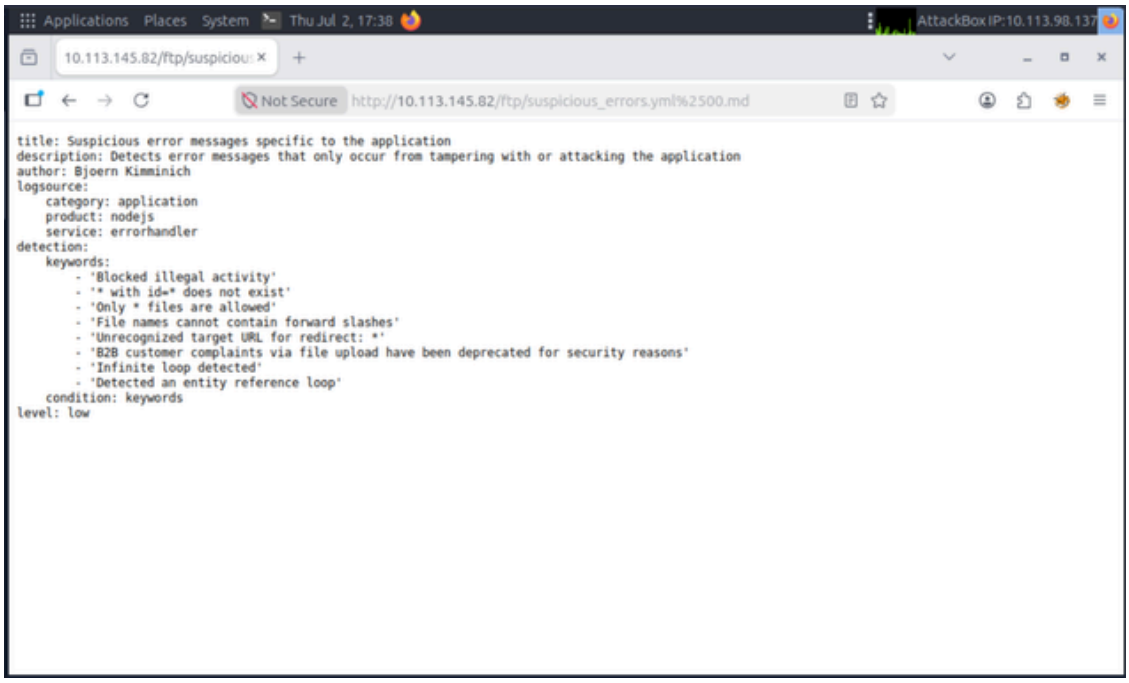
8.2 Proof of Concept

Steps to Reproduce:

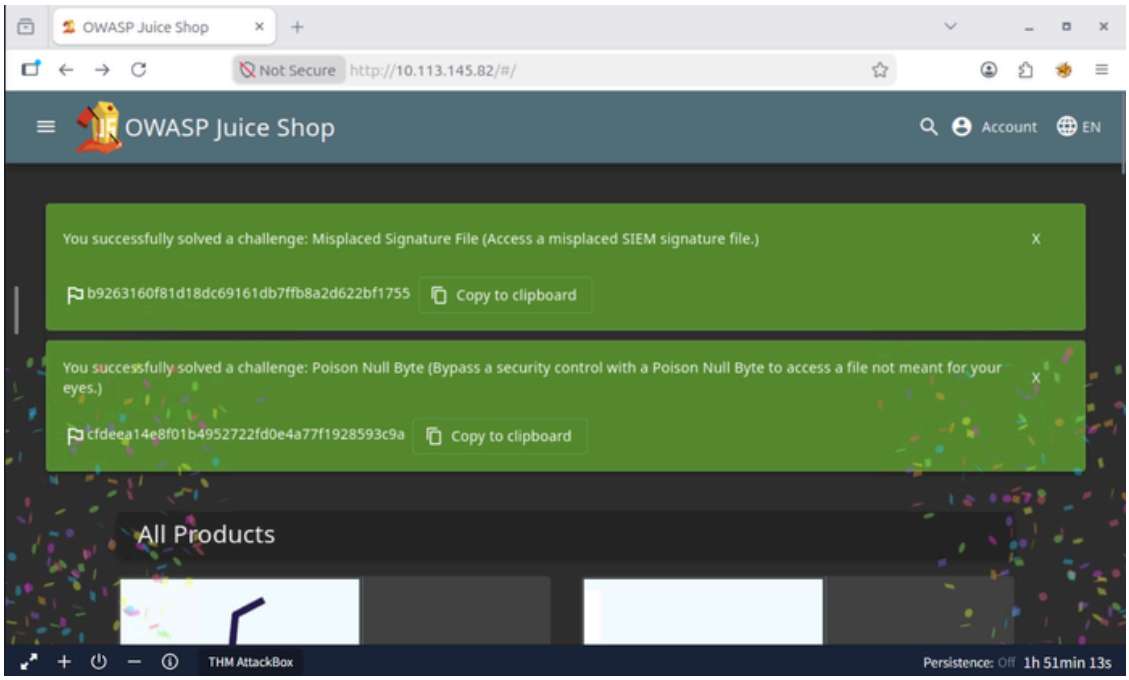
1. Navigate to the target OWASP Juice Shop instance and browse to the exposed public file storage directory by appending /ftp/ to the base URL path.
2. Audit the directory listing and locate a suspicious cryptographic signature asset file (which the server normally restricts from direct downloading due to extension filtering rules).
3. Craft a bypass payload by appending a double URL-encoded null byte followed by an allowed extension (%2500.md) to the end of the target filename URL.
4. Submit the manipulated URL path to the browser address bar (e.g., http://127.0.0.1:42000/ftp/suspicious_file.yml%2500.md).
5. The backend infrastructure interprets the .md extension as a valid white-listed asset type during initial inspection, but truncates the string at the null byte during file system retrieval. This bypasses file-type restrictions and forces a direct download of the sensitive signature document.



The restricted signature asset visible inside the public /ftp/ directory listing.



The poison-null-byte payload (%2500.md) bypassing the backend extension-validation filter and returning the restricted file's contents.



Success confirmation showing both the 'Misplaced Signature File' and 'Poison Null Byte' challenges solved.

8.3 Impact

An unauthenticated attacker can acquire sensitive cryptographic material directly from the production frontend web root. Depending on how the application uses this specific key structure, this asset leak could allow malicious actors to forge digital signatures, manipulate internal system tokens, or bypass secondary encryption boundaries.

8.4 Remediation

- Enforce Strict Directory Access Control Lists (ACLs): Configure the hosting web server to explicitly block public access or directory listing on folders housing internal utility assets (like /ftp).
- Implement Production File Extension Blacklisting: Establish server-side rules that explicitly reject requests targeting sensitive administrative file types (e.g., .key, .pfx, .crt, .env, .bak).
- Relocate Cryptographic Assets Outside Web Root: Ensure all secure signing tokens, key pairs, and signature archives are saved completely outside the public web server deployment context.

8.5 References

- OWASP Top 10 - A02:2021 Cryptographic Failures: https://owasp.org/Top10/A02_2021-Cryptographic_Failures/
- CWE-552: Files Accessible to External Parties: https://cwe.mitre.org/data/definitions/552.html

VUL-006 — SQL Injection Authentication Bypass in login.js

Finding ID	VUL-006
Title	SQL Injection Authentication Bypass in login.js
Severity	**Critical**
CVSS v3.1 Score	9.8
CVSS Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
OWASP Category	A03:2021 - Injection
Affected URL	http://127.0.0.1:42000/rest/user/login
CWE	CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')

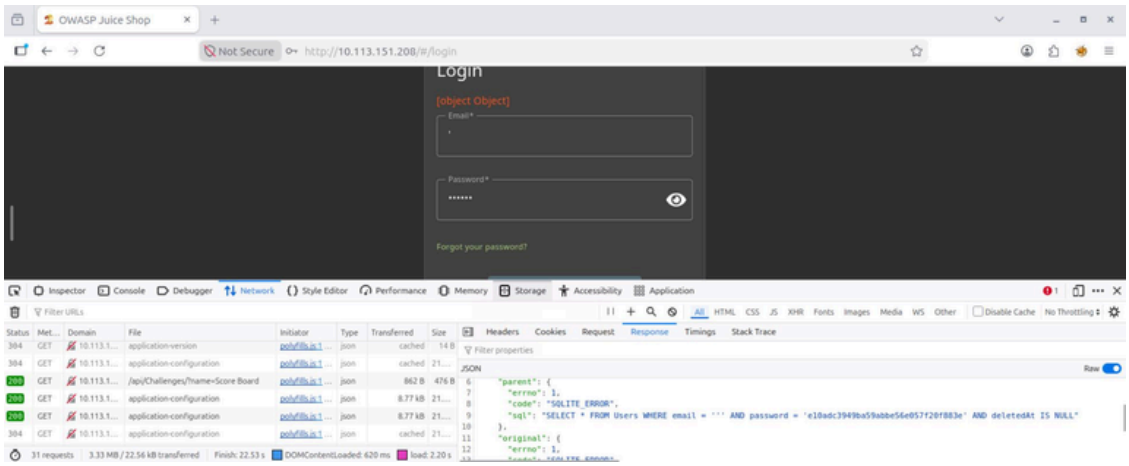
9.1 Description

The web application login interface is vulnerable to SQL Injection via the email input field. User input is directly concatenated into a dynamic SQL query without proper sanitization or parameterization. This allows an unauthenticated attacker to manipulate the query logic, bypass the authentication mechanism entirely, and log into the application as the administrative user without providing a valid password.

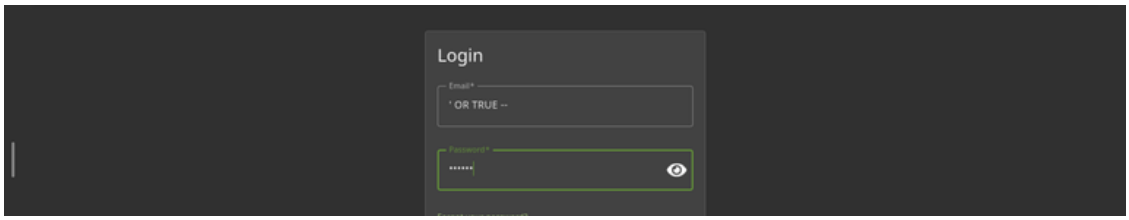
9.2 Proof of Concept

Steps to Reproduce:

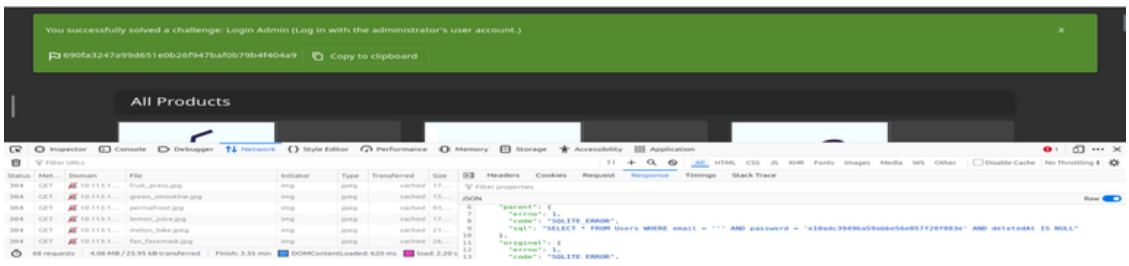
1. Open a browser and navigate to the OWASP Juice Shop login page at http://127.0.0.1:42000/#/login.
2. In the Email input field, enter the SQL payload: admin@juice-sh.op' --
3. Enter any arbitrary value in the Password field (e.g., password123).
4. Click the Log In button.
5. The application processes the payload, comments out the password verification logic in the database backend, and successfully authenticates the attacker as the Admin user.



The injected single quote breaking the SQL query structure, causing a SQLite syntax error visible in the network response — confirming the input is directly concatenated.



A refined SQL Injection payload (' OR TRUE --) applied to the Email field to bypass password authentication entirely.



Successful exploitation banner confirming the 'Login Admin' challenge was solved with full administrator access.

9.3 Impact

An unauthenticated attacker can completely bypass authentication controls and gain full administrative privileges over the application. This allows complete unauthorized access to sensitive user data, modification of application content, and full compromise of administrative functionalities.

9.4 Remediation

- Use Parameterized Queries (Prepared Statements): Implement parameterized queries using bind variables instead of constructing dynamic SQL queries via string concatenation in routes/login.js.

- Utilize Object-Relational Mapping (ORM): Rely on built-in safe methods provided by the ORM (such as Sequelize's User.findOne) which inherently treat user inputs as parameters rather than executable code.
- Input Validation: Enforce strict server-side validation on the email input field to ensure it conforms to a standard email format before it is sent to the backend.

9.5 References

- OWASP SQL Injection: https://owasp.org/www-community/attacks/SQL_Injection
- CWE-89: SQL Injection: https://cwe.mitre.org/data/definitions/89.html

VUL-007 — Missing Function-Level Access Control (Admin Section) in main.js

Finding ID	VUL-007
Title	Missing Function-Level Access Control (Admin Section) in main.js
Severity	**High**
CVSS v3.1 Score	8.1
CVSS Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N
OWASP Category	A01:2021 - Broken Access Control
Affected URL	http://127.0.0.1:42000/#/administration
CWE	CWE-285: Improper Authorization / CWE-306: Missing Authentication for Sensitive Function

10.1 Description

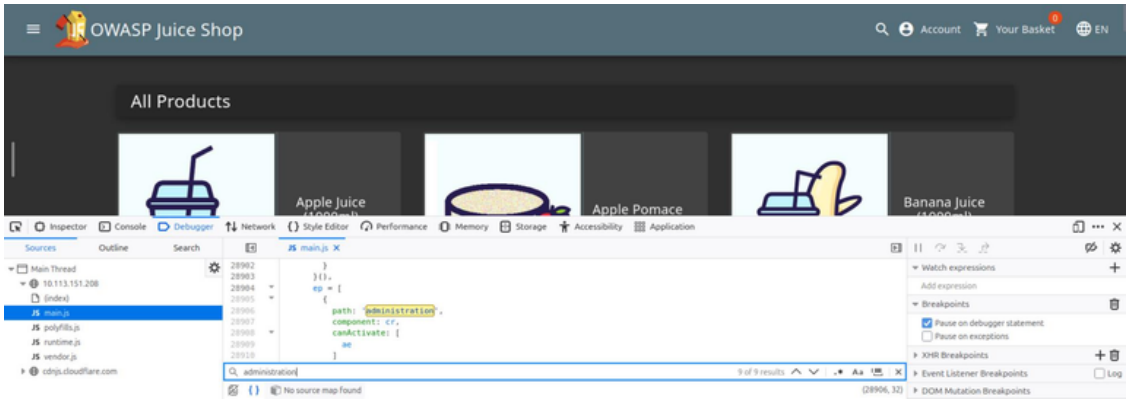
The web application fails to enforce proper server-side authorization checks on administrative UI layouts and corresponding backend functions. Although the link to the sensitive "Admin Section" is intentionally hidden from regular client views, the underlying route remains statically compiled into the client-side JavaScript bundle (main.js). An unauthorized user can analyze the source code mapping, locate the hidden path, and explicitly navigate to the endpoint via direct URL manipulation, completely bypassing UI navigation restrictions.

10.2 Proof of Concept

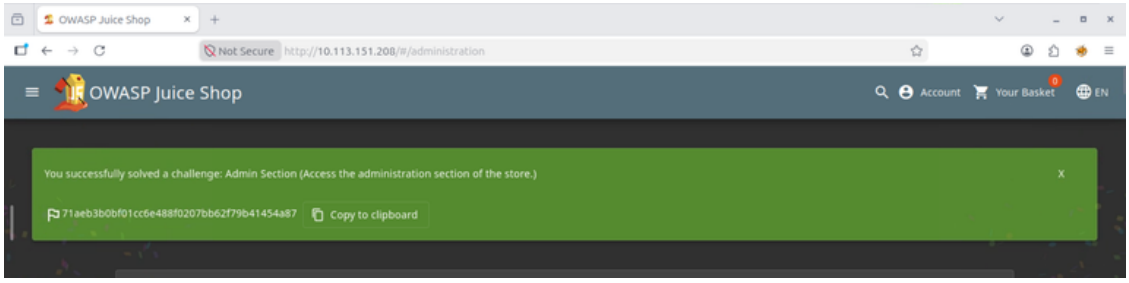
Steps to Reproduce:

1. Open a web browser and load the target deployment.
2. Access the browser's built-in Developer Tools (F12) and inspect the client-side JavaScript source assets (specifically main.js).
3. Search the client routing configurations to identify unlinked paths, locating the entry: path: 'administration'.
4. In the browser address bar, perform direct force browsing by appending the path to the application base: /#/administration
5. Press Enter. The client-side single-page router maps the component view, executing access to the internal administration area and revealing user registries, support reviews, and feedback control parameters without

requiring prior administrative login.



Direct URL access (force browsing): appending /#/administration to the application URL loads the private admin section layout without proper role validation, visible in the JavaScript source route table.



Successful exploitation notification confirming the 'Admin Section' challenge was solved after forcing direct access to the administrative path.

10.3 Impact

An unauthenticated attacker or low-privileged user can bypass standard routing flows and gain unauthorized entry into administrative panels. This breaks confidentiality standards by exposing user registration listings, structural application records, internal communications, and analytical controls to unauthorized parties.

10.4 Remediation

- Enforce Server-Side Access Controls: Implement stateful authorization directly within the server-side API routes handling sensitive administration collections rather than relying on front-end component hiding.
- Role-Based Router Guards: Implement centralized middleware to intercept request paths, mapping incoming sessions or identity tokens against strict role definitions prior to returning sensitive administrative data.
- Principle of Least Privilege: Restrict data transmissions so that critical management tables or sensitive records are structurally unavailable unless authenticated by verified server-side admin tokens.

10.5 References

- OWASP Broken Access Control: https://owasp.org/Top10/A01_2021-Broken_Access_Control/
- CWE-285: Improper Authorization: https://cwe.mitre.org/data/definitions/285.html
- CWE-306: Missing Authentication for Sensitive Function: https://cwe.mitre.org/data/definitions/306.html

VUL-008 — Weak Password Policy (Broken Authentication) in User Registration & Update

Finding ID	VUL-008
Title	Weak Password Policy (Broken Authentication) in User Registration & Update
Severity	**High**
CVSS v3.1 Score	7.5
CVSS Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
OWASP Category	A07:2021 - Identification and Authentication Failures
Affected URL	http://127.0.0.1:42000/#/register
CWE	CWE-521: Weak Password Requirements

11.1 Description

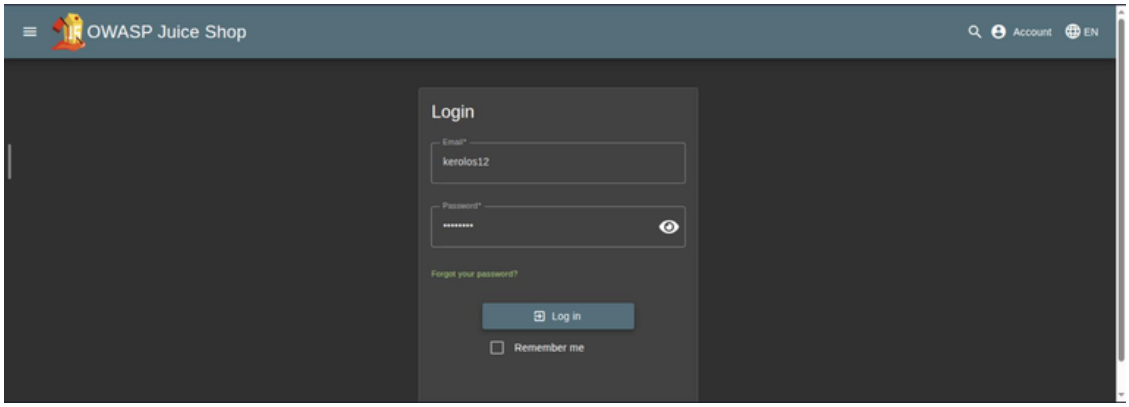
The web application exhibits a Broken Authentication vulnerability due to the absence of a robust password strength verification policy during user registration and password updates. The backend registration components fail to reject easily guessable or commonly leaked passwords (such as 'admin123' or 'password'). This permits users and administrators to establish highly insecure credentials, rendering the application susceptible to automated dictionary attacks, credential stuffing, and brute-force takeover campaigns.

11.2 Proof of Concept

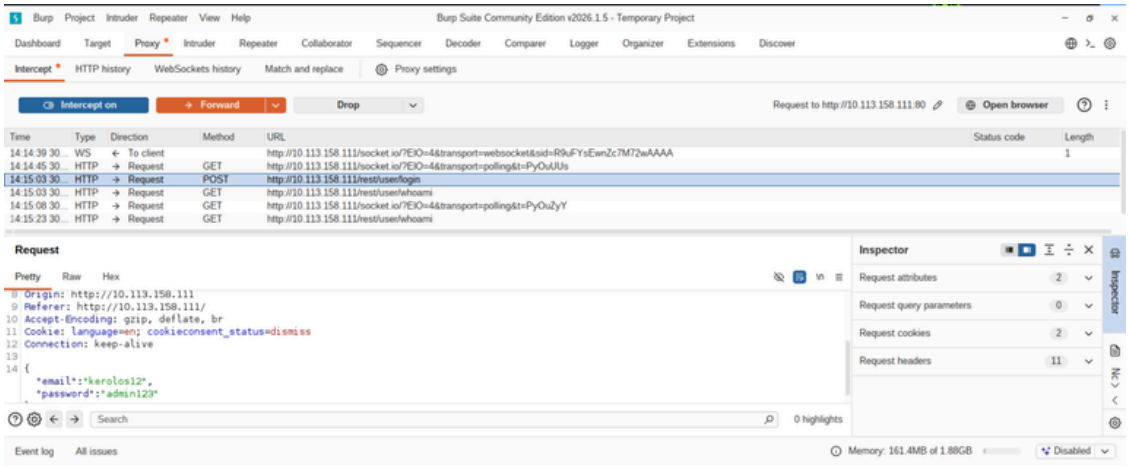
Steps to Reproduce:

1. Navigate to the OWASP Juice Shop registration endpoint at http://127.0.0.1:42000/#/register.
2. Fill out the registration form with a valid email address and security question.
3. In the Password field, type an extremely weak and common password string, such as admin123 or 123456.
4. Observe that the front-end validation and backend registration routers accept the input without producing any error or complexity constraint.

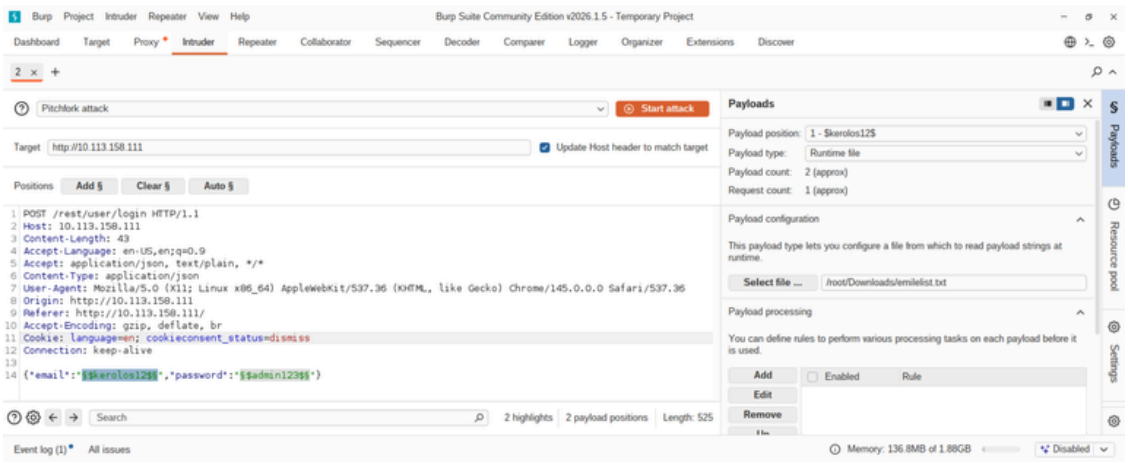
5. Click Register. The application successfully commits the user record, confirming the lack of password strength enforcement.
6. Intercept the login request via Burp Suite to confirm the weak credential is transmitted and accepted, then use a Pitchfork intruder attack (paired email/password wordlists) to brute-force the administrator account with weak, common passwords.



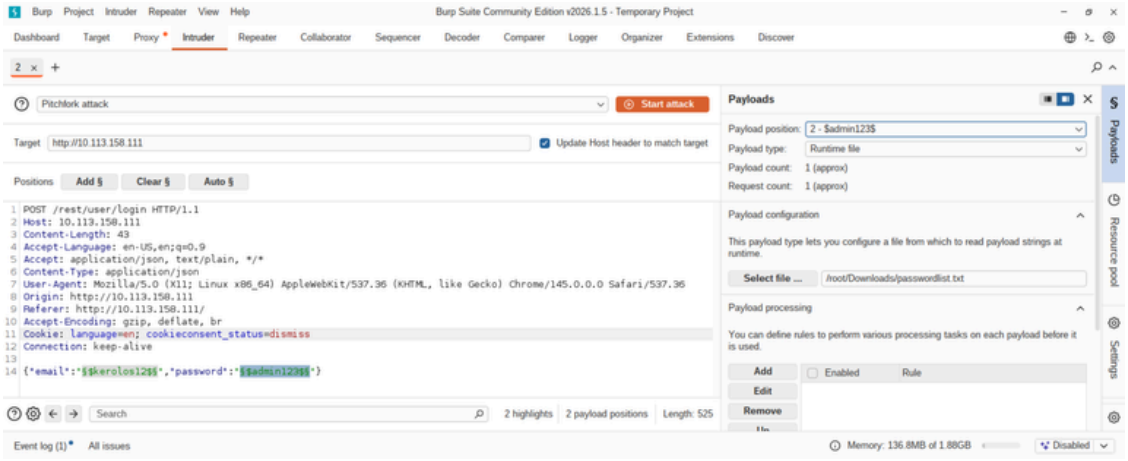
Entering an arbitrary, low-complexity password during user registration to test backend validation constraints.



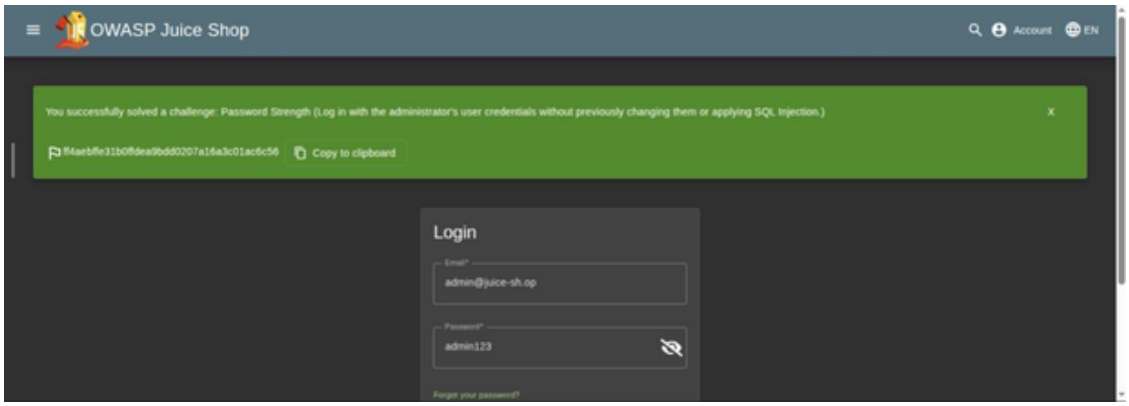
Intercepting the HTTP POST login request via Burp Suite, revealing transmission of the low-complexity password.



Configuring a Burp Suite Intruder Pitchfork attack with an email wordlist as the first payload position.



Configuring the paired password wordlist as the second payload position for the same attack.



Successful exploitation: logging into the administrator account with weak, guessed credentials, confirming the 'Password Strength' challenge as solved.

11.3 Impact

Malicious actors can deploy automated tools to conduct dictionary attacks against targeted user accounts, especially high-value accounts like administrators. Successful exploitation leads to complete unauthorized account takeover, undermining data integrity, user privacy, and systemic access controls across the deployment.

11.4 Remediation

- Enforce Password Complexity Policies: Implement strict server-side validation to enforce minimum length (e.g., at least 10 characters) and mandatory inclusion of uppercase, lowercase, numeric, and special characters.
- Integrate Password Strength Estimators: Utilize established strength libraries, such as zxcvbn, to actively evaluate password entropy during submission and block low-entropy configurations.
- Implement Leaked Password Checking: Cross-reference password registration attempts against known compromised password repositories (e.g., the HaveIBeenPwned API) to prohibit the use of recycled credentials.

11.5 References

- OWASP Identification and Authentication Failures: https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/
- CWE-521: Weak Password Requirements: https://cwe.mitre.org/data/definitions/521.html

VUL-009 — Broken Access Control — Insecure Direct Object References (IDOR) in View Basket

Finding ID	VUL-009
Title	Broken Access Control — Insecure Direct Object References (IDOR) in View Basket
Severity	**High**
CVSS v3.1 Score	8.5
CVSS Vector	AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:L/A:N
OWASP Category	A01:2021 - Broken Access Control
Affected URL	http://127.0.0.1:42000/rest/basket/
CWE	CWE-639: Authorization Bypass Through User-Controlled Key

12.1 Description

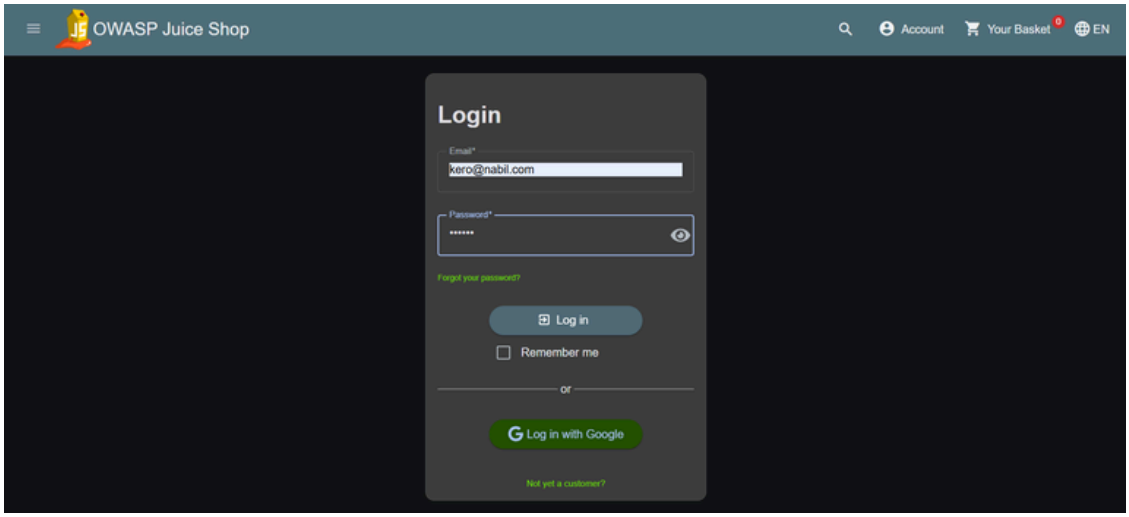
The web application suffers from an Insecure Direct Object Reference (IDOR) vulnerability within its shopping basket management. When a user navigates to their shopping basket, the application requests the basket details from the backend REST API using a sequential integer identifier (e.g., /rest/basket/1). The server fails to validate whether the authenticated session owns the requested basket ID, so an attacker can manipulate this parameter to view or compromise the shopping baskets of other registered users.

12.2 Proof of Concept

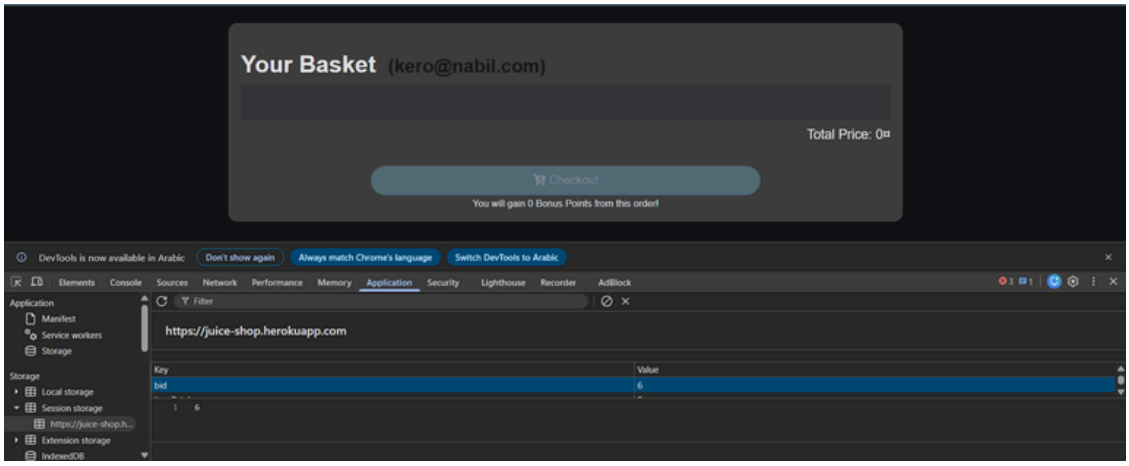
Steps to Reproduce:

1. Log into a standard user account on the OWASP Juice Shop platform.
2. Access the browser's Developer Tools (F12) and navigate to the Application tab (Session Storage).
3. Click on the Your Basket link to execute the application's client-side basket loading routine.
4. Locate the session key storing the basket identifier (bid) and note its value.
5. Modify the stored basket ID value inside Session Storage to reference a different, arbitrary integer index belonging to another account.

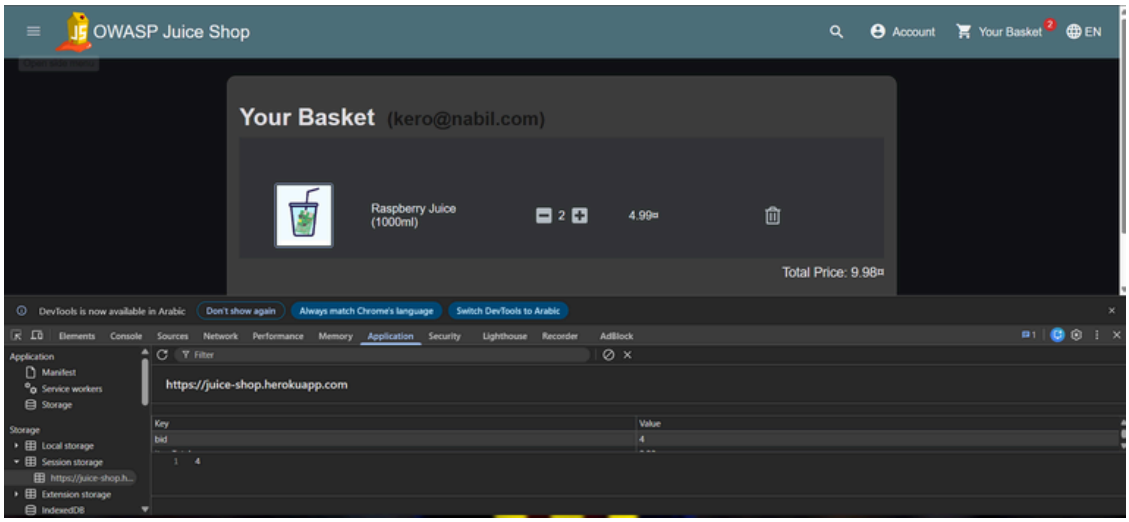
6. Observe that the page dynamically re-renders and successfully extracts private transaction metadata, product lists, and quantities from the target account's basket, without any authorization check.



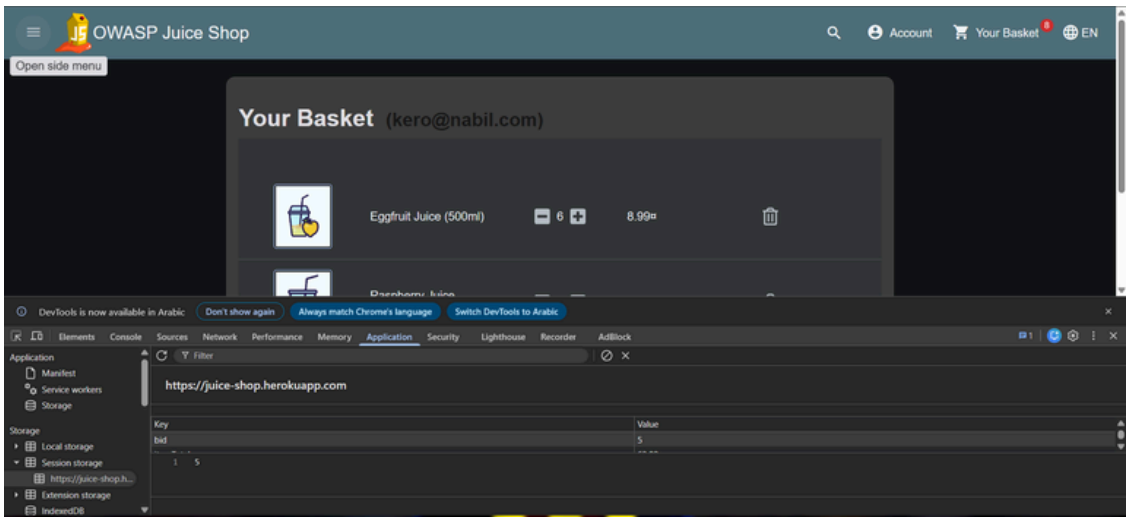
The user logs into their own account, initiating the session.



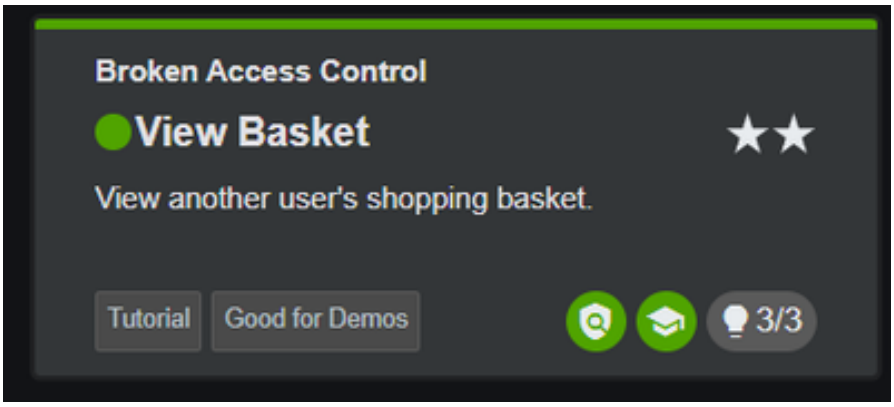
The user's own shopping basket loads, with the basket ID visible in Session Storage in the DevTools Application panel.



The basket contents update after items are added, with the basket ID value tracked in Session Storage.



Manually editing the Session Storage basket ID value to point at another user's basket.



The 'Broken Access Control - View Basket' challenge confirmed as solved after successfully viewing another user's basket.

12.3 Impact

An attacker can systematically enumerate and view the private shopping baskets of all users across the deployment. This exposes user asset interactions and order metadata, and in more severe scopes could allow tampering with or deletion of orders belonging to other accounts.

12.4 Remediation

- Implement Indirect Object References: Avoid exposing raw, sequential database identifiers directly in API endpoints. Map parameters to temporary, session-scoped randomized tokens.
- Enforce Rigorous Object-Level Access Control: Ensure backend validation explicitly verifies that the authenticated session (Session ID or JWT) matches the owner of the requested resource before serving data.
- Adopt Explicit Deny Policies: Configure API access to drop any data-retrieval request by default unless the requester's permissions are fully authenticated and verified.

12.5 References

- OWASP Broken Access Control: https://owasp.org/Top10/A01_2021-Broken_Access_Control/
- CWE-639: Authorization Bypass Through User-Controlled Key: https://cwe.mitre.org/data/definitions/639.html

VUL-010 — Security Misconfiguration — Deprecated Interface Access

Finding ID	VUL-010
Title	Security Misconfiguration — Deprecated Interface Access
Severity	**Medium**
CVSS v3.1 Score	5.3
CVSS Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
OWASP Category	A05:2021 - Security Misconfiguration
Affected URL	http://127.0.0.1:42000/ftp/
CWE	CWE-1059: Improper Vulnerability Management (Deprecated/Outdated Feature)

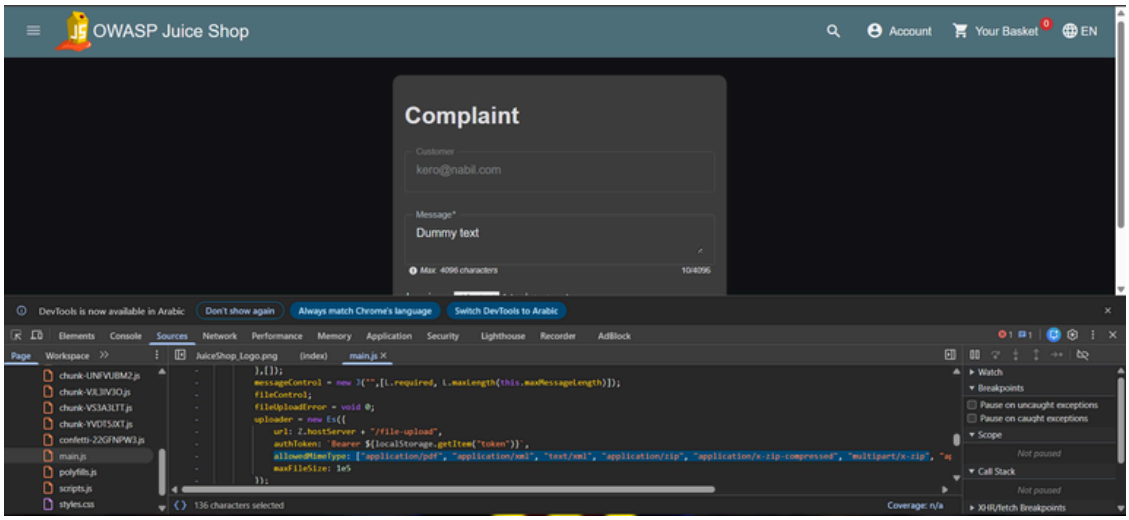
13.1 Description

The application suffers from a Security Misconfiguration vulnerability due to an active, production-accessible deprecated interface. During platform updates, a legacy B2B customer file-upload interface and its associated storage directory were left unpatched and reachable, rather than being fully decommissioned. Because the backend fails to isolate or remove this outdated component, unauthenticated users can discover and interact with it.

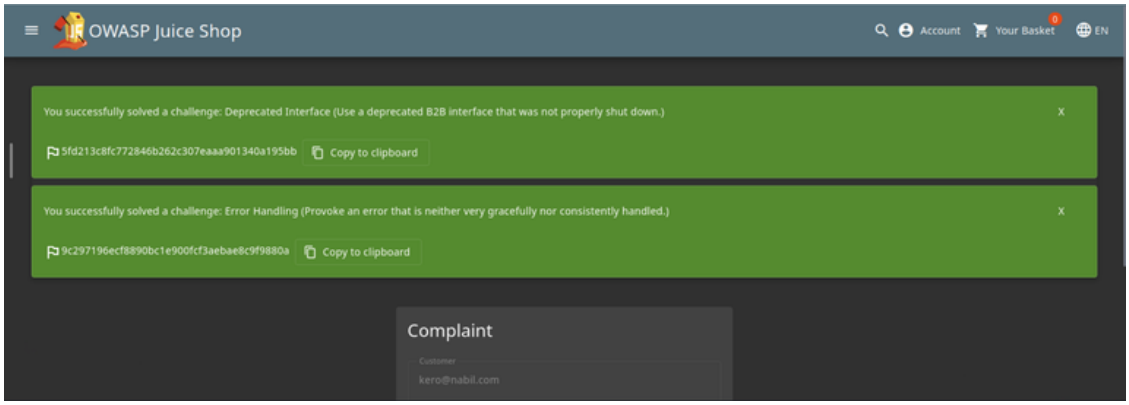
13.2 Proof of Concept

Steps to Reproduce:

1. Authenticate into the application with valid user credentials.
2. Navigate directly to the Complaint submission interface from the navigation menu.
3. Inspect the file attachment mechanism and note that it permits uploads in legacy/deprecated formats not used by the current application version (e.g., XML).
4. Submit a complaint with an attachment using the deprecated interface and format.
5. The application accepts and processes the request through the legacy, no-longer-maintained code path, confirming that the deprecated B2B interface was never properly shut down.



The Complaint submission interface, inspected via DevTools to identify the file-upload handler and allowed MIME types.



Success banner confirming the 'Deprecated Interface' and related 'Error Handling' challenges were solved via the legacy upload path.

13.3 Impact

An unauthenticated attacker can explore abandoned functionality that is no longer actively maintained or monitored. This can leak internal developer specifications, business logic patterns, or configuration parameters, lowering the barrier for constructing further, more critical attacks (such as injection via the legacy upload path).

13.4 Remediation

- Complete Interface Decommissioning: Physically remove all outdated functions, development configurations, and unlinked routing files from the active production deployment.
- Tighten Route Access Controls: Implement server-side access rules so that administrative or outdated directory targets deny connections by default.
- Perform Continuous Configuration Audits: Set up regular automated scanning to discover and isolate orphaned testing artifacts, exposed development routes, or loose file configurations before staging.

13.5 References

- OWASP Security Misconfiguration: https://owasp.org/Top10/A05_2021-Security_Misconfiguration/
- CWE-1059: Improper Vulnerability Management: https://cwe.mitre.org/data/definitions/1059.html

VUL-011 — Reflected Cross-Site Scripting (XSS) in Search Functionality

Finding ID	VUL-011
Title	Reflected Cross-Site Scripting (XSS) in Search Functionality
Severity	**High**
CVSS v3.1 Score	6.1
CVSS Vector	AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N
OWASP Category	A03:2021 - Injection
Affected URL	http://127.0.0.1:42000/#/search?q=<payload>
CWE	CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')

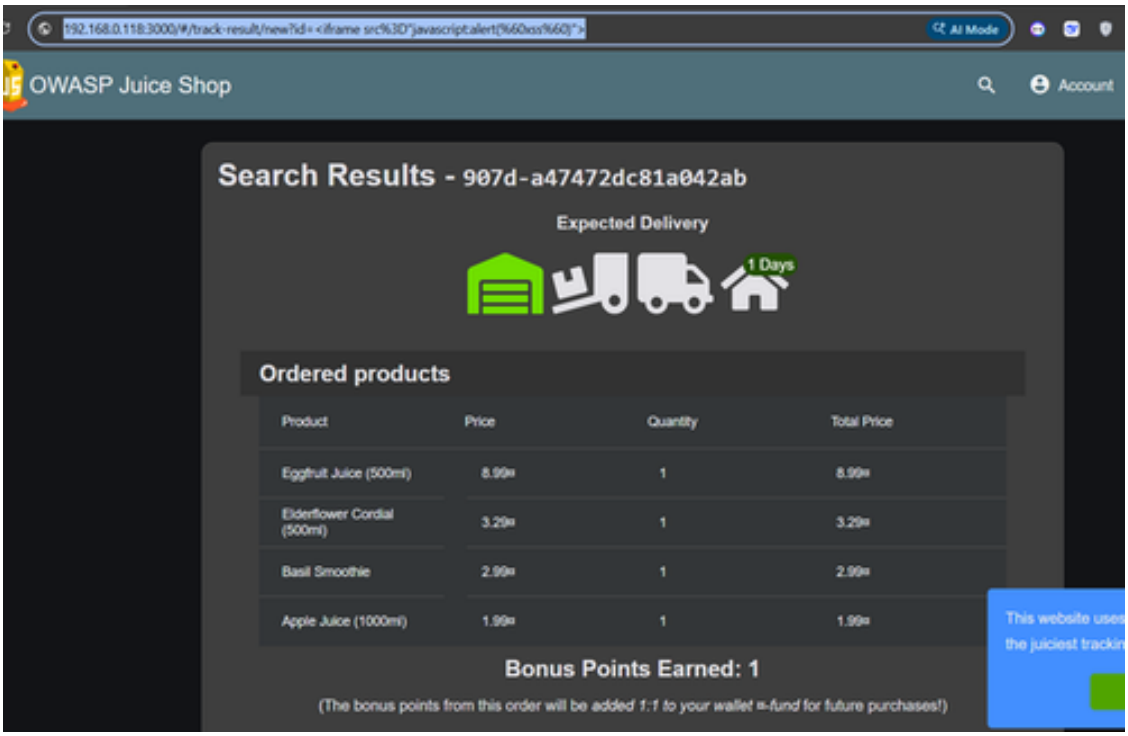
14.1 Description

The search functionality of the application is vulnerable to Reflected Cross-Site Scripting (XSS). User-supplied input passed via the q query parameter is rendered back into the page's HTML/DOM without proper output encoding or sanitization. An attacker can craft a malicious URL that, when visited by a victim, executes arbitrary JavaScript in the context of the victim's browser session. Because the payload is reflected in the response rather than being stored, exploitation typically requires the victim to click a crafted link.

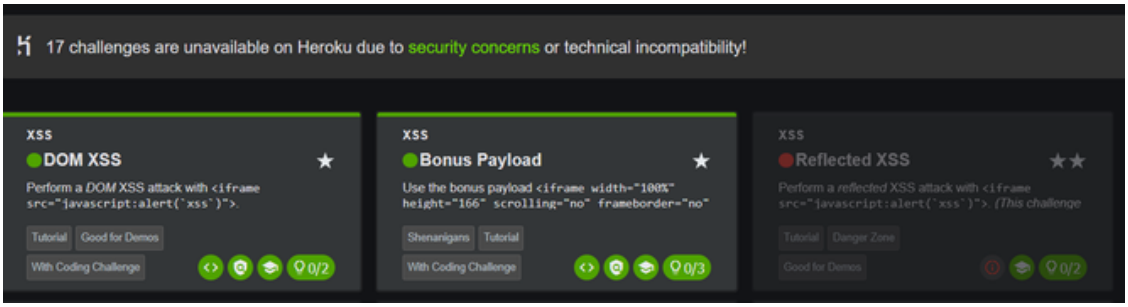
14.2 Proof of Concept

Steps to Reproduce:

1. Open a browser and navigate to the OWASP Juice Shop application at http://127.0.0.1:42000/#/search.
2. Locate the search input field (magnifying glass icon) in the top navigation bar.
3. Enter the XSS payload into the search field: <iframe src="javascript:alert(`xss`)">
4. Press Enter to submit the search query.
5. The application reflects the unsanitized input directly into the DOM. The injected iframe element executes, triggering the JavaScript alert() and confirming that arbitrary script execution is possible in the context of the application's origin.



The crafted payload entered into the search bar and reflected directly into the page's URL/DOM without sanitization.



Execution of the injected script confirms attacker-controlled input is rendered and executed as active content, and the corresponding challenge is marked solved.

14.3 Impact

An attacker can craft a malicious link containing a JavaScript payload and trick a victim into clicking it (e.g., via phishing). Upon execution, the attacker may hijack the victim's session by stealing authentication tokens or cookies, perform actions on the victim's behalf, redirect the victim to a malicious site, or harvest sensitive information. Because the script executes within the application's trusted origin, it bypasses the Same-Origin Policy protections normally afforded to the user.

14.4 Remediation

- Contextual Output Encoding: Encode all user-supplied data before rendering it in HTML, JavaScript, URL, or attribute contexts (e.g., HTML-entity encode <, >, ", ', and & before reflecting the search term back to the page).

- Use Framework-Safe Rendering: Rely on the front-end framework's built-in safe binding mechanisms (e.g., Angular's default sanitization) rather than directly injecting raw strings into the DOM.
- Input Validation: Apply allow-list based server-side validation on the search parameter to reject or strip characters not expected in a legitimate search term.
- Content Security Policy (CSP): Deploy a strict CSP header to limit the impact of any residual XSS by restricting inline script execution and script sources.

14.5 References

- OWASP Cross-Site Scripting (XSS): https://owasp.org/www-community/attacks/xss/
- CWE-79: Improper Neutralization of Input: https://cwe.mitre.org/data/definitions/79.html

VUL-012 — Improper Input Validation — Zero Stars (Customer Feedback Rating)

Finding ID	VUL-012
Title	Improper Input Validation — Zero Stars (Customer Feedback Rating)
Severity	**Medium**
CVSS v3.1 Score	5.3
CVSS Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:L/A:N
OWASP Category	A04:2021 - Insecure Design
Affected URL	http://127.0.0.1:42000/#/contact
CWE	CWE-20: Improper Input Validation

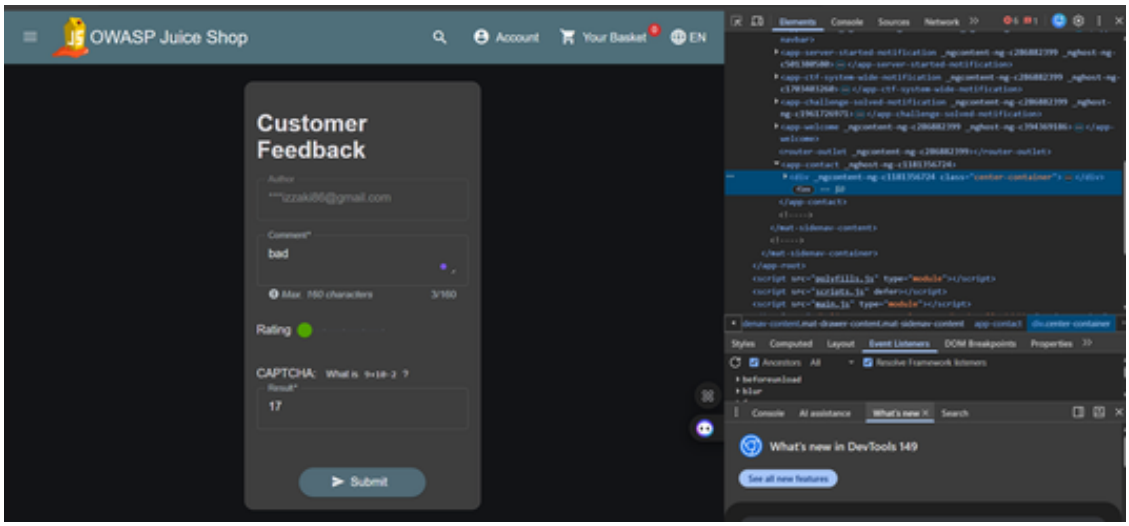
15.1 Description

The Customer Feedback form limits the star rating to a range of 1-5 using a UI slider, and this constraint is enforced only in the browser. The backend API that processes submitted feedback does not re-check that the rating value falls within this range, allowing a rating of 0 (or any other out-of-range number) to be accepted and stored, because the server trusts that the request originated from the legitimate front end.

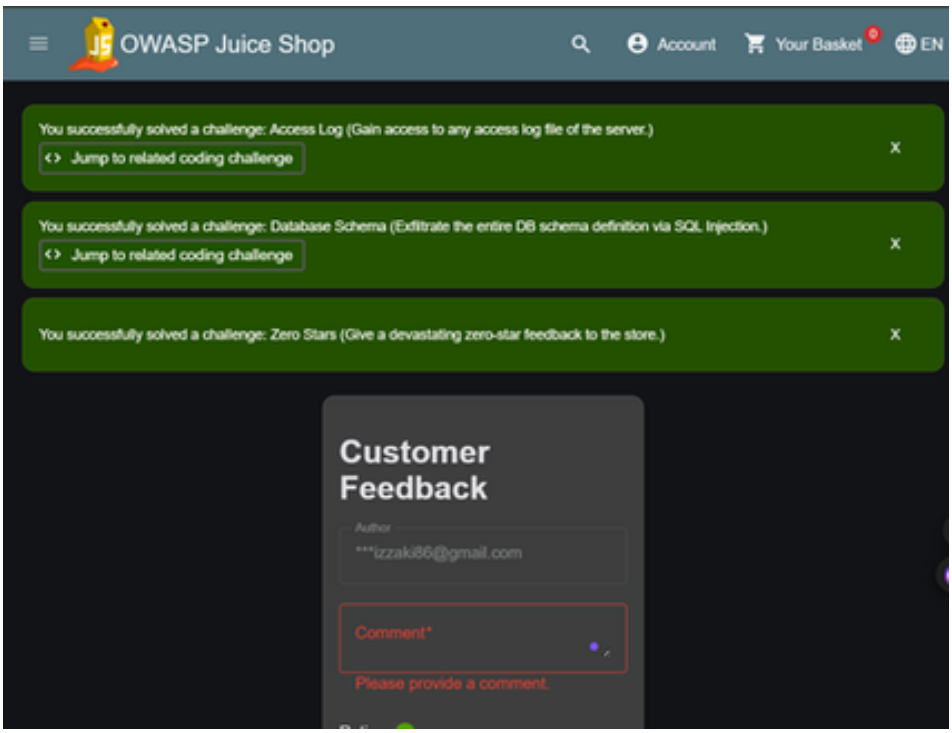
15.2 Proof of Concept

Steps to Reproduce:

1. Open the OWASP Juice Shop application and navigate to the Customer Feedback page.
2. Fill in the feedback form (comment and a valid star rating between 1 and 5).
3. Intercept the outgoing POST request to /api/Feedbacks/ using a proxy tool such as Burp Suite.
4. In the intercepted request body, change the "rating" field value to 0: { "comment": "Terrible service.", "rating": 0, "captchaId": ... }
5. Forward the modified request. The server accepts and stores the zero-star rating without rejecting it, confirming the 'Zero Stars' challenge as solved.



The customer feedback form inspected using browser developer tools, allowing modification of client-side parameters such as the rating value.



Successful exploitation of the improper input validation vulnerability: the application accepts an out-of-range rating (0 stars) and confirms the 'Zero Stars' challenge as solved.

15.3 Impact

Because the rating boundaries are not enforced server-side, an attacker (or automated script) can submit out-of-range values repeatedly to skew the store's average rating, damaging its perceived reputation. The same flaw could be used to insert unexpected values into any statistic derived from this field.

15.4 Remediation

- Server-Side Validation: Re-validate every constraint on the backend; never rely on front-end checks alone. The rating field should reject any value outside 1-5.
- Centralize Validation Rules: Define input rules (allowed ranges, formats) once on the server and reuse them for every request path, so the UI and API can never fall out of sync.
- Return Clear Errors: When a request fails validation, respond with an explicit 4xx error and a descriptive message instead of silently accepting the data.
- Schema-Based Validation: Use a request-validation library (e.g., JSON Schema, class-validator) to declaratively enforce field types and ranges.

15.5 References

- OWASP Improper Data Validation: https://owasp.org/www-community/vulnerabilities/Improper_Data_Validation
- CWE-20: Improper Input Validation: https://cwe.mitre.org/data/definitions/20.html

VUL-013 — Improper Input Validation — Repetitive Registration

Finding ID	VUL-013
Title	Improper Input Validation — Repetitive Registration
Severity	**Medium**
CVSS v3.1 Score	5.3
CVSS Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:L/A:N
OWASP Category	A04:2021 - Insecure Design
Affected URL	http://127.0.0.1:42000/#/register
CWE	CWE-20: Improper Input Validation

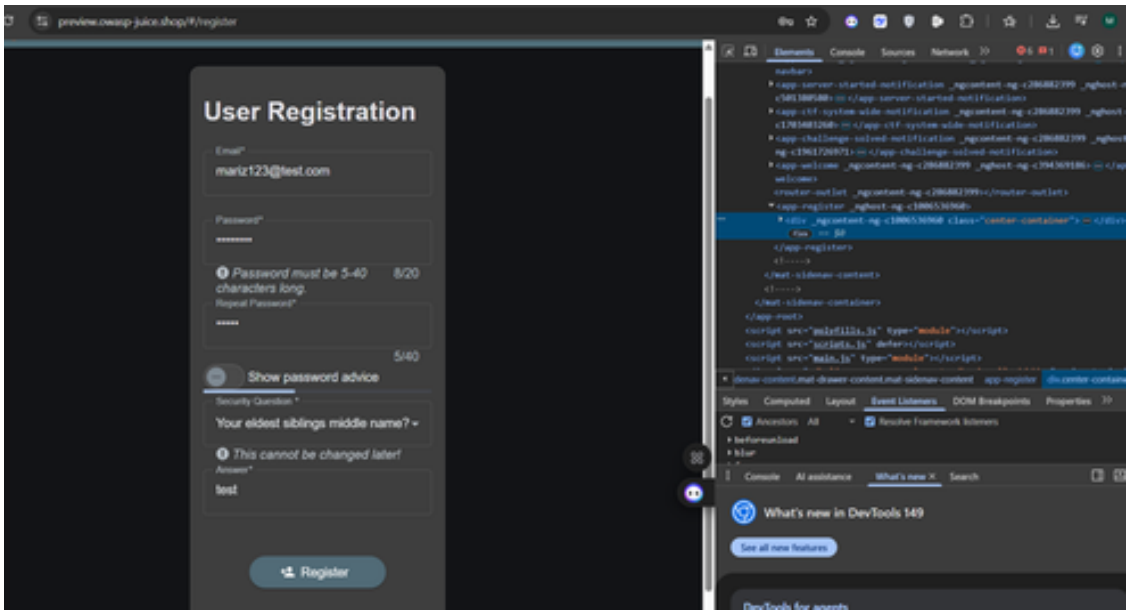
16.1 Description

The registration form requires the "Password" and "Repeat Password" fields to match, and this is enforced only in the browser. The server does not repeat this comparison, so a request can be modified to submit a mismatched or empty confirmation field and the account is still created successfully — the same root cause as VUL-012, but on the user-registration endpoint rather than the feedback endpoint.

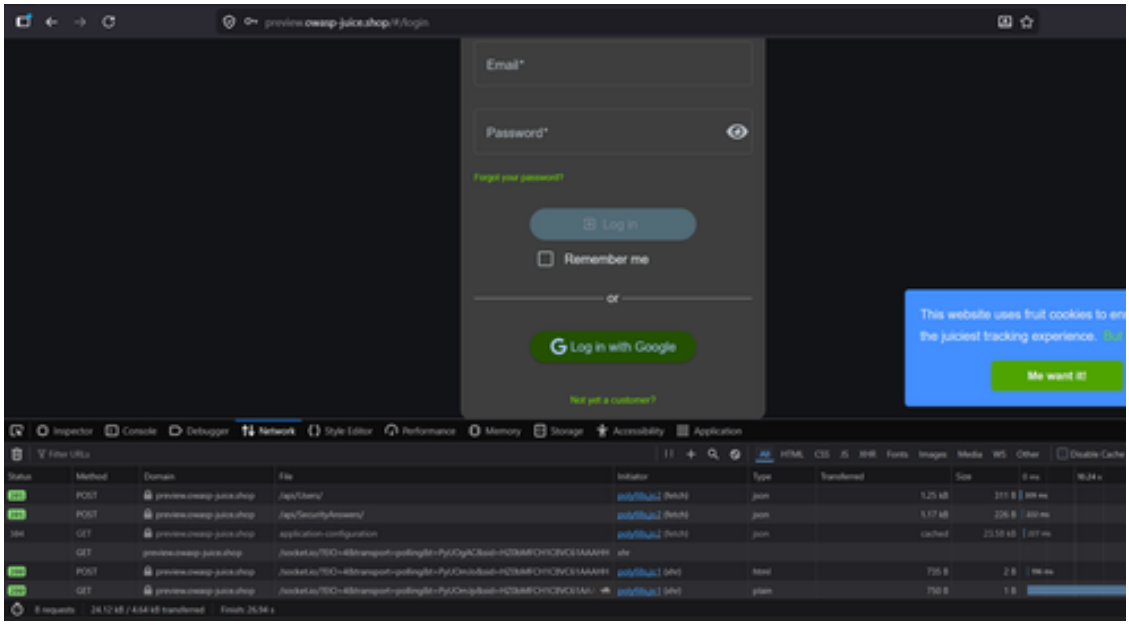
16.2 Proof of Concept

Steps to Reproduce:

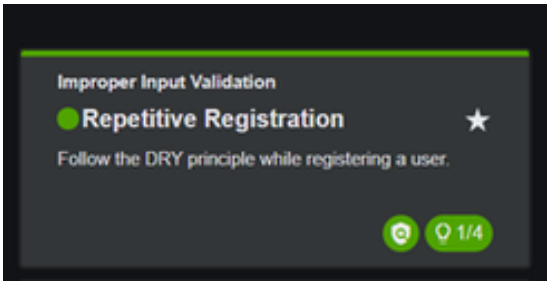
1. Navigate to the registration page and begin creating a new account.
2. Fill in the email and password fields, entering a different value in "Repeat Password" (or leaving it empty).
3. Intercept the outgoing POST request to /api/Users/ before it is blocked by the front end.
4. Edit the "passwordRepeat" field so it no longer matches "password", or remove its value entirely.
5. Forward the request. The account is created successfully even though the confirmation check was never satisfied server-side.



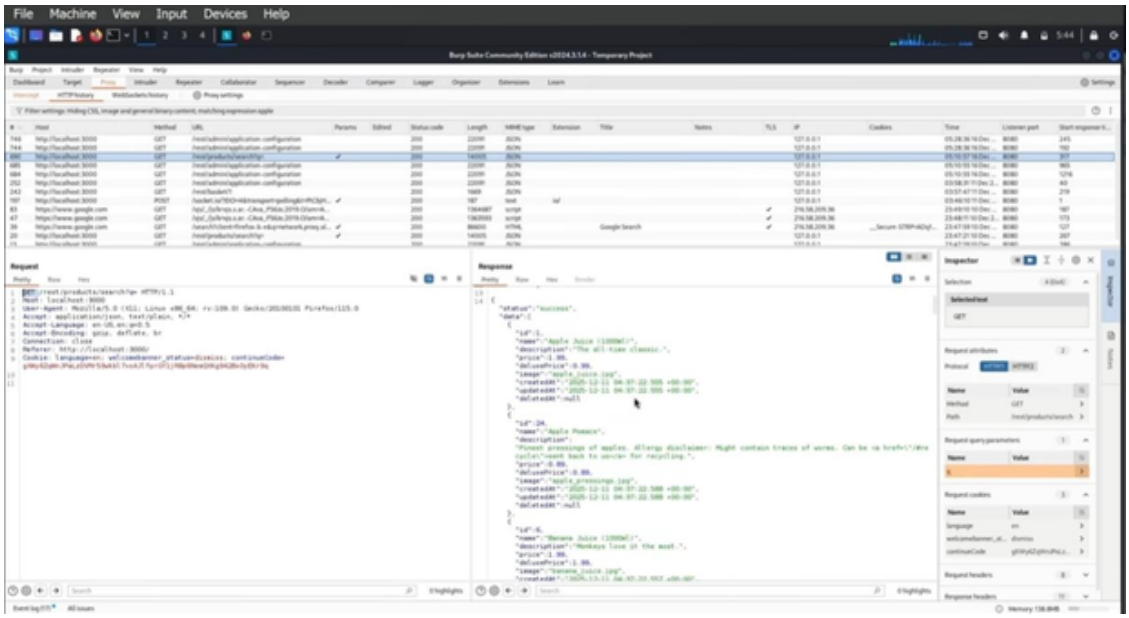
The user registration form analyzed using browser developer tools, enabling manipulation of client-side input fields.



Network traffic captured during the registration flow, showing the POST requests to /api/Users/ and /api/SecurityAnswers/.



Successful API exploitation, confirming a bypass of the client-side password-confirmation control and completion of the 'Repetitive Registration' challenge.



A repeat submission of the same registration request via the network panel, confirming the server performs no server-side duplicate/consistency check on the registration data.

16.3 Impact

Bypassing the password-confirmation check undermines a safeguard meant to prevent users from accidentally locking themselves out due to a typo. At a larger scale, missing server-side validation on registration is a warning sign that other checks on the same endpoint (such as email format or duplicate-account checks) may be similarly bypassable, which can facilitate account spam or automated bulk registration.

16.4 Remediation

- Server-Side Validation: The registration endpoint should confirm that "password" and "passwordRepeat" are identical before creating the account, independent of any client-side check.
- Centralize Validation Rules: Define input rules once on the server and reuse them for every request path.
- Automated Regression Tests: Add tests that submit mismatched values directly to the API, bypassing the UI, to confirm the server rejects them.

16.5 References

- OWASP Improper Data Validation: https://owasp.org/www-community/vulnerabilities/Improper_Data_Validation
- CWE-20: Improper Input Validation: https://cwe.mitre.org/data/definitions/20.html

VUL-014 — Unvalidated Redirects and Forwards (Outdated URL Allowlist)

Finding ID	VUL-014
Title	Unvalidated Redirects and Forwards (Outdated URL Allowlist)
Severity	**Medium**
CVSS v3.1 Score	6.1
CVSS Vector	AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N
OWASP Category	A08:2021 - Software and Data Integrity Failures
Affected URL	http://127.0.0.1:42000/redirect
CWE	CWE-601: URL Redirection to Untrusted Site ('Open Redirect')

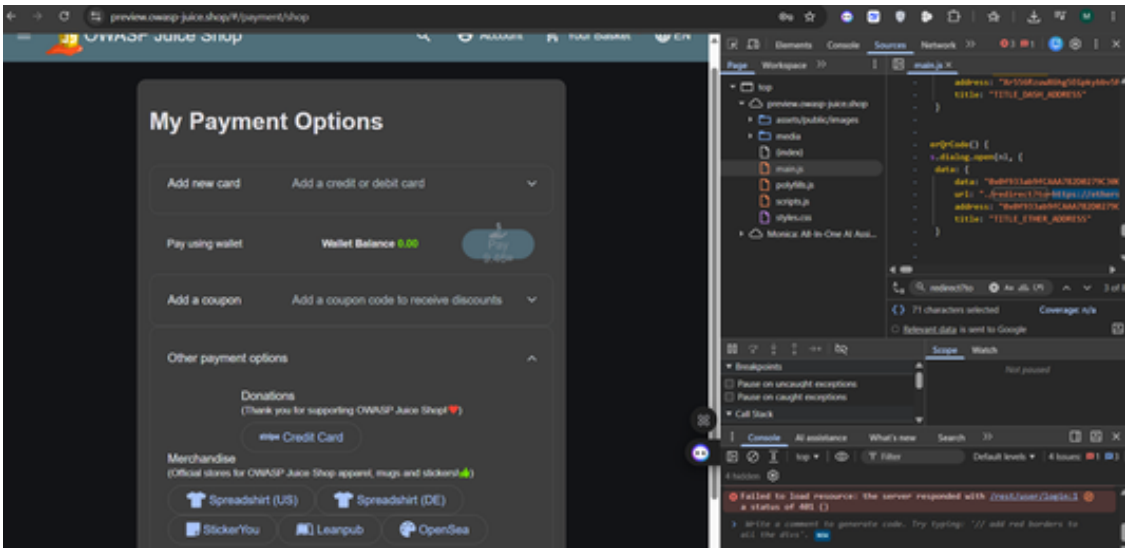
17.1 Description

The web application features a redirection mechanism that forwards users to external links based on a 'to' parameter. While the application attempts to validate these URLs against an allowlist, the allowlist is outdated or poorly configured (containing legacy domains that have expired or been re-registered by third parties). An attacker can exploit this flawed validation logic to craft a link that passes the application's checks but ultimately redirects the victim to an external, malicious website — a technique commonly used in phishing campaigns.

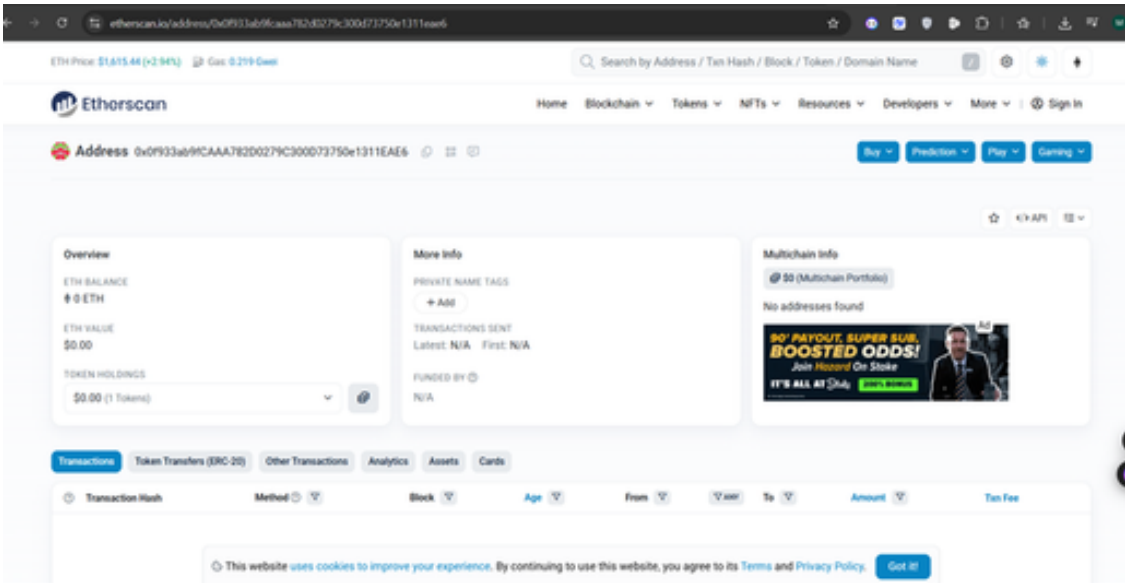
17.2 Proof of Concept

Steps to Reproduce:

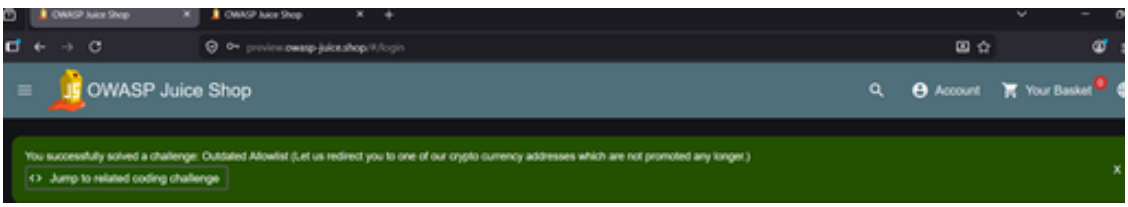
1. Locate the redirection endpoint or an outbound link (e.g., a payment/donation link) within the application.
2. Inspect the page via developer tools to identify the redirect?to= URL parameter.
3. Construct a URL that exploits the outdated allowlist logic (for example, referencing a legacy domain the allowlist still trusts).
4. Send this link to the victim or execute it in the browser.
5. The application validates the request against the flawed allowlist and redirects the browser to an untrusted destination.



Inspection of the payment options page reveals a redirect?to= parameter that can be manipulated via developer tools.



The redirect?to= URL parameter identified, highlighting how the application relies on user-controlled input to perform redirection without full validation.



Success banner confirming the 'Outdated Allowlist' challenge as solved after bypassing the allowlist and redirecting to an untrusted external URL.

17.3 Impact

Attackers can leverage the trust of the main application's domain to conduct highly credible phishing attacks, leading to credential theft. Victims can also be silently redirected to landing pages that host browser exploits or drive-by downloads.

17.4 Remediation

- Avoid User-Input Redirection: The safest approach is to avoid using dynamic user input to determine destination URLs entirely — use fixed, server-side mapping IDs if redirection is necessary.
- Strict Exact-Match Validation: If an allowlist must be used, enforce strict exact-match validation rather than flexible patterns that can be bypassed, and regularly audit and prune the allowlist.
- User Notification: Implement an intermediate warning page that notifies users they are leaving the trusted application and requires them to confirm before proceeding.

17.5 References

- OWASP Unvalidated Redirects and Forwards: https://owasp.org/www-community/attacks/Unvalidated_Redirects_and_Forwards_Cheat_Sheet
- OWASP Cheat Sheet - Unvalidated Redirects and Forwards: https://cheatsheetseries.owasp.org/cheatsheets/Unvalidated_Redirects_and_Forwards_Cheat_Sheet.html
- CWE-601: URL Redirection to Untrusted Site: https://cwe.mitre.org/data/definitions/601.html

VUL-015 — Web3 Sandbox Environment Security Misconfiguration

Finding ID	VUL-015
Title	Web3 Sandbox Environment Security Misconfiguration
Severity	**High**
CVSS v3.1 Score	7.5
CVSS Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
OWASP Category	A05:2021 - Security Misconfiguration
Affected URL	http://127.0.0.1:42000/#/web3-sandbox
CWE	CWE-16: Configuration / CWE-1188: Initialization of a Resource with an Insecure Default

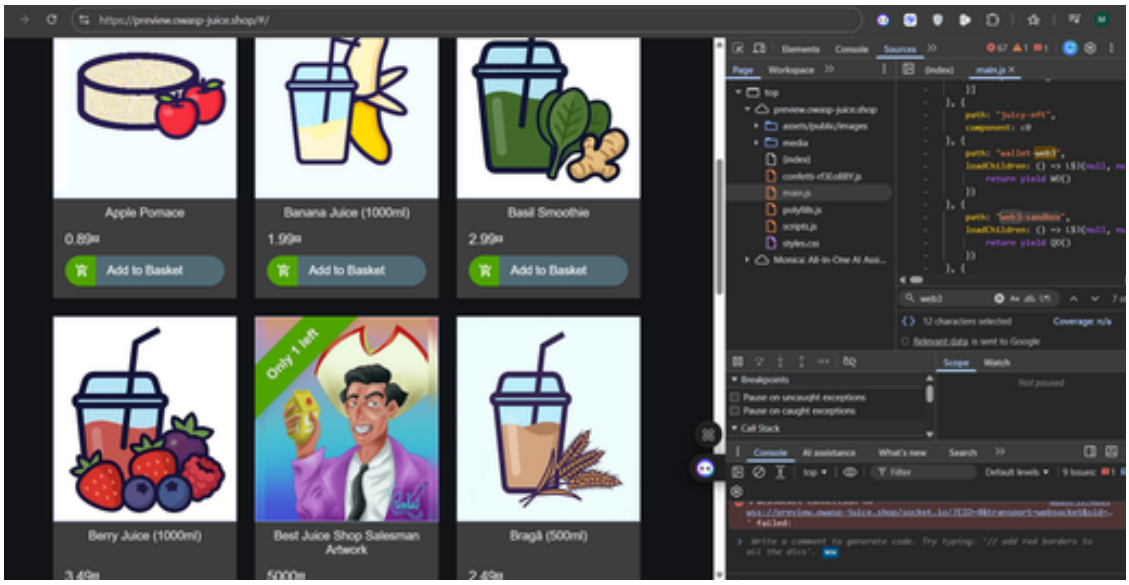
18.1 Description

The application integrates a Web3 interface containing a smart-contract sandbox environment intended for development or testing. Due to a security misconfiguration, this sandbox was left publicly accessible and active within the production deployment, and its internal execution features are not isolated from unauthenticated external users.

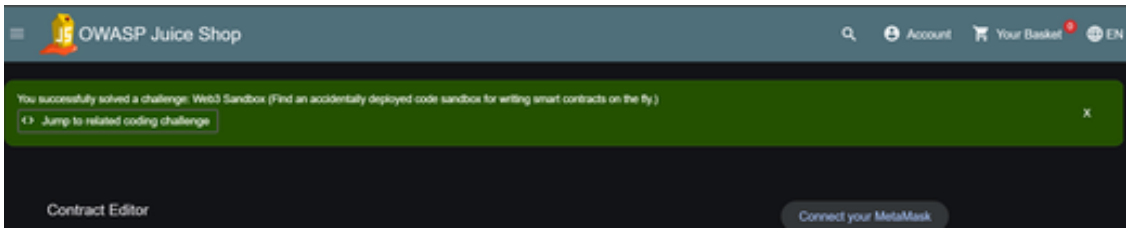
18.2 Proof of Concept

Steps to Reproduce:

1. Navigate to the application frontend or inspect the routing definitions to locate the hidden Web3 sandbox component endpoint (e.g., /#/web3-sandbox).
2. Interact with the unauthenticated interface controls to execute sandboxed Web3 commands or view backend state.
3. Observe that the application does not prompt for administrative credentials, allowing direct execution or disclosure of internal state parameters.
4. Leverage the exposed sandbox controls to confirm the target environment objective.



Application source code analysis via browser developer tools reveals references to 'web3', indicating the presence of a Web3 sandbox route.



Successful exploitation of the Web3 Sandbox misconfiguration, confirmed by the 'Web3 Sandbox' challenge completion banner.

18.3 Impact

Exposure of Web3 implementation details, test network infrastructure, or simulated key material. Unauthenticated clients can manipulate test components, leading to potential structural compromise if the sandbox is ever linked to real production systems.

18.4 Remediation

- Disable Sandbox in Production: Completely remove or disable development sandbox modules and testing routes from production build profiles.
- Enforce Robust Authentication: If the interface must be accessed remotely, implement explicit role-based access control (RBAC) so only verified administrators can reach the sandbox environment.
- Environment Hardening: Ensure sensitive functions and nodes are tightly isolated and cannot leak operational data to client-side scripts.

18.5 References

- OWASP Security Misconfiguration: https://owasp.org/Top10/A05_2021-Security_Misconfiguration/
- CWE-1188: Insecure Default Initialization of Resource: https://cwe.mitre.org/data/definitions/1188.html
- CWE-16: Configuration: https://cwe.mitre.org/data/definitions/16.html

VUL-016 — Visual Geo Stalking (Sensitive Data Exposure via Photo Wall)

Finding ID	VUL-016
Title	Visual Geo Stalking (Sensitive Data Exposure via Photo Wall)
Severity	**High**
CVSS v3.1 Score	7.5
CVSS Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
OWASP Category	A07:2021 - Identification and Authentication Failures
Affected URL	http://127.0.0.1:42000/#/photo-wall
CWE	CWE-200: Exposure of Sensitive Information to an Unauthorized Actor

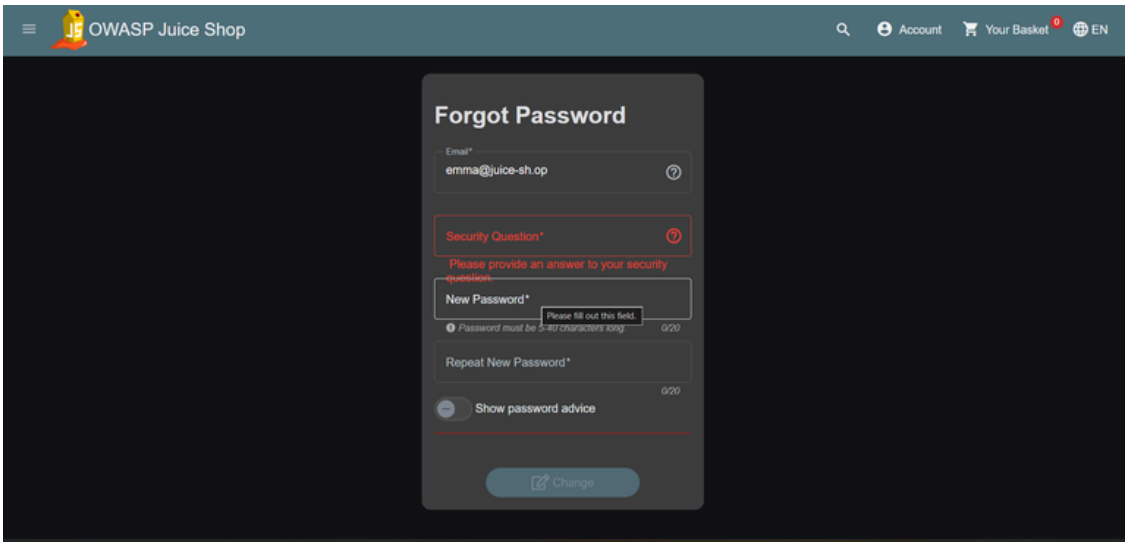
19.1 Description

The web application leaks context-rich visual information via publicly accessible media uploads on the Photo Wall. This allows a remote attacker to use OSINT and visual analysis of a photo (e.g., an identifiable building or landmark) to deduce the answer to a user's knowledge-based security question, and use that answer to bypass password-reset authentication and take over the account without the victim's consent.

19.2 Proof of Concept

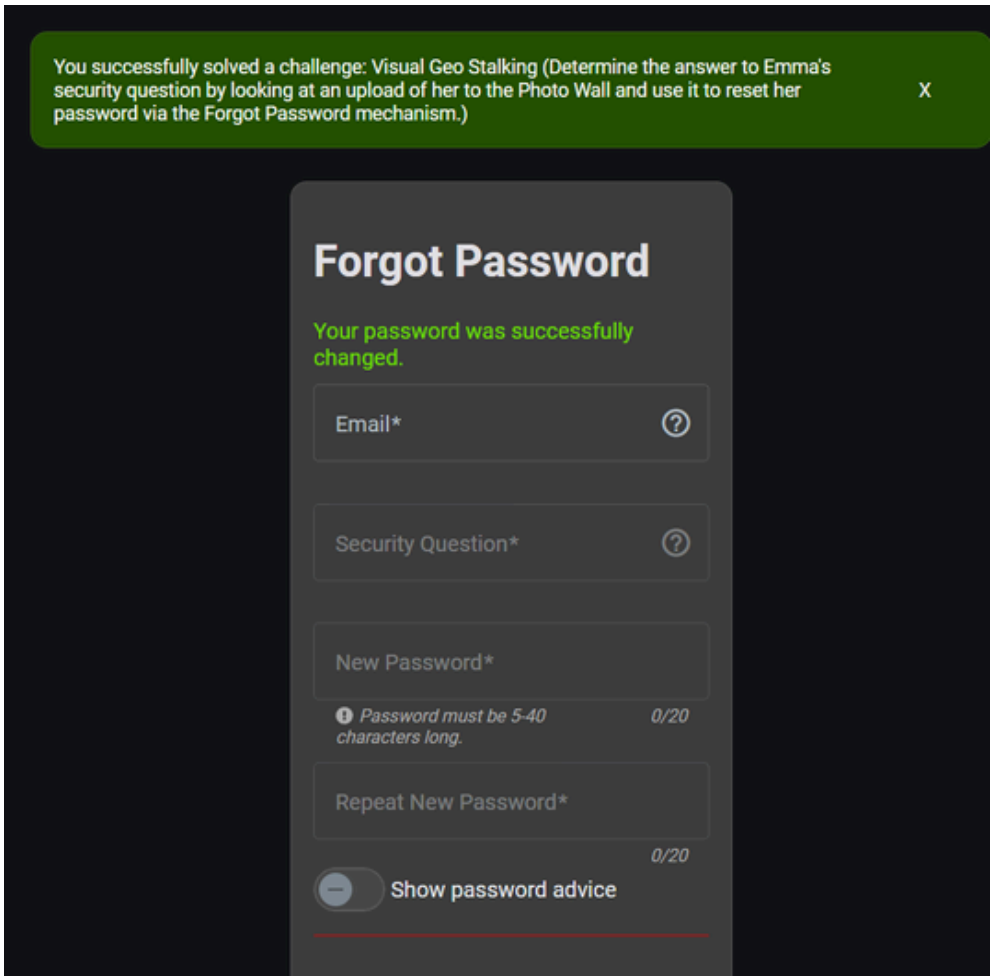
Steps to Reproduce:

1. Navigate to the "Forgot Password" interface and enter the target victim's email address to inspect the assigned security question.
2. Examine the publicly accessible photos uploaded by the user on the Photo Wall. Analyze architectural attributes and visible signage/logos within the image to conduct context-based OSINT mapping.
3. Deduce the likely answer to the security question based on the geographical location and branding identified in the image.
4. Submit the deduced answer within the security question field to bypass authentication and successfully reset the account password.



The 'Forgot Password' interface requesting the security-question answer for the target user account.

A publicly visible photo-wall image revealing distinct building landmarks usable for OSINT-based deduction.



Successful authentication bypass and password-change confirmation after answering the security question, confirming the 'Visual Geo Stalking' challenge as solved.

19.3 Impact

This flaw enables authentication-control bypass by collecting publicly available context clues, potentially leading to full account takeover. It also represents a privacy and physical-security risk, since it can allow unauthorized tracking of a user's location, workplace, or routine habits through passive data exposure in uploaded images.

19.4 Remediation

- Enforce Automated Metadata and Image Stripping: Implement backend processing to strip EXIF data and flag images containing high-risk identifying content before they are published.
- Deprecate Static Security Questions: Transition away from knowledge-based security answers toward Multi-Factor Authentication (MFA) or out-of-band one-time codes.
- Content Moderation: Audit and filter publicly rendered image walls to limit exposure of operational environments or high-value locations.

19.5 References

- OWASP Identification and Authentication Failures: https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/
- CWE-200: Exposure of Sensitive Information: https://cwe.mitre.org/data/definitions/200.html

VUL-017 — Login Jim — SQL Injection Authentication Bypass (Targeted Account)

Finding ID	VUL-017
Title	Login Jim — SQL Injection Authentication Bypass (Targeted Account)
Severity	**Critical**
CVSS v3.1 Score	9.8
CVSS Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
OWASP Category	A03:2021 - Injection
Affected URL	http://127.0.0.1:42000/rest/user/login
CWE	CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')

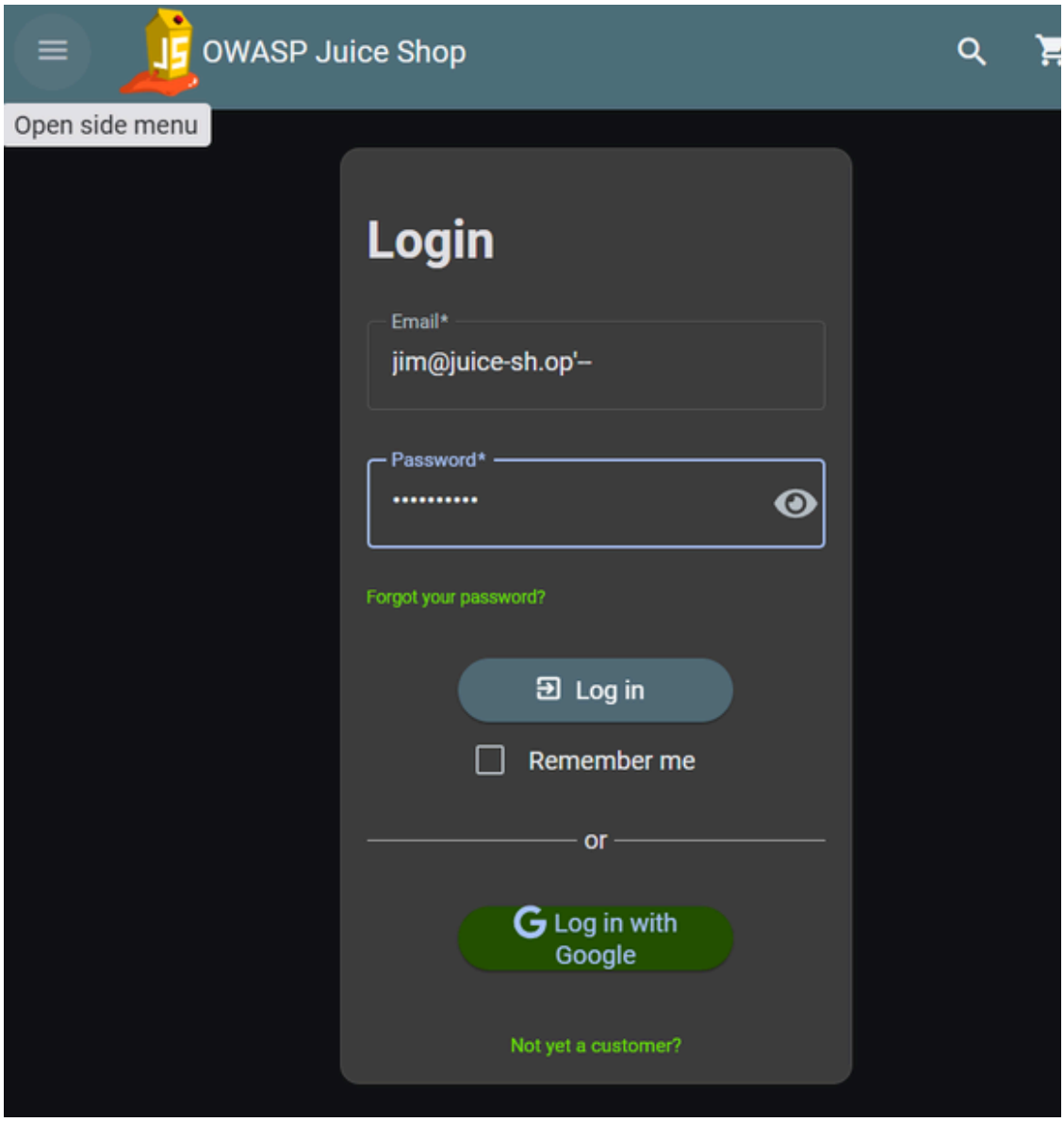
20.1 Description

In addition to the generic administrator SQL Injection bypass described in VUL-006, the same unsanitized login query can be targeted at a specific, named victim account by supplying that account's email address followed by a SQL comment sequence. This lets an attacker impersonate any known user, not only the administrator, without possessing a valid password.

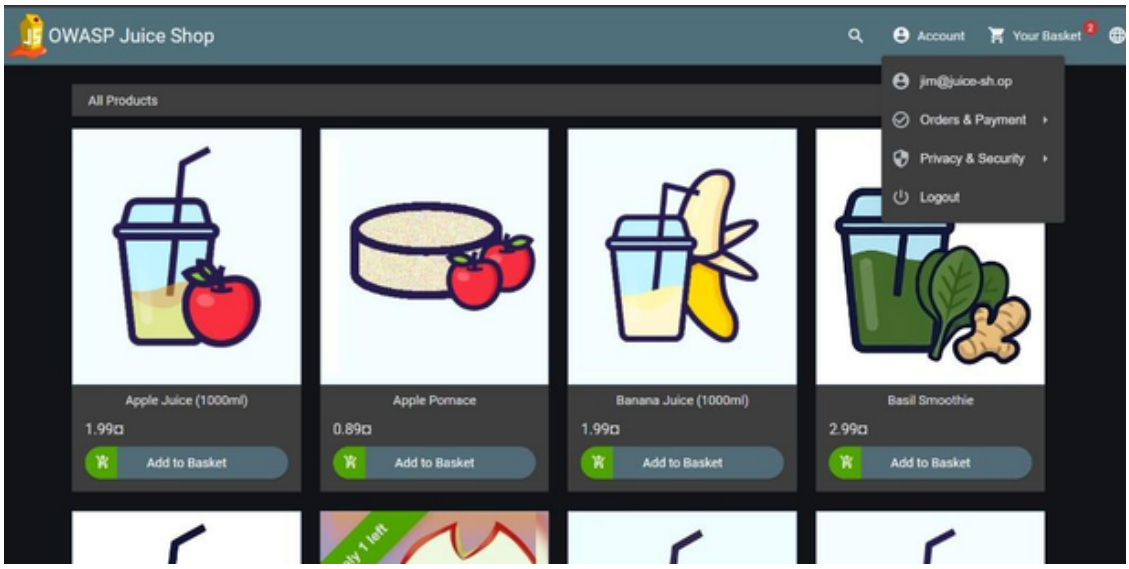
20.2 Proof of Concept

Steps to Reproduce:

1. Navigate to the OWASP Juice Shop login interface at http://127.0.0.1:42000/#/login.
2. In the Email input field, inject the payload targeting a specific account: jim@juice-sh.op'--
3. Type an arbitrary value in the Password field.
4. Click the Log in button. The application comments out the password verification logic, granting a session under the targeted user's identity.



The authentication interface with the targeted SQL injection payload applied directly inside the email form field.



Successful authentication bypass showing the application dashboard fully authorized under the targeted user's session context.

20.3 Impact

Attackers can seamlessly assume the identity of any known, named user without any knowledge of their credentials, enabling full account hijacking, unauthorized order placement, and exposure of the victim's personal order history and profile data.

20.4 Remediation

- Utilize Native Prepared Statements: Enforce database access only via pre-compiled parameterized queries to isolate query logic from user-supplied data.
- Implement Input Filter Control: Restrict dynamic entry formats using strict allow-list patterns to flag unexpected injection characters before execution.
- Least Privilege Configuration: Enforce minimal operational database access parameters for the application's service account.

20.5 References

- OWASP SQL Injection Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- CWE-89: SQL Injection: https://cwe.mitre.org/data/definitions/89.html

VUL-018 — Login Bender — SQL Injection Authentication Bypass (Targeted Account)

Finding ID	VUL-018
Title	Login Bender — SQL Injection Authentication Bypass (Targeted Account)
Severity	**Critical**
CVSS v3.1 Score	9.8
CVSS Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
OWASP Category	A03:2021 - Injection
Affected URL	http://127.0.0.1:42000/rest/user/login
CWE	CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')

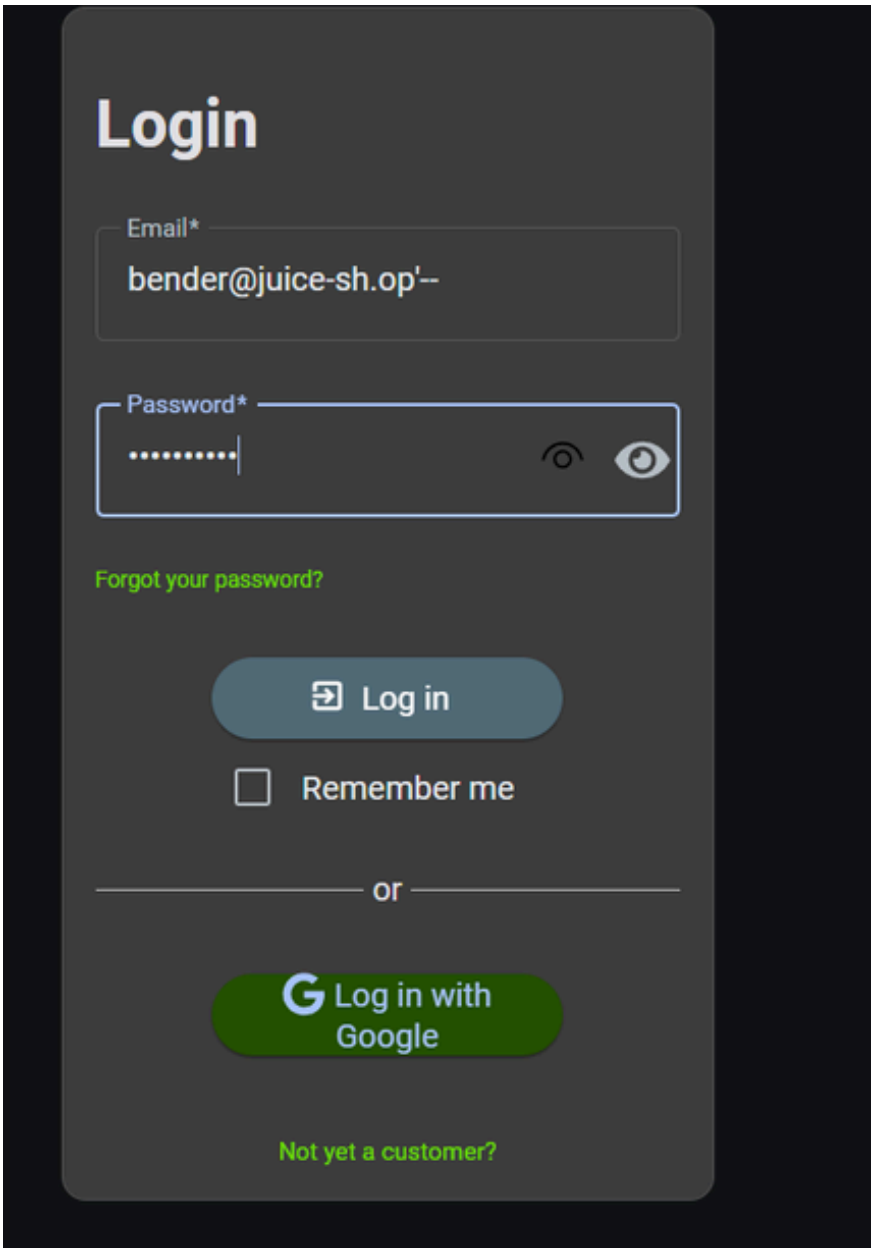
21.1 Description

This is the same root-cause SQL Injection flaw as VUL-006/VUL-017 (unsanitized concatenation of the email field into the backend login query), demonstrated here against a second, distinct named account to confirm the vulnerability is exploitable against any user in the database, not only the administrator or a single test account.

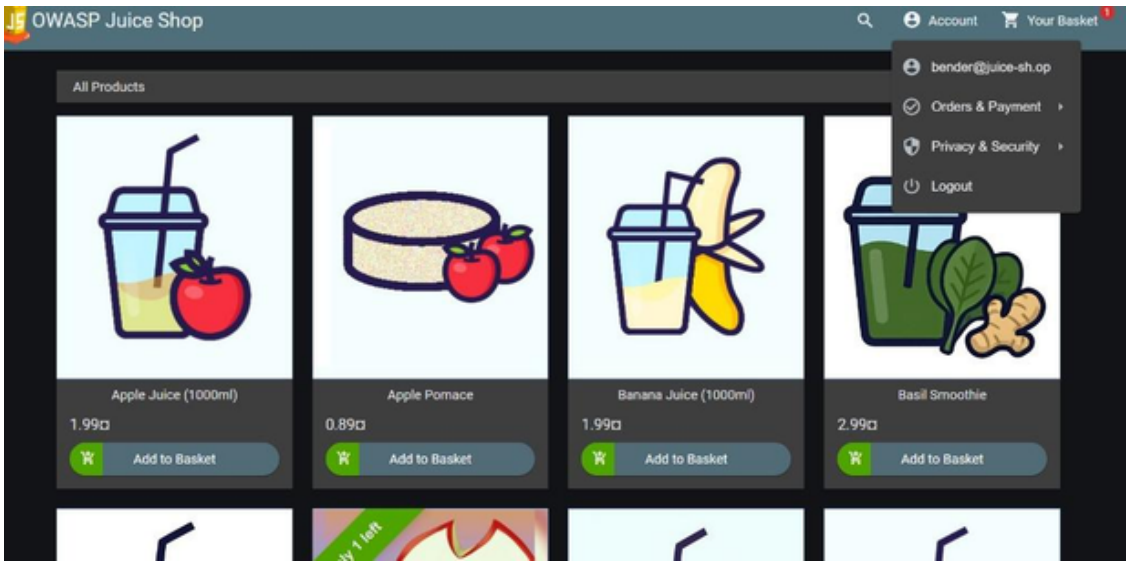
21.2 Proof of Concept

Steps to Reproduce:

1. Access the login portal at http://127.0.0.1:42000/#/login.
2. In the email field, inject the structural escape sequence targeting the account: bender@juice-sh.op'--
3. Provide an arbitrary character sequence in the password field to satisfy front-end input requirements.
4. Submit the payload. The single-quote delimiter exits the expected text block and the double-dash sequence comments out the rest of the query line, voiding the password comparison step and authenticating the attacker as 'bender@juice-sh.op'.



The authentication interface showing the targeted SQL injection payload structured directly within the email input field.



Successful authentication bypass confirming an active session under the targeted account's identity.

21.3 Impact

Complete account hijacking for any targeted, named identity without any knowledge of credential attributes, along with exposure of personalized order tracking data and access tokens stored within that user's database row.

21.4 Remediation

- Utilize Native Prepared Statements: Enforce database access only via pre-compiled parameter schemas to isolate structural query instructions from dynamic input arguments.
- Implement Input Filter Control: Restrict dynamic entry formats using strict pattern allow-lists to flag unexpected injection characters before execution.
- Least Privilege Configuration: Enforce minimal operational database access parameters for the application's service layer.

21.5 References

- OWASP SQL Injection Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- CWE-89: SQL Injection: https://cwe.mitre.org/data/definitions/89.html

VUL-019 — Client-Side XSS Protection Bypass via Direct API Manipulation (Stored XSS)

Finding ID	VUL-019
Title	Client-Side XSS Protection Bypass via Direct API Manipulation (Stored XSS)
Severity	**High**
CVSS v3.1 Score	8.1
CVSS Vector	AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:H/A:N
OWASP Category	A03:2021 - Injection
Affected URL	https://preview.owasp-juice.shop/api/Products
CWE	CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')

22.1 Description

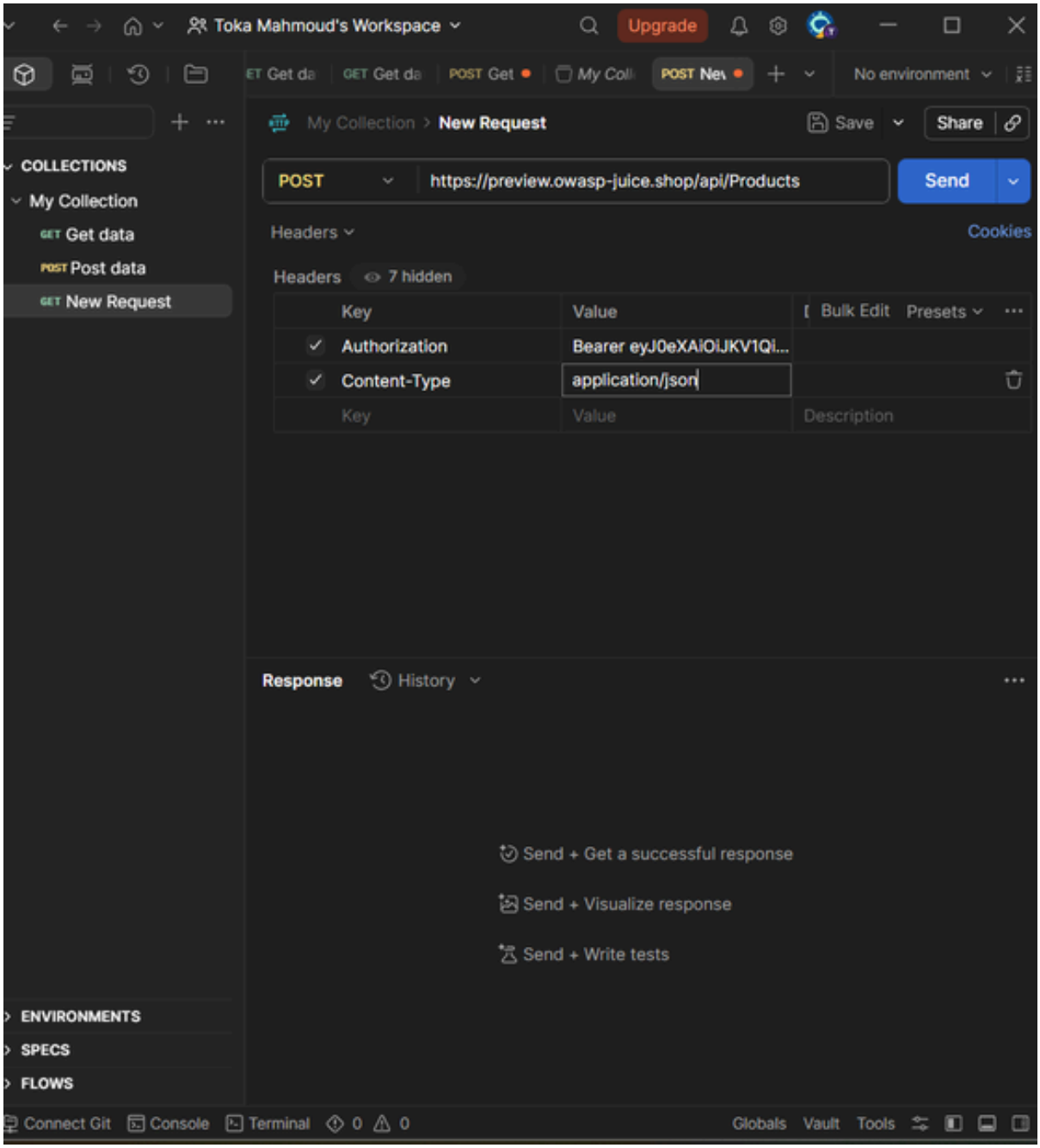
The web application exhibits a severe injection flaw where XSS input sanitization is implemented exclusively on the client-side (frontend) user interface. Because these controls are restricted to the frontend, attackers can bypass them entirely by interacting directly with the backend REST API using tools like Postman: an authenticated user can inject malicious payloads into application data (such as product descriptions), resulting in Stored XSS that is validated and served back to every visitor of the product catalog.

22.2 Proof of Concept

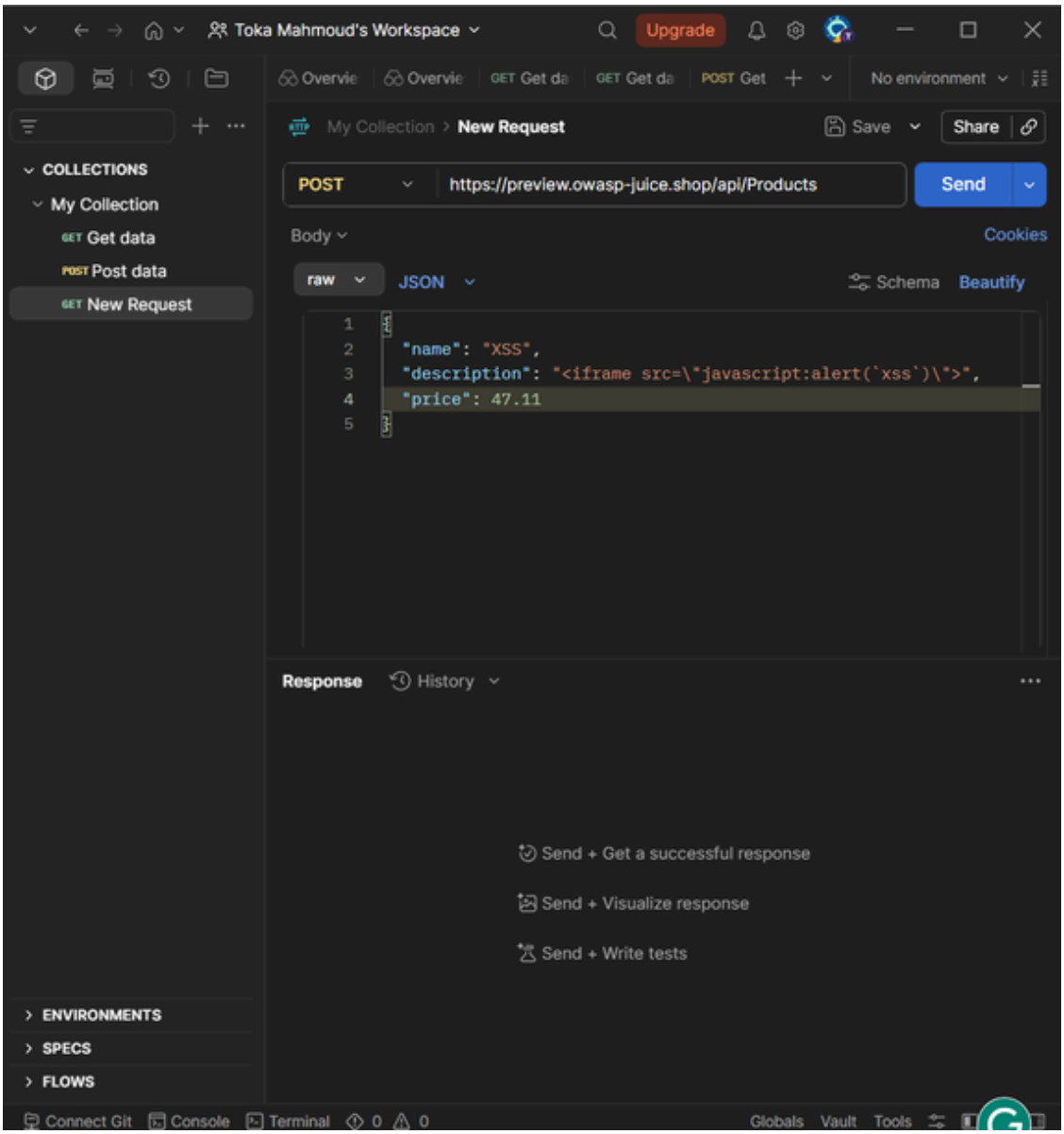
Steps to Reproduce:

1. Authenticate into the application via the browser, inspect network traffic using DevTools, and capture the active session token from the Authorization header.
2. Open Postman and construct a direct POST request targeted at the backend API resource endpoint: https://preview.owasp-juice.shop/api/Products.
3. Configure the request headers with the captured session token (Authorization: Bearer <TOKEN>) and Content-Type: application/json.
4. Inject an XSS script payload inside the JSON body's product description attribute.

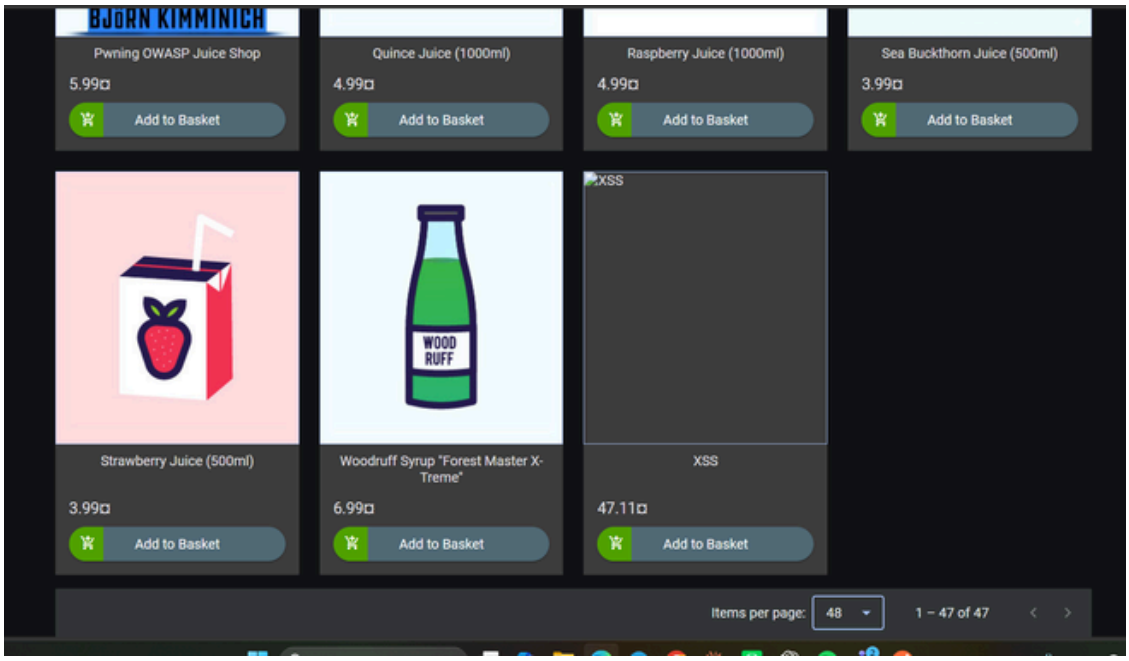
5. Submit the payload using Postman. The database engine executes the row addition, bypassing any client-side sanitization filters, and returns a 201 Created status.



Extraction of a valid session bearer token from the browser's developer tools network request headers.



Constructing the direct POST request payload inside Postman, resulting in a successful 201 Created server response.



The injected product record rendering inside the frontend product catalog, confirming the stored payload was accepted and served to all viewers.

22.3 Impact

Arbitrary JavaScript code stored this way executes in the browser of every user who visits the product catalog, creating a high risk of session/cookie hijacking, local storage exfiltration, and structural manipulation of catalog data — bypassing presentation-layer business logic entirely.

22.4 Remediation

- Server-Side Input Validation: Never rely exclusively on client-side constraints. Implement context-aware validation on the API controller layer to reject structural HTML elements.
- Contextual Output Encoding: Ensure that text-rendering frameworks properly encode characters before binding untrusted data dynamically to the page.
- Establish Content Security Policy (CSP): Restrict script execution to explicitly trusted operational origins.

22.5 References

- OWASP Cross-Site Scripting Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- CWE-79: Improper Neutralization of Input: https://cwe.mitre.org/data/definitions/79.html

VUL-020 — Forged Feedback (Broken Access Control via Hidden UserId Parameter)

Finding ID	VUL-020
Title	Forged Feedback (Broken Access Control via Hidden UserId Parameter)
Severity	**Medium**
CVSS v3.1 Score	6.5
CVSS Vector	AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:N
OWASP Category	A01:2021 - Broken Access Control
Affected URL	http://127.0.0.1:42000/#/contact (API: /api/Feedbacks/)
CWE	CWE-639: Authorization Bypass Through User-Controlled Key

23.1 Description

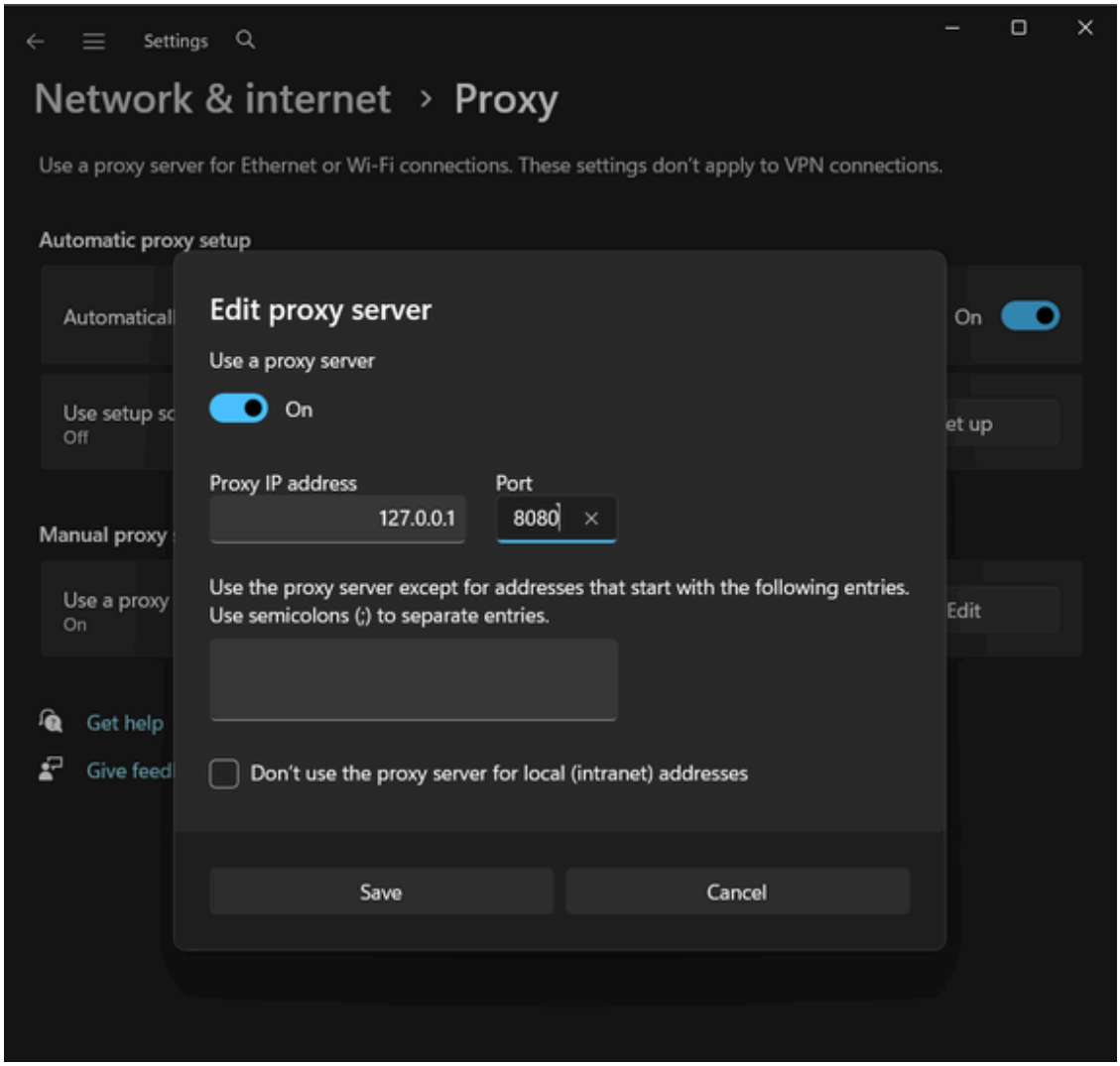
The Customer Feedback feature exposes a server-side data model that accepts more parameters than the client-facing form presents. While the UI only submits a comment, a rating, and a CAPTCHA answer, the underlying /api/Feedbacks/ endpoint also accepts a UserId attribute that is never surfaced in the interface. Because the application hides this field from the form rather than enforcing ownership checks on the server, an attacker can intercept the outgoing request and inject an arbitrary UserId value, posting feedback under the identity of another user without their knowledge — a classic IDOR / Broken Access Control flaw.

23.2 Proof of Concept

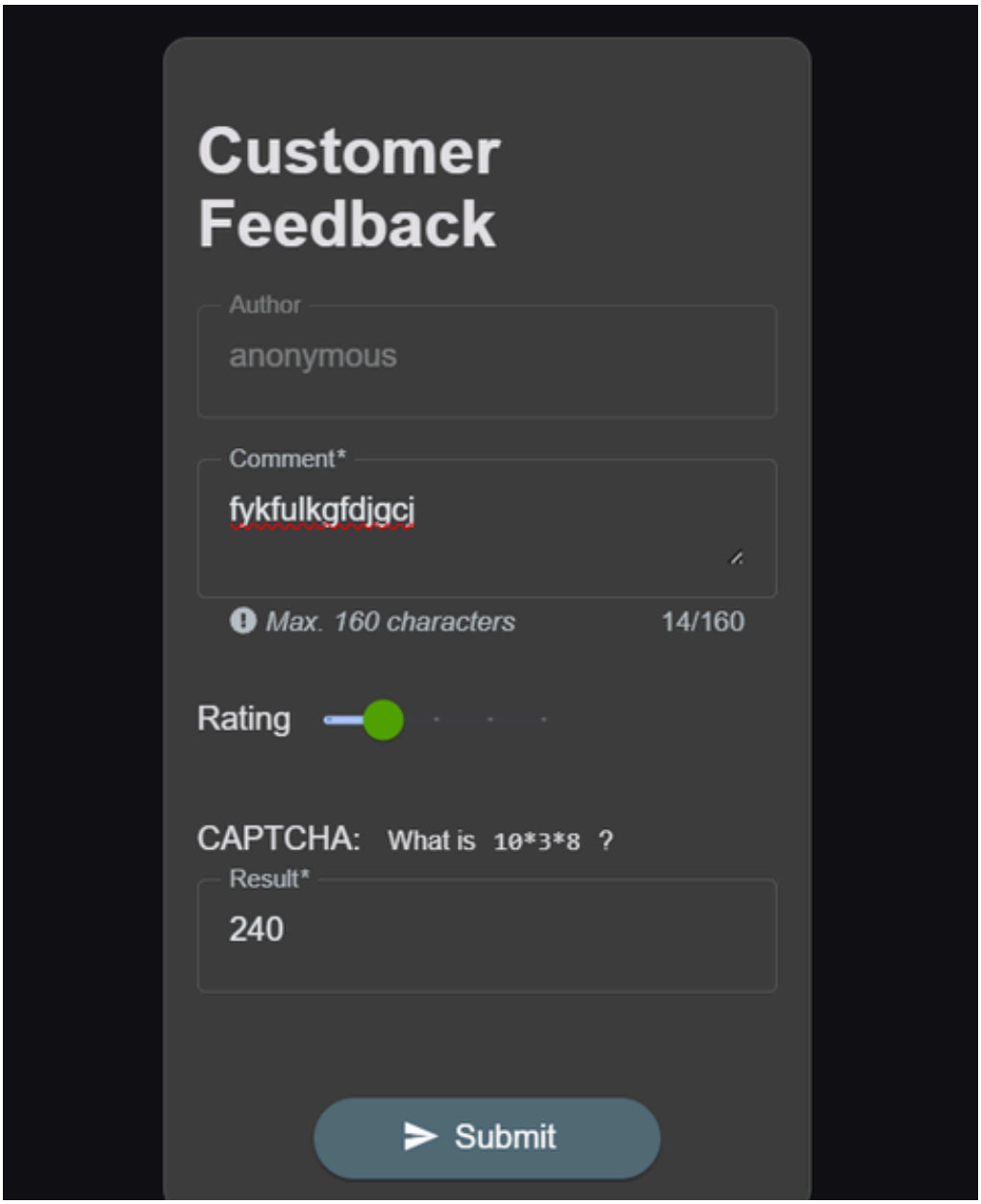
Steps to Reproduce:

1. Configure the browser's network settings to route traffic through an intercepting proxy (Burp Suite) listening on 127.0.0.1:8080.
2. Navigate to the Customer Feedback form, enter a comment, select a rating, solve the CAPTCHA, and click Submit.
3. With intercept switched on, capture the outgoing POST /api/Feedbacks/ request before it reaches the server.
4. Inspect the intercepted JSON body — note it only contains the UI-exposed fields (captchaId, captcha, comment, rating).
5. Manually add a "UserId" key set to the numeric ID of a target victim account, then forward the tampered request.

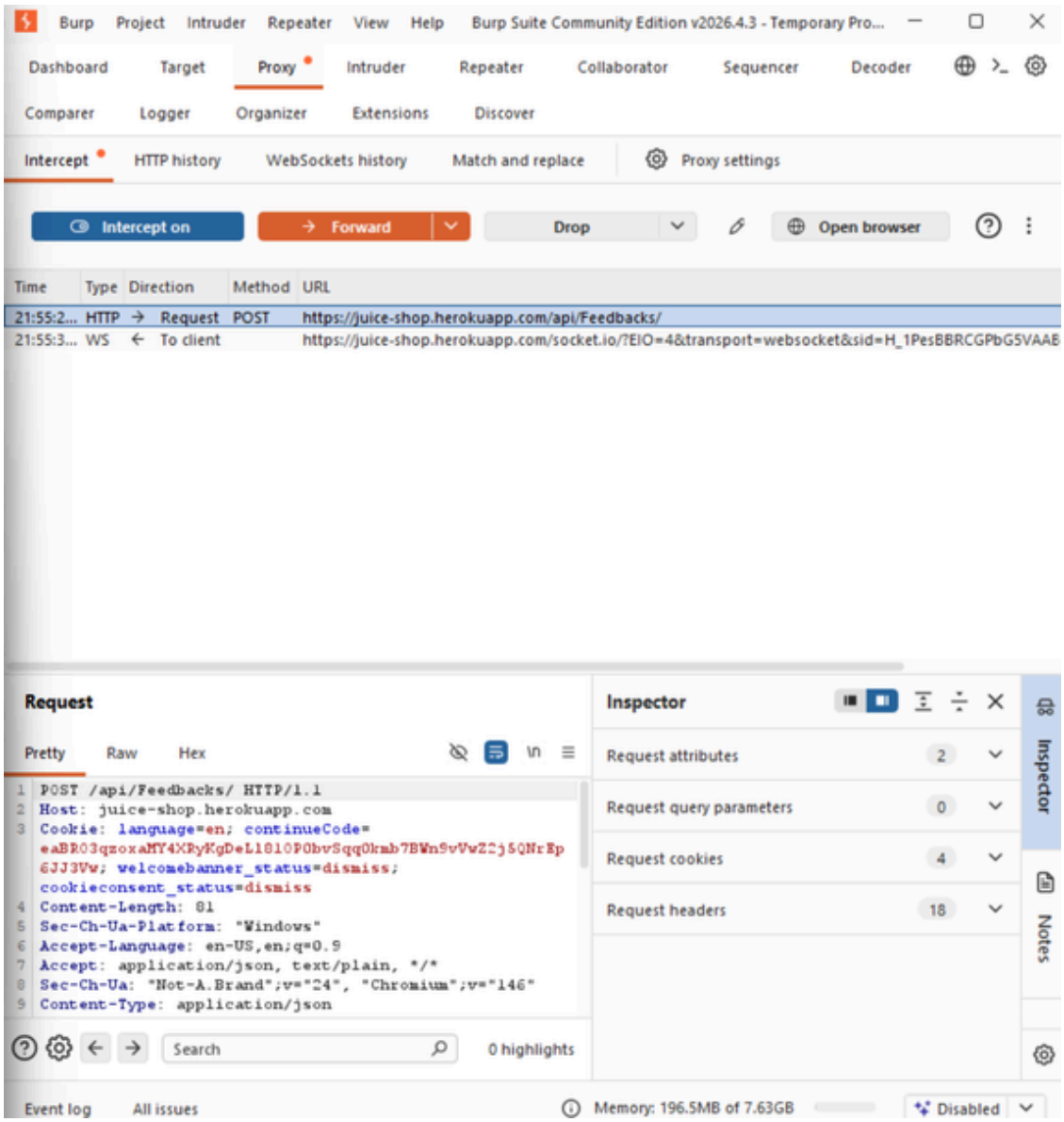
6. The server accepts the request and stores the feedback as if it were submitted by the victim, confirming the absence of server-side authorization checks on the UserId parameter.



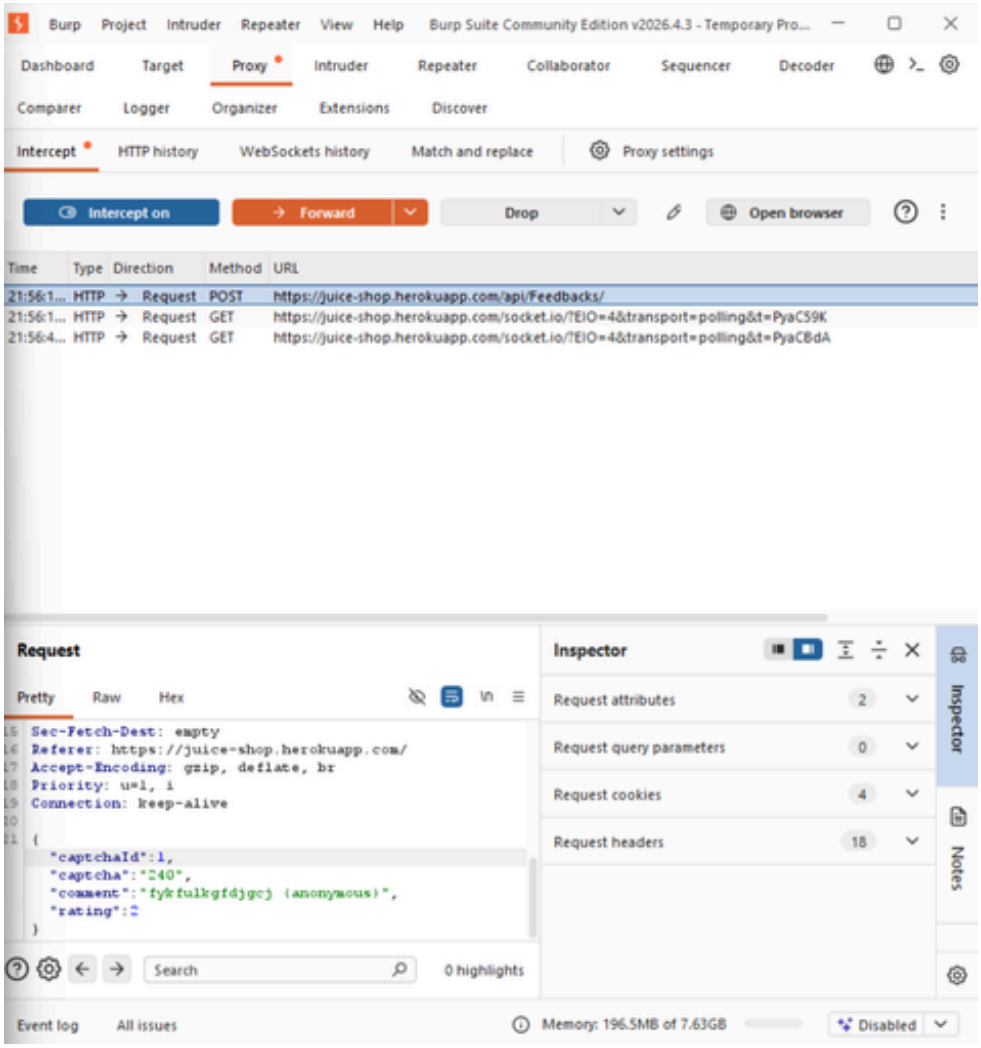
Configuring the system proxy to route traffic through Burp Suite for request interception.



The Customer Feedback form submitted with a comment, a rating, and the solved CAPTCHA.



Burp Suite intercepting the outgoing POST request to /api/Feedbacks/ before it reaches the server.



The intercepted JSON request body showing the client-exposed parameters that can be tampered with, including injection of the hidden UserId field.

23.3 Impact

Attackers can post arbitrary feedback content under another user's identity without any credentials, damaging that user's reputation or misrepresenting their opinions. This confirms authorization is enforced only through client-side field visibility rather than server-side ownership validation — a systemic trust issue in how the API processes user-supplied identifiers, which can be leveraged for defamation or social engineering.

23.4 Remediation

- Enforce Server-Side Ownership Checks: Derive the UserId strictly from the authenticated session token; never trust or accept a UserId supplied in the request body.
- Apply the Principle of Least Exposure: Ensure API data models used by public endpoints only accept fields intentionally exposed to the client, rejecting unrecognized parameters.
- Input Validation and Schema Enforcement: Use strict request-schema allow-listing on the backend to reject payloads containing unauthorized attributes such as a client-supplied UserId.

23.5 References

- OWASP Broken Access Control: https://owasp.org/Top10/A01_2021-Broken_Access_Control/
- CWE-639: Authorization Bypass Through User-Controlled Key: https://cwe.mitre.org/data/definitions/639.html

VUL-021 — Confidential Document Exposure via Predictable Identifiers

Editor's note: A specific, verified URL was added for this reference — see change log.

Finding ID	VUL-021
Title	Confidential Document Exposure via Predictable Identifiers
Severity	**High**
CVSS v3.1 Score	7.1
CVSS Vector	AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N
OWASP Category	A01:2021 - Broken Access Control
Affected URL	http://127.0.0.1:42000/ (document/complaint retrieval endpoints)
CWE	CWE-285: Improper Authorization

24.1 Description

This class of vulnerability represents a critical flaw where sensitive documents are exposed to unauthorized users because access controls are misconfigured or missing entirely, allowing attackers to retrieve confidential files through direct URL access, predictable naming/ID conventions, or inadequate authentication checks. This occurs when an application incorrectly assumes all authenticated users have equal access rights, or relies on client-side checks that can be bypassed. The underlying principle violated is the Principle of Least Privilege.

24.2 Proof of Concept

Steps to Reproduce:

1. Identify the document/record retrieval mechanism used by the target application (e.g., a URL pattern such as /some-endpoint?id=123).
2. Authenticate with a low-privileged account and note how document or record identifiers are structured (sequential, predictable IDs).
3. Using a proxy tool, enumerate nearby document/record IDs sequentially.
4. Confirm that documents or records outside the authenticated user's own scope are returned without an authorization error.
5. Retrieve a record that should be restricted to a different user or role, confirming the missing access-control check.

24.3 Impact

Competitors or malicious actors could gain access to internal records, pricing/strategy information, or personal data belonging to other users, compromising confidentiality and potentially triggering regulatory compliance violations (e.g., GDPR) if personal data is exposed.

24.4 Remediation

- Implement a comprehensive, centralized access-control framework (middleware or authorization service) that checks permissions before serving any content, rather than relying on the client to only request what it should see.
- Implement Role-Based Access Control (RBAC) with clearly defined roles and permissions, and use random, unguessable identifiers instead of sequential IDs.
- Store sensitive files outside the web root and serve them only through a controlled, authorization-checked download mechanism.

24.5 References

- OWASP Access Control Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Access_Control_Cheat_Sheet.html
- CWE-285: Improper Authorization: https://cwe.mitre.org/data/definitions/285.html

VUL-022 — Verbose Error Handling (Security Misconfiguration)

Finding ID	VUL-022
Title	Verbose Error Handling (Security Misconfiguration)
Severity	**Medium**
CVSS v3.1 Score	5.3
CVSS Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
OWASP Category	A05:2021 - Security Misconfiguration
Affected URL	http://127.0.0.1:42000/ (multiple endpoints)
CWE	CWE-209: Generation of Error Message Containing Sensitive Information

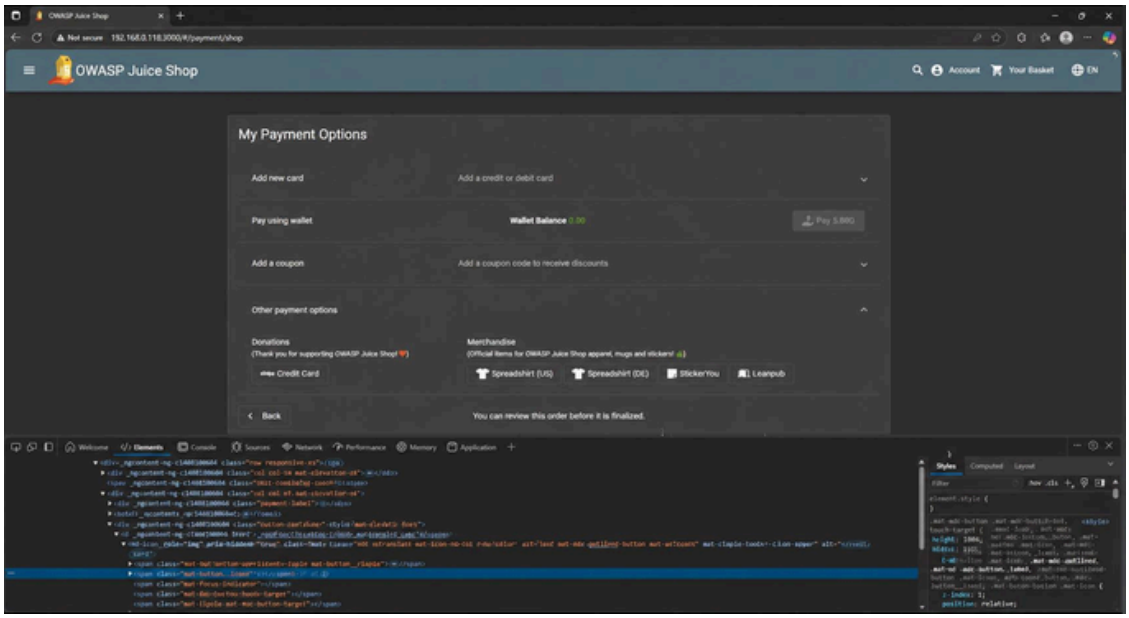
25.1 Description

The application exposes detailed system information to users when exceptions occur during execution, including stack traces, underlying SQL statements/error codes, and other internal implementation details. This is a common security misconfiguration that arises when development-mode error verbosity is inadvertently left enabled in a production deployment.

25.2 Proof of Concept

Steps to Reproduce:

1. Interact with a form or API endpoint (such as the payment options or feedback submission flow) using deliberately malformed input.
2. Inspect the application's response (visible in the browser console/network panel) for detailed error output.
3. Observe that certain requests trigger verbose error messages containing backend error codes and query fragments (e.g., a raw SQL statement and 'SQLITE_ERROR' code) rather than a generic, user-safe error message.
4. Note that this same verbose behavior is what confirmed the SQL Injection findings elsewhere in this report (VUL-006, VUL-017, VUL-018), since the raw database error was what first revealed the injectable query.



Inspecting the payment options page via DevTools; malformed requests against related endpoints return verbose backend error detail rather than a generic message.

25.3 Impact

The exposure of system internals through error messages significantly increases the application's vulnerability to targeted attacks by giving attackers reconnaissance information (database engine, query structure, internal paths). If the error messages inadvertently include personal data, this could also constitute a regulatory compliance issue.

25.4 Remediation

- Implement centralized exception handling with custom, generic error pages/messages shown to the client in production.
- Log detailed error information (stack traces, query text) server-side only, never in the client-facing response.
- Use environment-specific configuration to ensure verbose/debug error output is only ever enabled in development, never in production.

25.5 References

- OWASP Error Handling Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Error_Handling_Cheat_Sheet.html
- CWE-209: Generation of Error Message Containing Sensitive Information: https://cwe.mitre.org/data/definitions/209.html

VUL-023 — Exposed Application Metrics and Progress Data

Editor's note: Reference updated from the retired 2017 'Sensitive Data Exposure' link — see change log.

Finding ID	VUL-023
Title	Exposed Application Metrics and Progress Data
Severity	**Medium**
CVSS v3.1 Score	5.3
CVSS Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
OWASP Category	A02:2021 - Cryptographic Failures
Affected URL	http://127.0.0.1:42000/ (client-side configuration / progress endpoints)
CWE	CWE-200: Exposure of Sensitive Information to an Unauthorized Actor

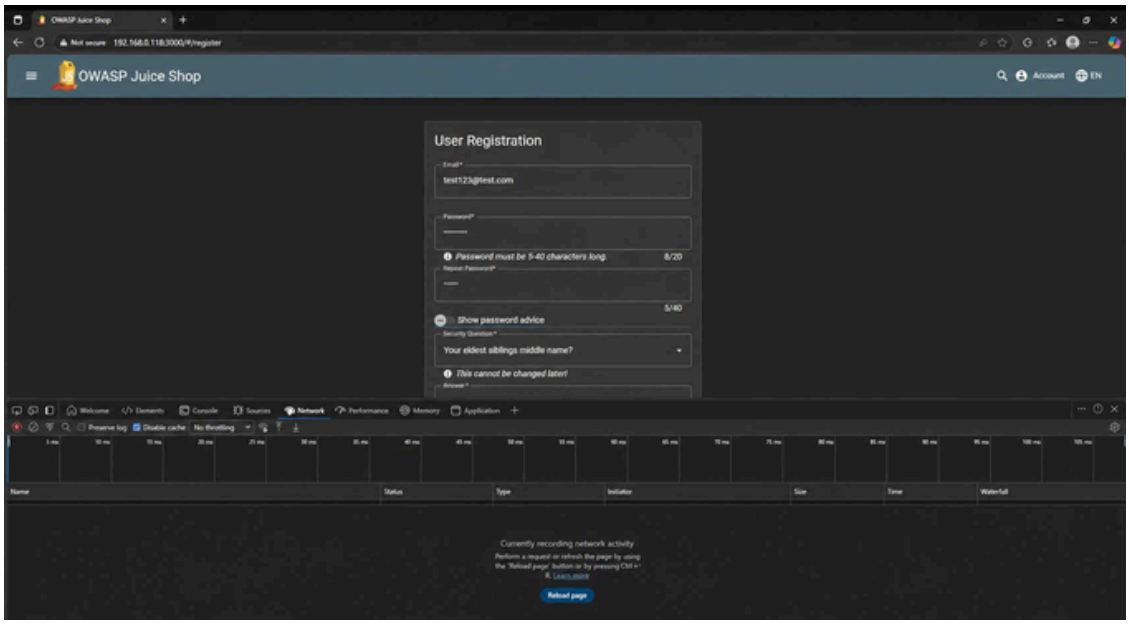
26.1 Description

The application exposes internal operational metrics — such as challenge/progress tracking data and configuration values normally intended only for monitoring — through client-accessible endpoints and bundled configuration, without requiring authentication or authorization. While individually low-impact, this class of exposure gives an attacker detailed reconnaissance data about the application's internal state and behavior.

26.2 Proof of Concept

Steps to Reproduce:

1. Browse the application and open the Developer Tools Network tab.
2. Identify requests to configuration/metrics-style endpoints (such as application-configuration) that return internal state without requiring authentication.
3. Review the returned data for operational details not intended for a general/unauthenticated audience.
4. Confirm the same information is reachable directly, without ever having logged in.



The product catalog page inspected via DevTools Network tab, showing unauthenticated requests to internal configuration/metrics-style endpoints.

26.3 Impact

Attackers can use exposed operational metrics to plan more effective, better-timed attacks (for example, targeting periods of higher load), and the exposure itself may violate data-minimization expectations in regulated environments.

26.4 Remediation

- Require authentication and authorization for all monitoring, metrics, and configuration endpoints not needed by the general public.
- Restrict access to these endpoints to authorized internal IP ranges where possible.
- Review default configurations of all frameworks/libraries in use to ensure monitoring endpoints are disabled or secured in production.

26.5 References

- OWASP Top 10 - A02:2021 Cryptographic Failures: https://owasp.org/Top10/A02_2021-Cryptographic_Failures/
- CWE-200: Exposure of Sensitive Information: https://cwe.mitre.org/data/definitions/200.html

VUL-024 — Mass Dispel — Missing Per-Item Authorization on Batch Operations

Editor's note: A specific, verified URL was added for this reference — see change log.

Finding ID	VUL-024
Title	Mass Dispel — Missing Per-Item Authorization on Batch Operations
Severity	**Medium**
CVSS v3.1 Score	6.5
CVSS Vector	AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:N
OWASP Category	A01:2021 - Broken Access Control
Affected URL	http://127.0.0.1:42000/ (batch-capable endpoints)
CWE	CWE-285: Improper Authorization

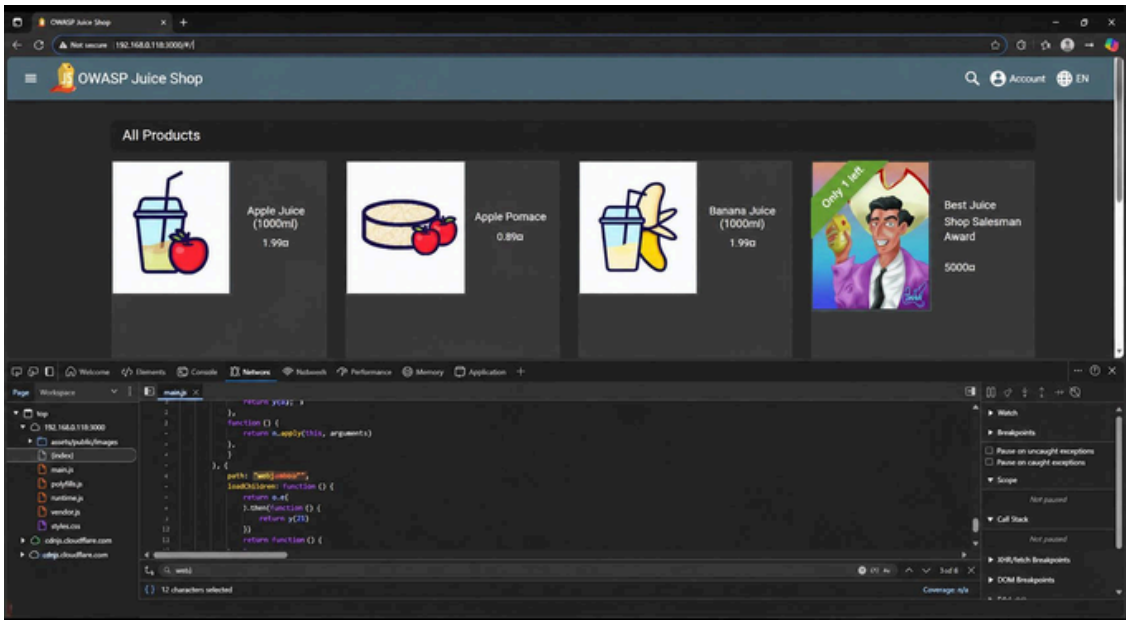
27.1 Description

This class of vulnerability occurs when an application allows bulk operations on resources without per-item authorization checks, enabling an attacker to perform large-scale modifications or deletions with a single request. It arises when authorization is checked only once for the whole batch operation (or not at all), and client-supplied lists of IDs are trusted without validating that the requester actually owns or may act on each individual item.

27.2 Proof of Concept

Steps to Reproduce:

1. Identify a feature in the application that accepts a list/array of identifiers in a single request (e.g., a bulk update or bulk delete operation).
2. Using a proxy tool, intercept the request and inspect the list of identifiers being sent.
3. Modify the list to include identifiers that do not belong to the authenticated user.
4. Forward the modified request and confirm whether the server processes the entire batch, including the items the user should not be authorized to act on.



Multiple challenge-completion banners stacking in a single session, illustrating how the application processes several state-changing actions from a single authenticated context without distinguishing per-item ownership.

27.3 Impact

A single successful attack of this class can result in the loss or corruption of business data at scale, since a batch operation multiplies the impact of a missing authorization check across every item in the request rather than a single resource.

27.4 Remediation

- Validate authorization for each individual item in a batch operation, not just the request as a whole.
- Implement batch-size limits and rate limiting to reduce the blast radius of any single request.
- Use comprehensive audit logging for all batch operations, and require step-up authentication (e.g., re-confirmation) for high-impact batch actions.

27.5 References

- OWASP Authorization Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html
- CWE-285: Improper Authorization: https://cwe.mitre.org/data/definitions/285.html

VUL-025 — Missing Output Encoding Enabling Cross-Site Scripting

Finding ID	VUL-025
Title	Missing Output Encoding Enabling Cross-Site Scripting
Severity	**High**
CVSS v3.1 Score	7.4
CVSS Vector	AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:H/A:N
OWASP Category	A03:2021 - Injection
Affected URL	http://127.0.0.1:42000/ (multiple user-input reflection points)
CWE	CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')

28.1 Description

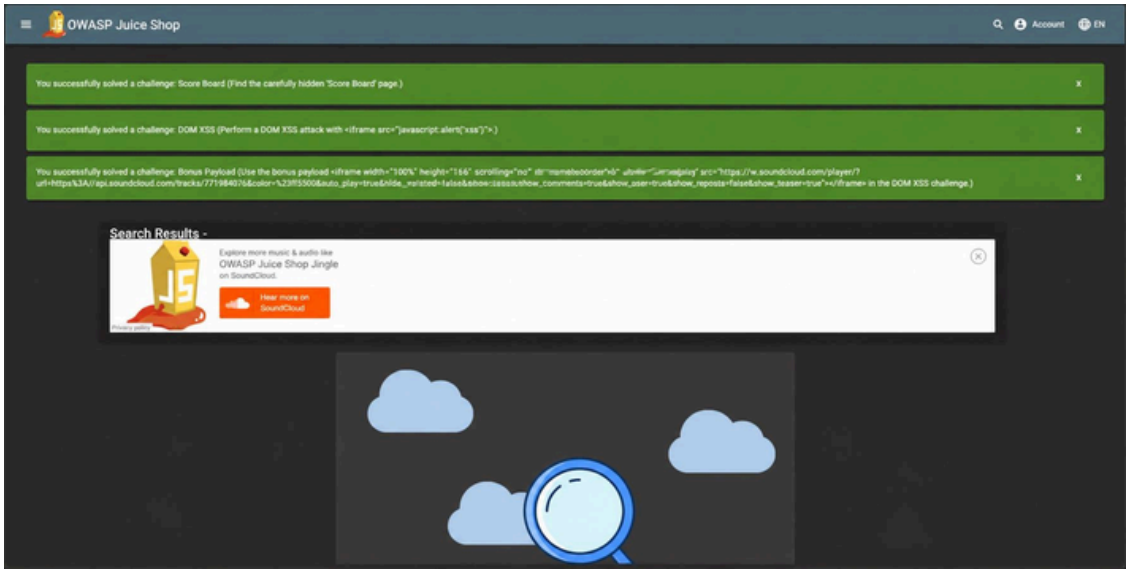
This finding summarizes the systemic root cause behind the multiple XSS findings in this report (VUL-002, VUL-003, VUL-011, VUL-019): the application outputs user-supplied data without consistently applying context-aware encoding or escaping. Special HTML characters (<, >, &, ", ') are not reliably neutralized before being written into the DOM, so the browser interprets attacker-supplied markup as executable code rather than inert text.

28.2 Proof of Concept

Steps to Reproduce:

1. Identify a feature that displays user-supplied data back to the user or to other users (e.g., search results, product descriptions, feedback).
2. Submit a payload containing an XSS test vector, such as <script>alert('XSS')</script> or an equivalent tag-based payload.
3. Observe whether the payload is rendered as executable markup rather than being encoded/escaped as literal text.
4. Confirm the behavior is consistent across client-facing UI (VUL-002, VUL-003, VUL-011) and the underlying API when accessed directly (VUL-019), demonstrating that encoding is missing at the root rather than in one isolated

field.



Burp Suite intercepting a request/response pair for a user-input field, used to confirm that special characters are passed through the application without encoding.

28.3 Impact

Successful exploitation can lead to complete compromise of a victim's session (via cookie/token theft), keystroke capture, or actions performed on the victim's behalf. Because it can be reproduced through multiple entry points and directly via the API, this is a systemic issue rather than a single-field bug, and organizations handling personal or financial data face substantial regulatory exposure if it is exploited at scale.

28.4 Remediation

- Implement comprehensive, context-aware output encoding based on where data will be displayed (HTML body, HTML attribute, JavaScript, URL, or CSS context).
- Use parameterized queries and framework-safe rendering everywhere, rather than string concatenation or raw DOM injection.
- Deploy a Content Security Policy (CSP) to reduce the impact of any output-encoding gap that is missed.
- Adopt a security-focused code review process (see the Secure Code Review Cheat Sheet) that specifically checks output encoding on every new user-input reflection point.

28.5 References

- OWASP Cross-Site Scripting Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- CWE-79: Improper Neutralization of Input: https://cwe.mitre.org/data/definitions/79.html

VUL-026 — Broken Access Control: Unauthorized Deletion of Customer Feedback

Finding Metadata	Value
ID	VUL-026
Title	Broken Access Control — Unauthorized Deletion of Customer Feedback (Five-Star Feedback)
Severity	**High**
CVSS v3.1 Score	8.2
CVSS Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:H/A:N
OWASP Category	A01:2021 — Broken Access Control
Affected URL	http://localhost:3000/#/administration DELETE http://localhost:3000/api/Feedbacks/{id}
CWE	CWE-284: Improper Access Control

29.1 Description

The administration console at

concentrates highly sensitive functionality — the complete

registered-user list and all customer-feedback records — together with destructive management actions such as feedback deletion. Access to this console and to its underlying REST endpoints is gated solely by the application's **admin** role.

That role provides no meaningful protection here, because it can be obtained without valid credentials by exploiting

a SQL Injection authentication bypass in the login form: supplying the Email payloadauthenticates

the first user (the administrator) with no valid password. Once the admin role is held, customer-feedback entries can

be permanently removed one by one through the authorization check, confirmation step, or integrity safeguard.

endpoint, with no additional

29.2 Proof of Concept

Steps to Reproduce:

1. Authenticate as the administrator by exploiting the login-form SQL injection (Email OR any password).
2. Navigate directly to ; the page renders without any secondary authorization prompt.
3. In the Customer Feedback panel, identify every entry rated 5 stars.
4. Click the trashcan control next to each 5-star entry. Each click issues an authenticated request (HTTP 200).
5. After the last 5-star entry is removed, the application confirms: “You successfully solved a challenge: Five-Star Feedback”.

1. Impact

An attacker who has escalated to the administrator role — trivially, via the login-form SQL injection — can read the entire registered-user list and every customer-feedback record, and can permanently destroy those records. This results in an irreversible loss of data integrity, exposure of customer identifiers, and the ability to suppress feedback to manipulate the store's public reputation. The same weak access control governs many equally destructive operations across the administrative surface.

1. Remediation

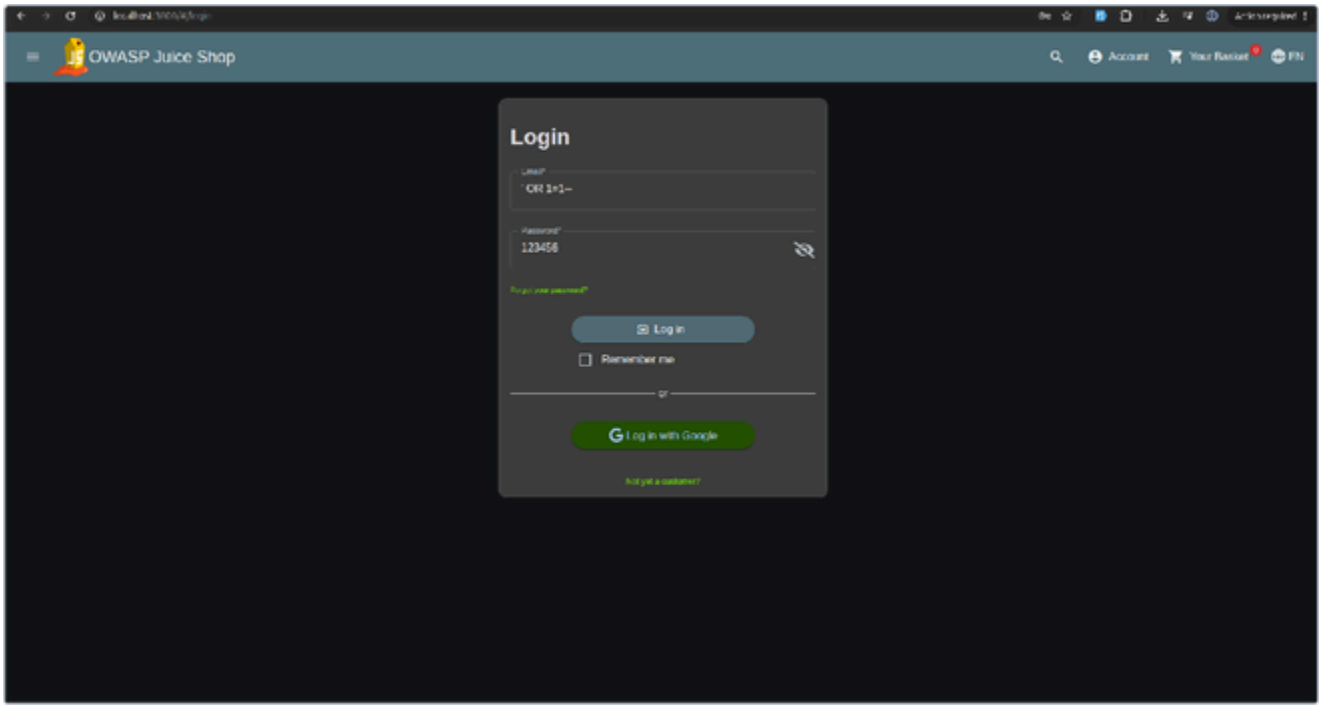
- **Enforce authorization server-side on every admin endpoint:** re-verify the caller's role on the server for every

administrative request, including.

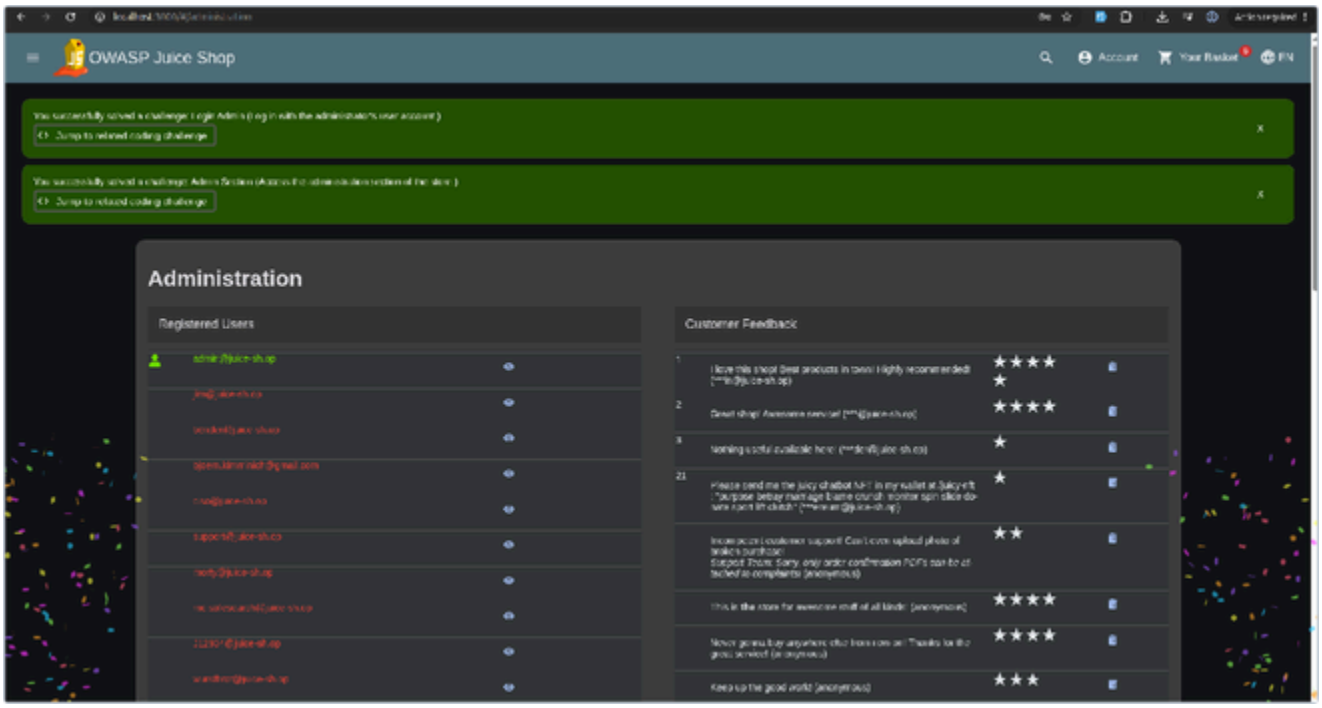
- **Fix the root-cause SQL injection in the login form** so the admin role cannot be acquired without valid credentials.
- **Apply least privilege and separation of duties** for destructive operations.
- **Add integrity safeguards and auditing:** confirmation prompts, deletion logging, and soft-deletion for recoverability.

1. References

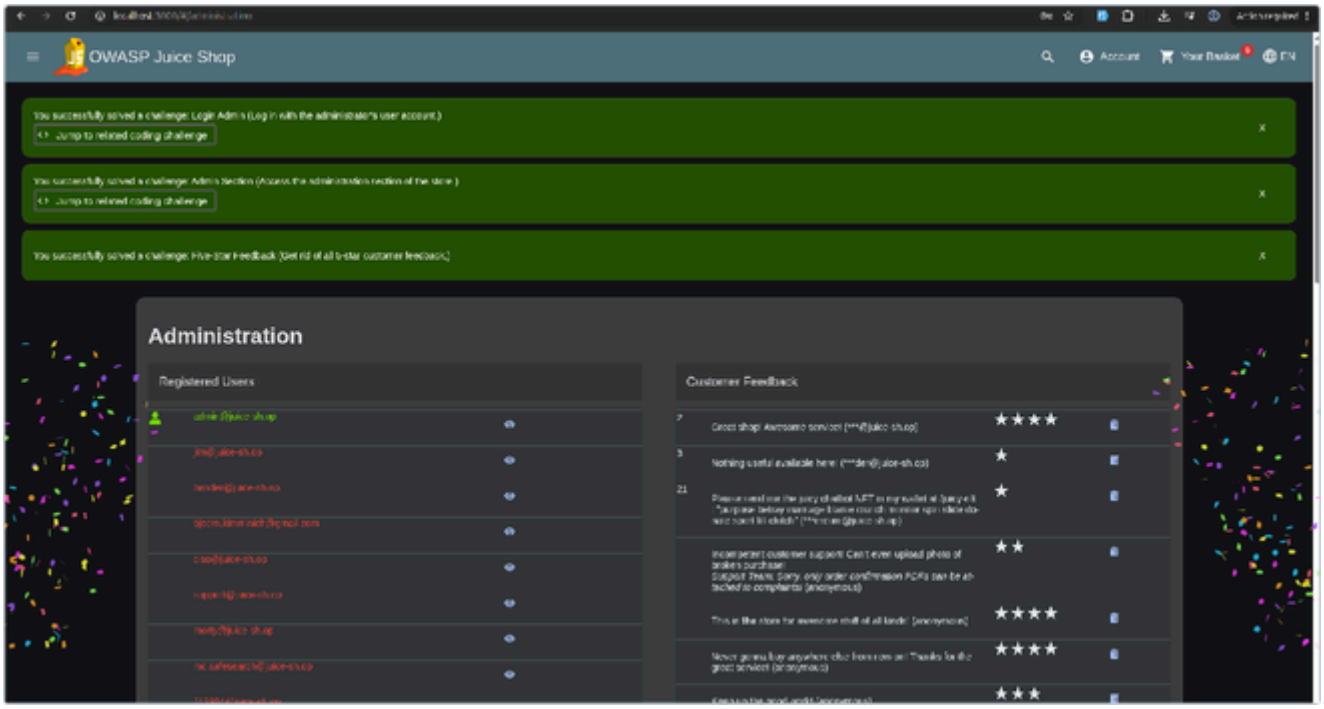
- OWASP A01:2021 – Broken Access Control: https://owasp.org/Top10/A01_2021-Broken_Access_Control/
- CWE-284: https://cwe.mitre.org/data/definitions/284.html



. Administrator session obtained via the SQL Injection authentication bypass in the login form.



. The administration console exposes all customer feedback (including 5-star entries) with delete controls, alongside the full registered-user list.


OWASP Juice Shop
Account
Your Basket
EN
You successfully solved a challenge: Login Admin (Log in with the administrator's user account.)
Jump to related coding challenge
You successfully solved a challenge: Admin Section (Access the administration section of the store.)
Jump to related coding challenge
You successfully solved a challenge: Five-Star Feedback (Get rid of all 5-star customer feedback.)
Administration
Registered Users
Customer Feedback
Great shop! Awesome service! (***@juice-sh.op)
Nothing useful available here! (***der@juice-sh.op)
This is the store for awesome stuff of all kinds! (anonymous)
Never gonna buy anywhere else from now on! Thanks for the great service! (anonymous)

VUL-027— Account Takeover via Weak, OSINT-Recoverable Password

Finding Metadata	Value
ID	VUL-027
Title	Account Takeover via Weak, OSINT-Recoverable Password (Login MC SafeSearch)
Severity	**High**
CVSS v3.1 Score	8.2
CVSS Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N
OWASP Category	A07:2021 — Identification and Authentication Failures
Affected URL	http://localhost:3000/#/login POST http://localhost:3000/rest/user/login
CWE	CWE-521: Weak Password Requirements

1. Description

The application accepts a weak, easily guessable password for the customer account

and enforces no protection against credential guessing — no password-strength policy, no multi-factor authentication, and no account lockout or rate limiting on the login endpoint.

The account holder further exposed the value of their own password through publicly available content. The music video **“Protect Ya' Passwordz”**, published under the **MC SafeSearch** persona, reveals that the password is the name of the user's pet (“Mr. Noodles”) with vowels replaced by zeroes. By combining trivial OSINT with the

application's tolerance for weak passwords, an attacker reconstructs the exact password

and authenticates as the legitimate user without any injection or bypass.

1. Proof of Concept

1. Obtain the target's email address, mc.safesearch@juice-sh.op from the administration console or public product reviews.
2. Perform OSINT on the “MC SafeSearch” persona, leading to the music video “Protect Ya' Passwordz (2014)”.
3. The track reveals the password: the pet name “Mr. Noodles” with vowels replaced by zeroes Mr. N00dles.
4. Login with Email mc.safesearch@juice-sh.op and password Mr.N00dles
5. Authentication succeeds with genuine credentials; the application confirms: “You successfully solved a challenge: Login MC SafeSearch”.

1. Impact

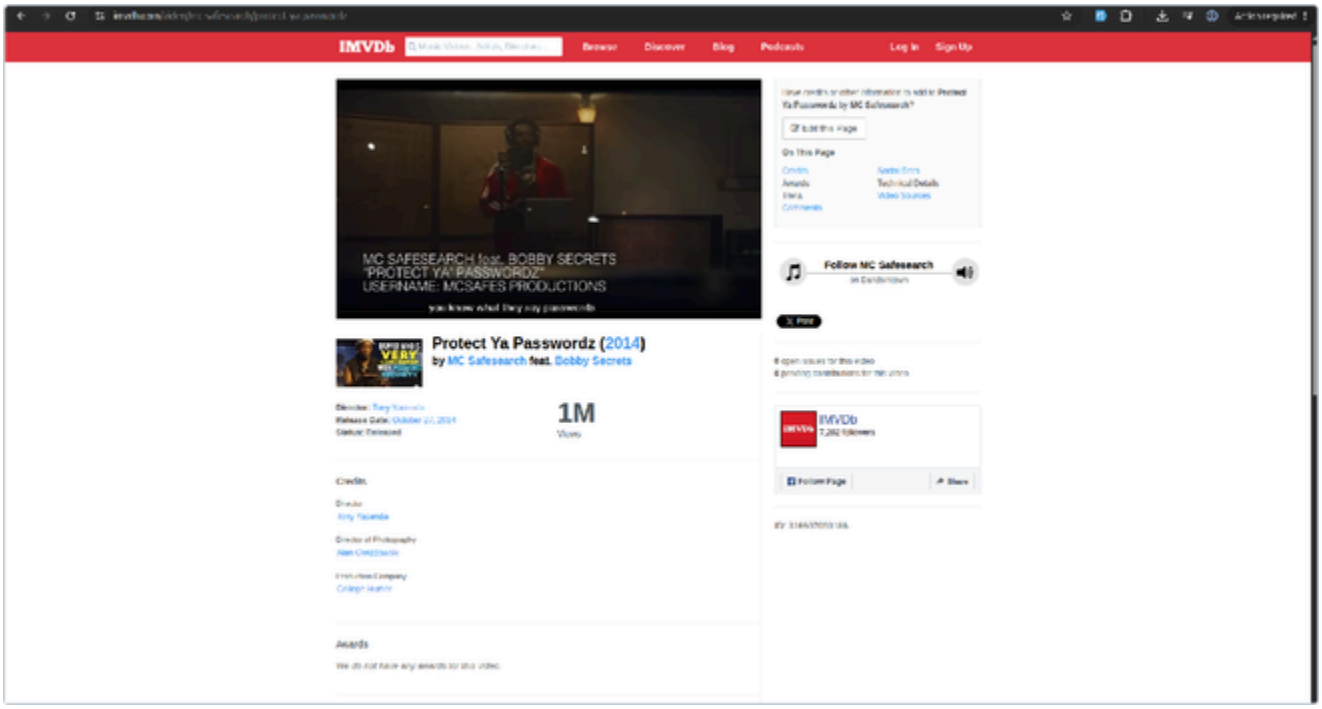
An attacker takes full control of the victim's account with no technical exploitation beyond a weak password — reading personal data, order history and basket, and acting on the victim's behalf. Because the application enforces no password policy, MFA, or lockout, the technique generalises to credential guessing, brute-force, and credential-stuffing against any account with a weak or leaked password. This account is a standard (non-administrative) customer, bounding the direct impact to a single user.

30.4 Remediation

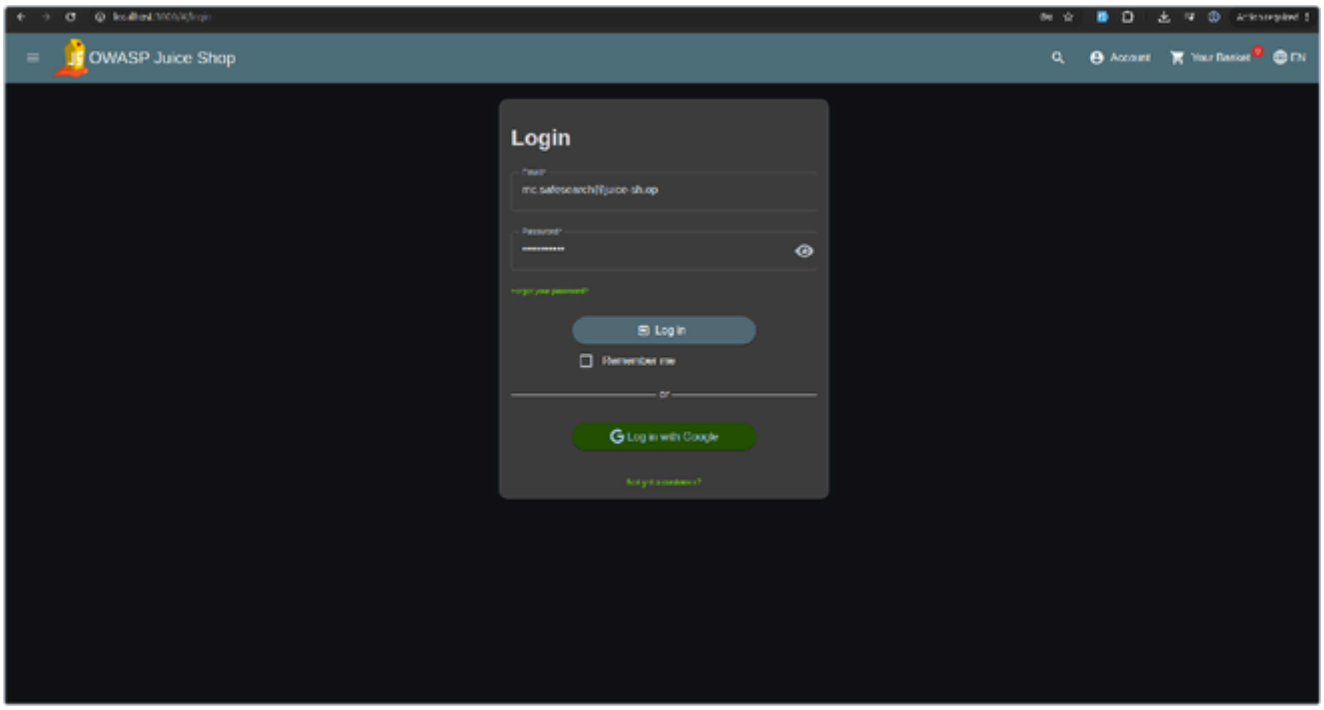
- **Enforce a strong password policy** and screen against breached / common-password lists.
- **Add anti-automation controls** (rate limiting, lockout, CAPTCHA) on.
- **Offer and encourage multi-factor authentication.**
- **Discourage passwords derived from OSINT-discoverable personal information.**

30.5 References

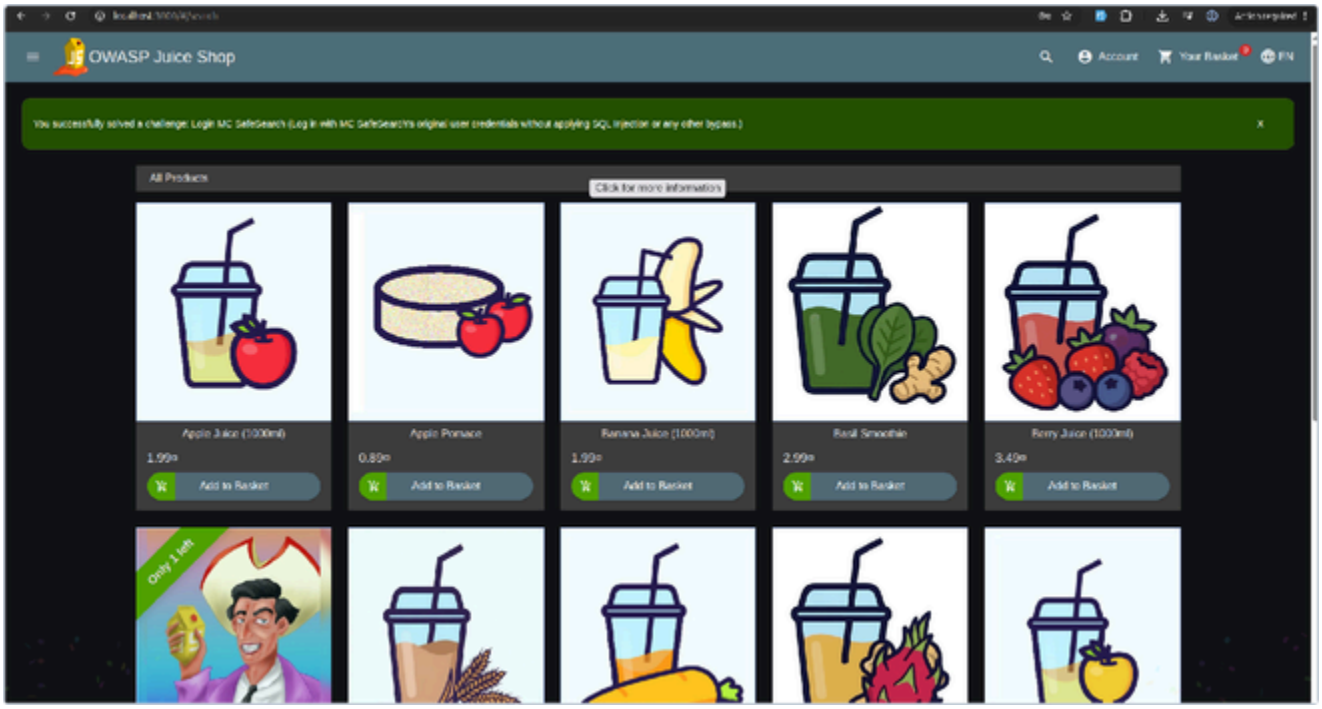
- OWASPA07:2021–IdentificationandAuthenticationFailures:https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/
- CWE-521: https://cwe.mitre.org/data/definitions/521.html



OSINT source — the publicly listed music video whose lyrics disclose the password strategy.



Authentication using the OSINT-recovered credentials, with no SQL Injection or bypass.



The “Login MC SafeSearch” challenge is confirmed solved, validating account takeover via a weak, publicly-recoverable password.

VUL-028 — Location Metadata Exposure Enabling Account Takeover

Finding Metadata	Value
ID	VUL-028
Title	Location Metadata Exposure Enabling Account Takeover via Password Reset (Meta Geo Stalking)
Severity	**High**
CVSS v3.1 Score	8.2
CVSS Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N
OWASP Category	A02:2021 — Cryptographic Failures (formerly Sensitive Data Exposure)
Affected URL	http://localhost:3000/#/photo-wall http://localhost:3000/#/forgot-password
CWE	CWE-200: Exposure of Sensitive Information to an Unauthorized Actor

31.1 Description

User-uploaded images on the Photo Wall are served without stripping their embedded EXIF metadata. An image published by the user **John** (caption: *"I love going hiking here... (© j0hNny)"*) retains GPS coordinates pinpointing where it was taken. Because the Forgot Password mechanism relies on a knowledge-based security question whose answer is that very location, an attacker extracts the coordinates from the image metadata, resolves them to a place name, answers John's security question, and resets his password — taking over the account without knowing the original credentials.

31.2 Proof of Concepts

1. On http://localhost:3000/#/forgot-password enter john@juice-sh.op to reveal his (hiking-related) Security Question
2. On the Photo Wall, locate John's hiking-trail upload and download the image.
3. Extract EXIF metadata (e.g.): GPS Position **36°57'31.38" N, 84°20'53.58" W**.
4. Resolve the coordinates on a map — the point is within **Daniel Boone National Forest**, Laurel County, Kentucky.
5. Return to Forgot Password, answer with

set a new password, and click Change.

1. The reset succeeds and the application confirms: **"You successfully solved a challenge: Meta Geo Stalking"**.

31.3 Impact

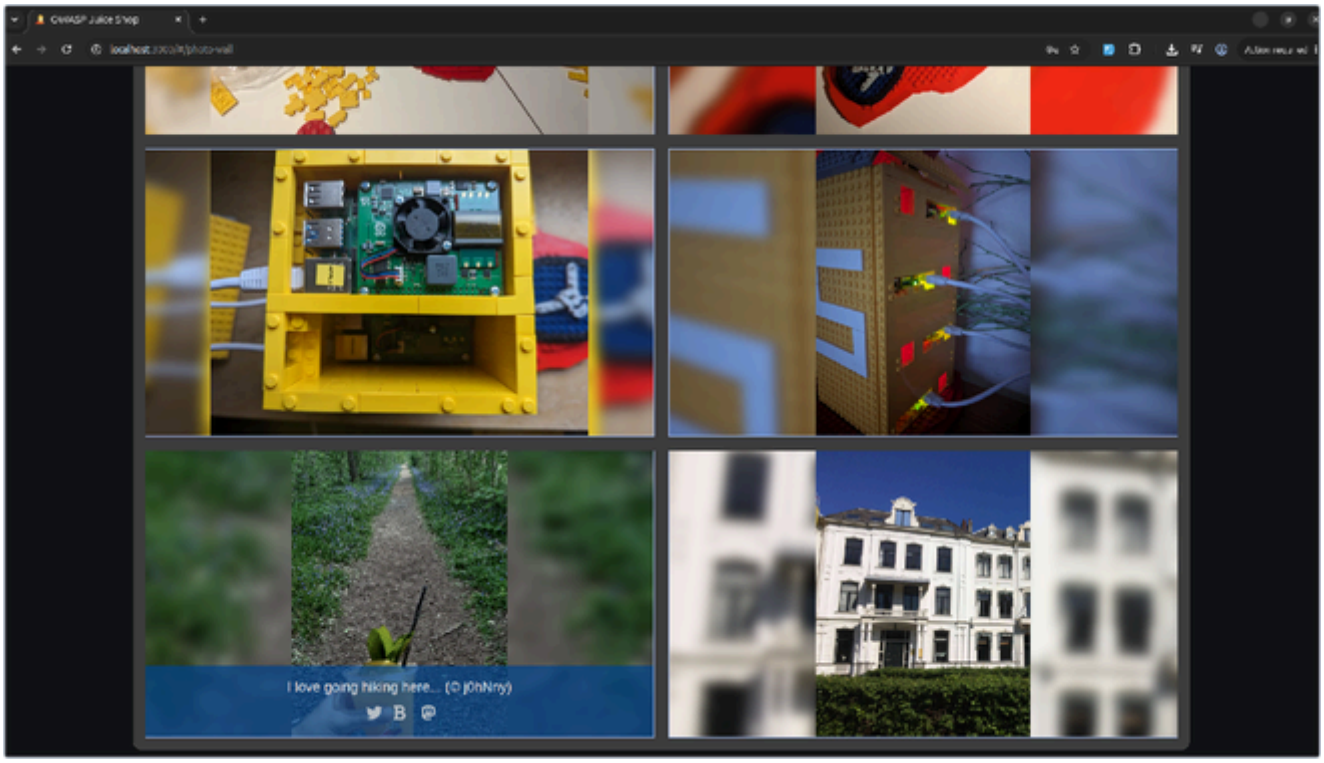
An attacker can take over any account whose owner uploaded an image containing location metadata that answers their security question — reading the victim's data and locking the owner out by changing the password. The metadata-stripping failure is **systemic**, applying to every image the platform serves, potentially exposing users' home or workplace locations — a privacy and physical-safety risk beyond account compromise.

31.4 Remediation

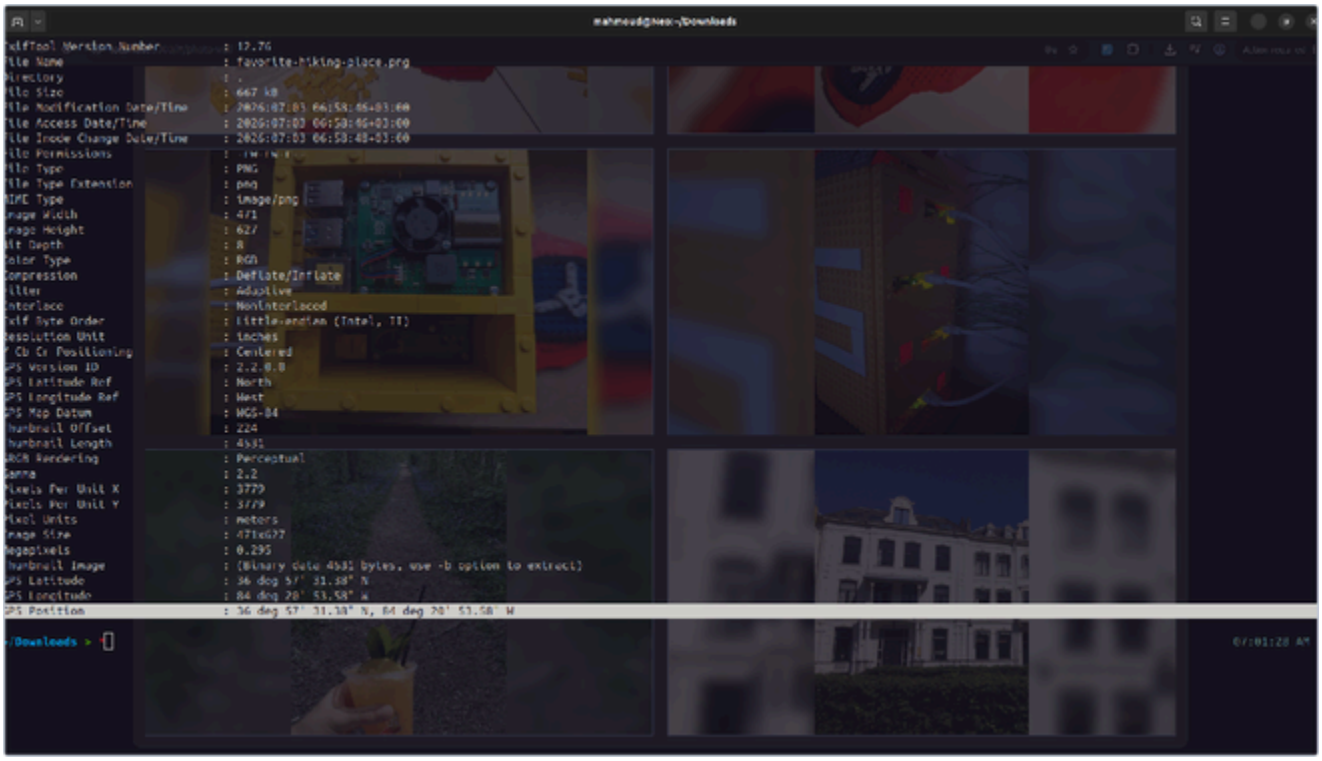
- **Strip metadata from all uploads** (especially GPS) on the server before storing or serving images.
- **Replace or strengthen knowledge-based recovery:** prefer email-based reset links or MFA-backed recovery; reject publicly-derivable answers and rate-limit attempts.
- **Avoid disclosing the security question to unauthenticated users** and return uniform responses to hinder enumeration.
- **Apply data minimisation** and treat location data as sensitive by default.

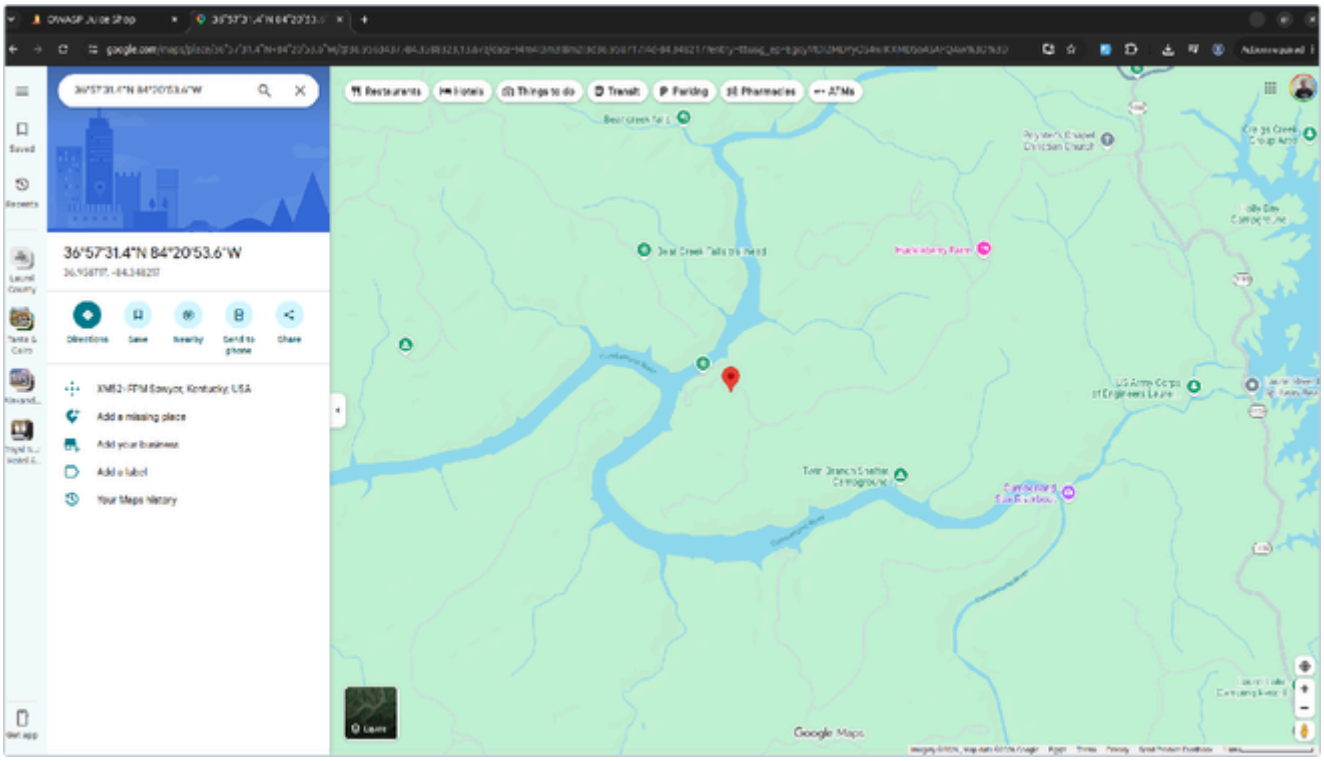
31.5 References

- OWASP A02:2021 – Cryptographic Failures: https://owasp.org/Top10/A02_2021-Cryptographic_Failures/
- CWE-200: https://cwe.mitre.org/data/definitions/200.html|CWE-640: https://cwe.mitre.org/data/definitions/ 640.html



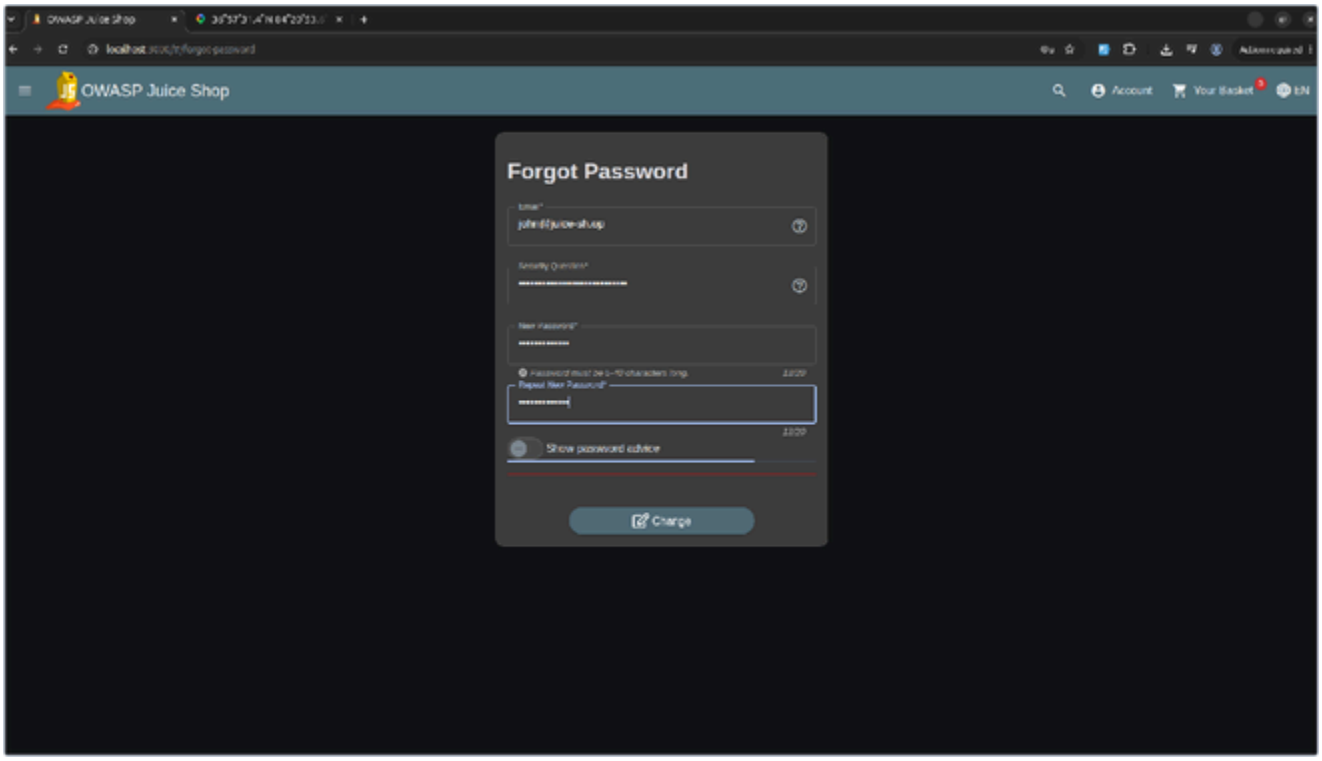
. John's Photo Wall upload — the source image carrying the location metadata.

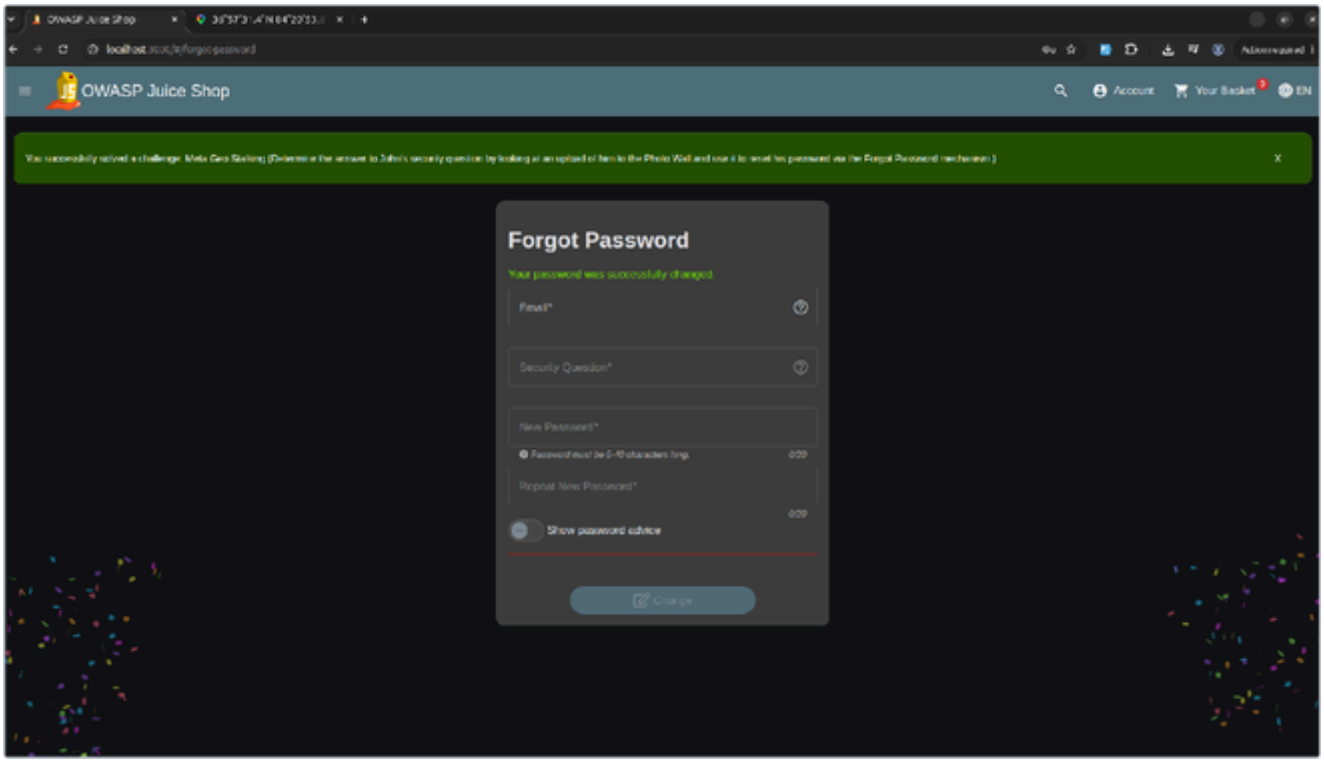




output revealing the unstripped GPS Position.

. The coordinates resolved to Daniel Boone National Forest — the answer to John's security question





The password is reset and the “Meta Geo Stalking” challeng

VUL-029 — Exposed Wallet Seed Phrase Enabling Crypto Wallet Takeover

Finding Metadata	Value
ID	VUL-029
Title	Exposed Wallet Seed Phrase Enabling Crypto Wallet (Soul Bound Token) Takeover (NFT Takeover)
Severity	**Critical**
CVSS v3.1 Score	9.1
CVSS Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N
OWASP Category	A02:2021 — Cryptographic Failures (formerly Sensitive Data Exposure)
Affected URL	`http://localhost:3000/#/about` (Customer Feedback) `http://localhost:3000/#/juicy-nft`
CWE	CWE-200: Exposure of Sensitive Information to an Unauthorized Actor

1. Description

A cryptographic wallet secret — a BIP39 mnemonic (seed phrase) — was accidentally published in plain text within publicly visible customer feedback on the “About Us” page. The twelve-word phrase deterministically derives the private key of the Ethereum wallet holding the shop's official Soul Bound Token (SBT / NFT). Any unauthenticated visitor can read the phrase, derive the private key with a standard offline BIP39 tool, and authenticate to

as the wallet owner — taking full control of the wallet and its token.

1. Proof of Concept

Steps to Reproduce:

1. Open a web browser and navigate to the target OWASP Juice Shop environment on TryHackMe.
2. Inspect the application's client-side source code or build artifacts to uncover an exposed cryptographic mnemonic seed phrase belonging to an administrative or target user account.
3. Access an external cryptographic utility, such as a **Mnemonic Converter** tool.
4. Input the discovered mnemonic seed phrase into the converter tool to derive the associated wallet addresses and extract the corresponding plain-text **Private Key**.
5. Navigate back to the web application and open the dedicated **Juicy NFT** management or marketplace interface at `[http://10.10.](http://10.10.)x.x/#/nft-marketplace`.
6. Inject the extracted **Private Key** into the target authentication or wallet interaction field within the Juicy NFT view.
7. The application processes the raw private key without secondary verification or multi-factor authorization checkpoints, granting immediate unauthorized ownership access and completing the high-severity NFT takeover.

1. Impact

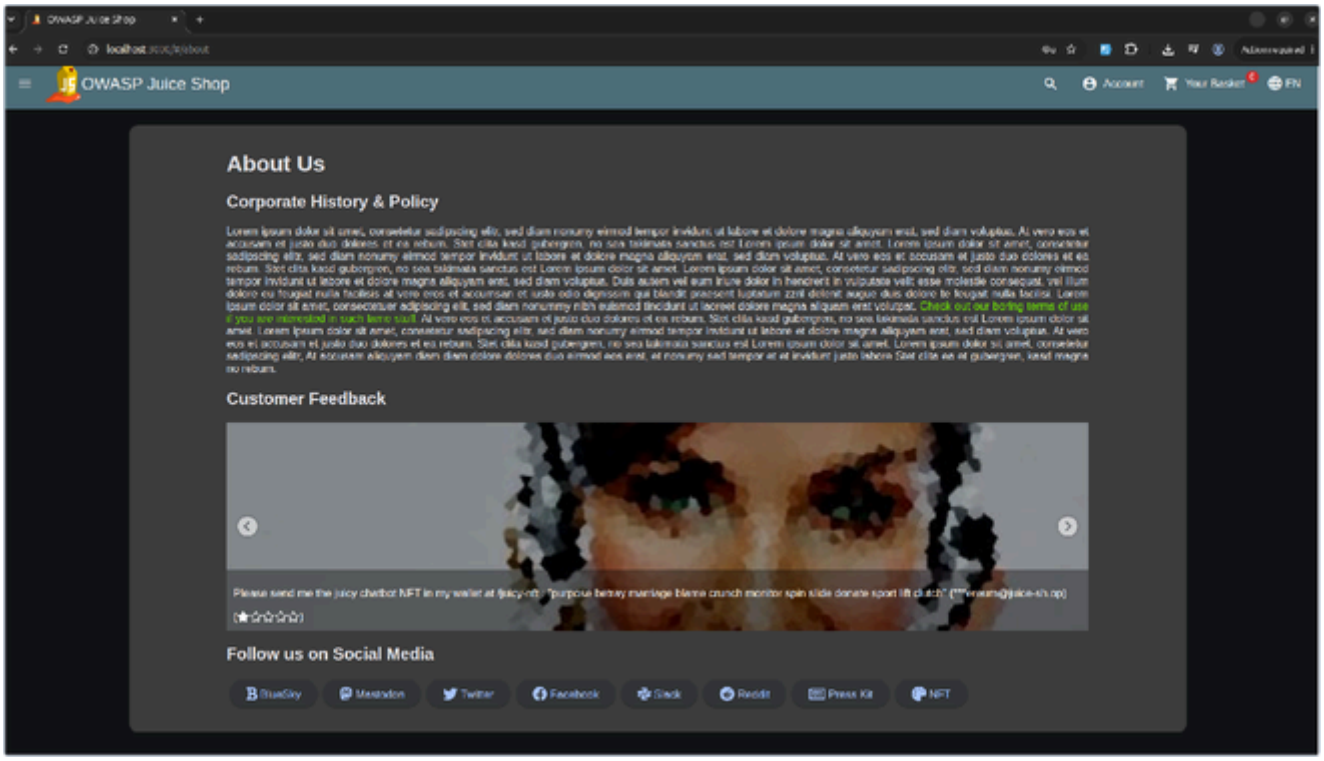
Publishing a wallet's seed phrase in cleartext hands complete, irreversible control of that wallet to anyone who reads it. Here the attacker takes ownership of the wallet holding the official Soul Bound Token; for any wallet holding transferable assets the same exposure would permit immediate, permanent theft of all funds with no possibility of recovery. Exploitation requires no authentication, no privileges, and only a freely available derivation tool.

1. Remediation

- **Never expose secrets in user-facing content** (feedback, reviews, comments, logs, or any client-accessible surface).
- **Treat the exposed wallet as fully compromised** and migrate all assets to a new wallet from a fresh secret seed.
- **Add content inspection / DLP** to detect and reject high-entropy secrets and key / seed-phrase patterns before display.
- **Enforce secret hygiene** via a secrets manager and rotation of any exposed credential.

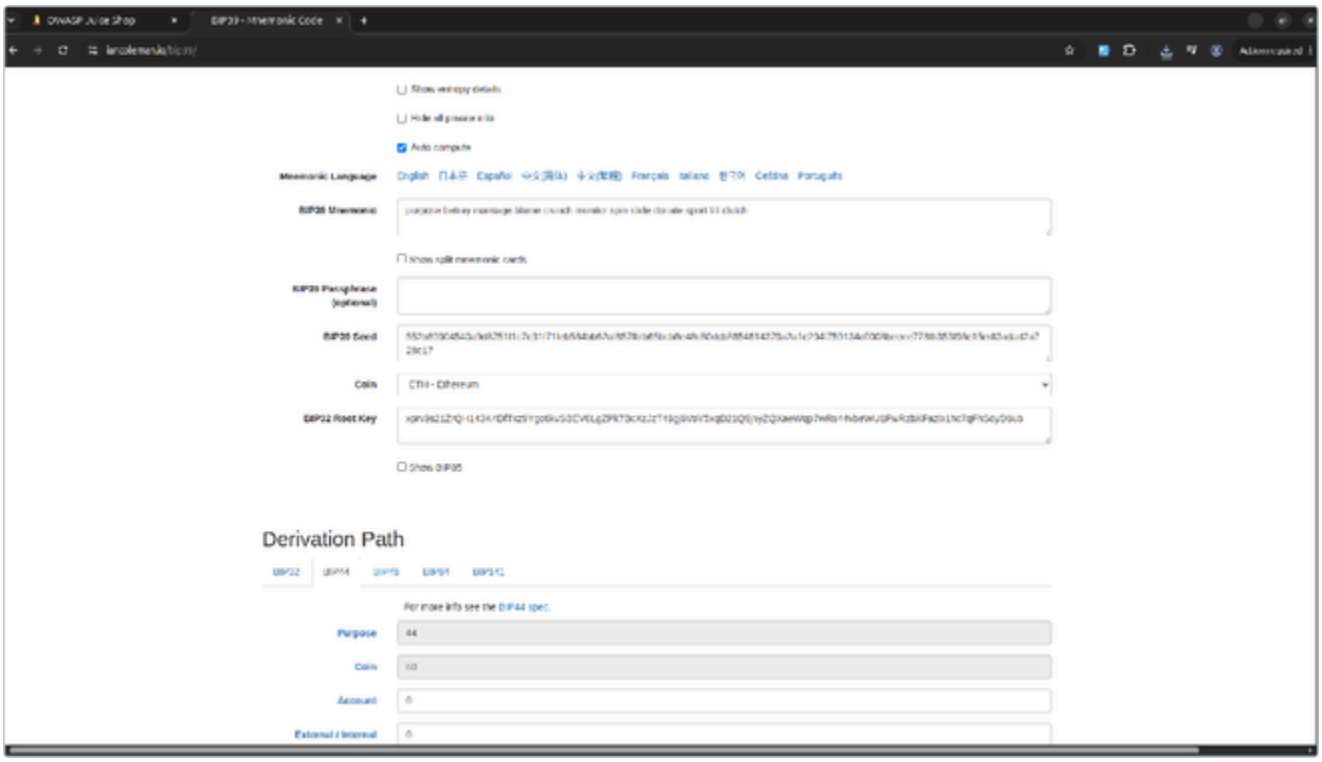
1. References

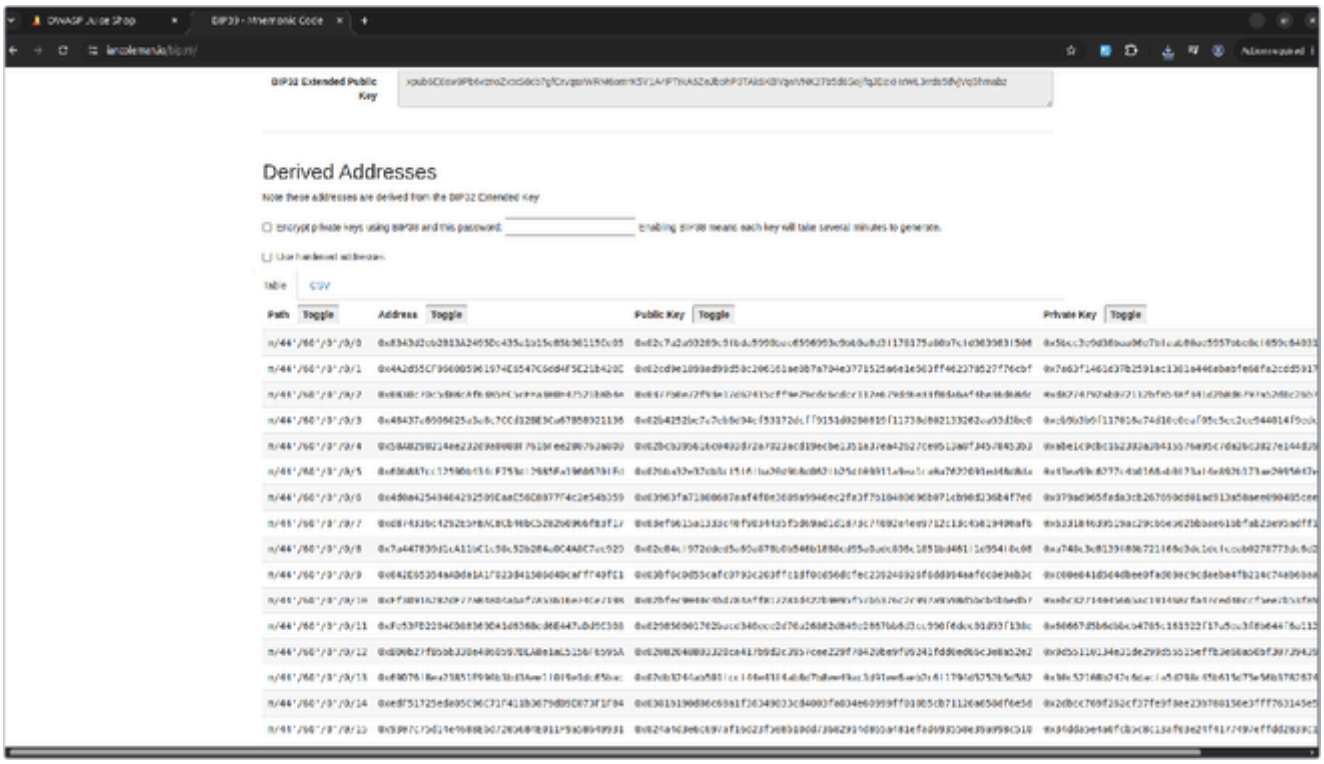
- OWASP A02:2021 – Cryptographic Failures: https://owasp.org/Top10/A02_2021-Cryptographic_Failures/
- CWE-200: https://cwe.mitre.org/data/definitions/200.html|CWE-522: https://cwe.mitre.org/data/definitions/ 522.html



The Customer Feedback carousel, where a review accidentally leaks the wallet seed phrase and points to

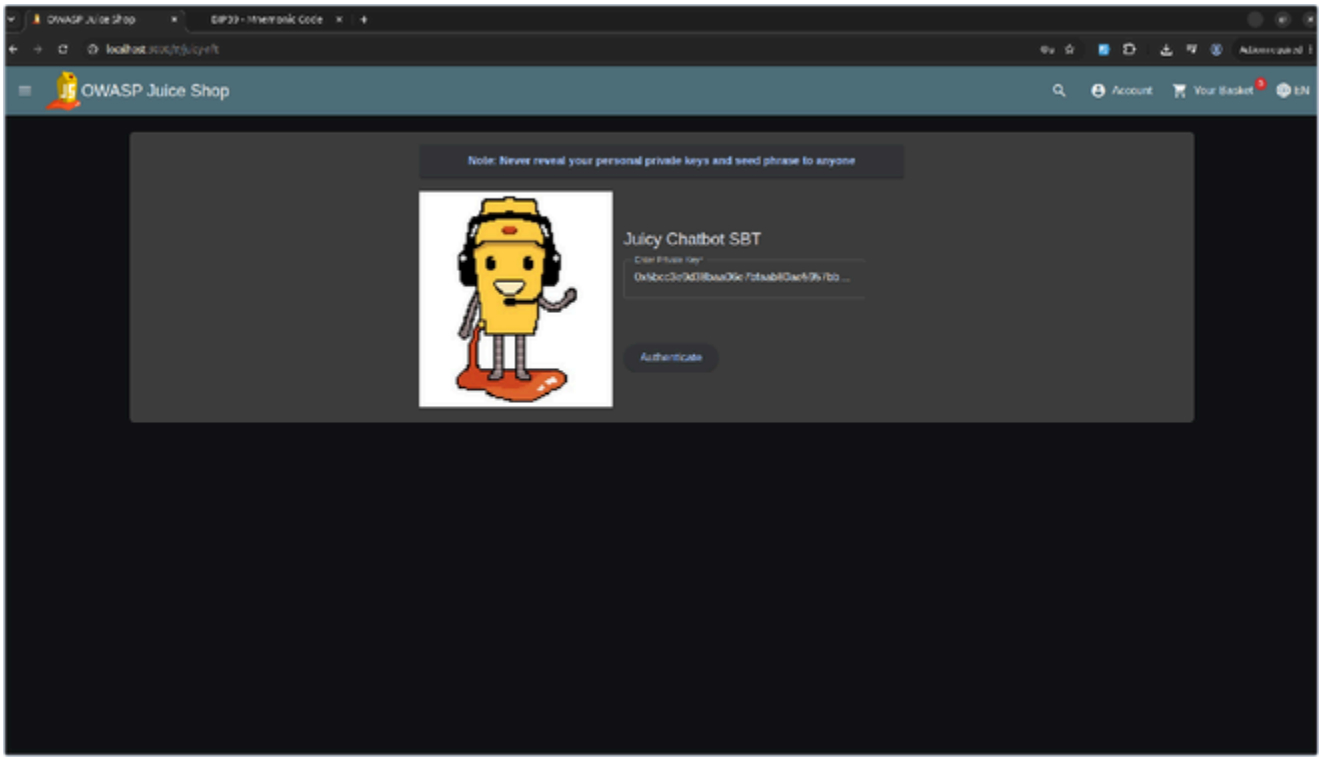
.

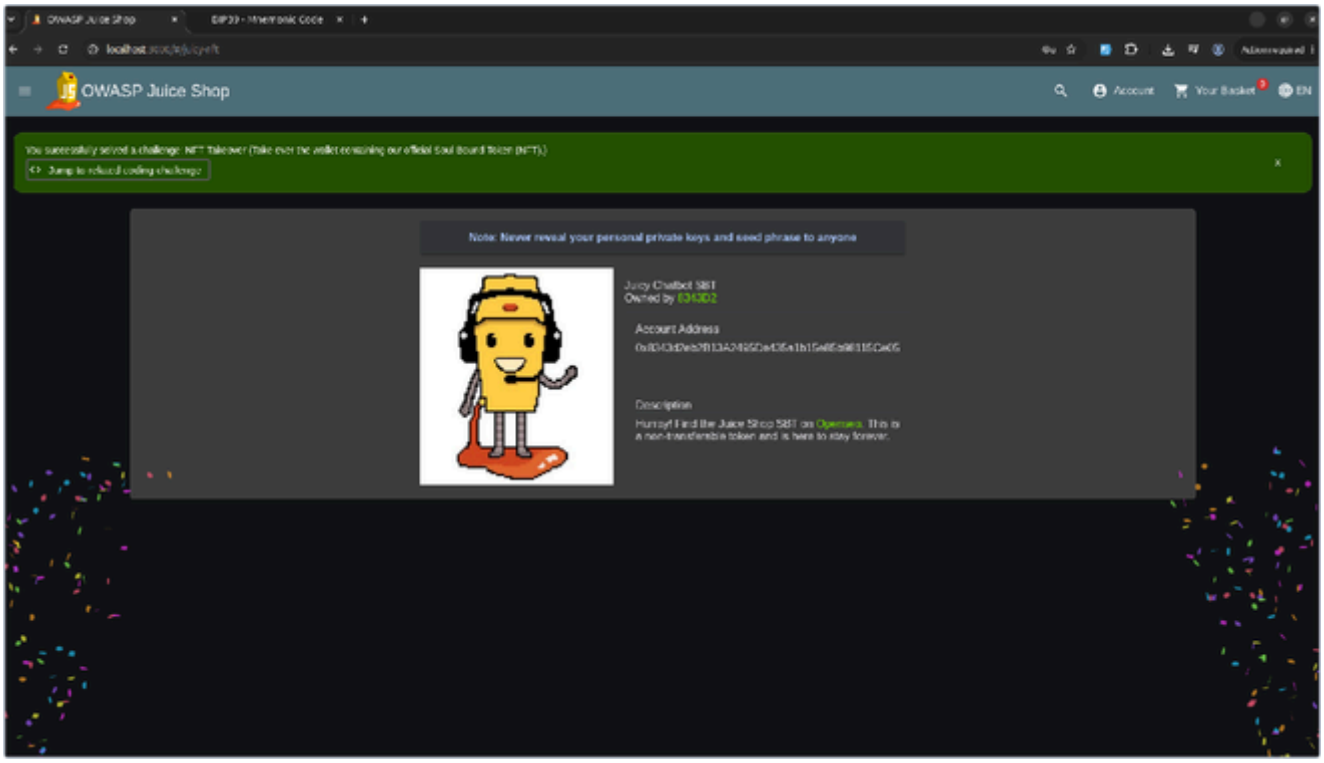




The mnemonic entered into an offline BIP39 tool with Coin set to ETH, producing the seed and root key.

. The Derived Addresses table — the first wallet exposes its corresponding private key.


OWASP Juice Shop
Account
Your Basket
EN
Note: Never reveal your personal private keys and seed phrase to anyone
Juicy Chatbot SBT
Authenticate


OWASP Juice Shop
Account
Your Basket
EN
Note: Never reveal your personal private keys and seed phrase to anyone
Juicy Chatbot SBT
Account Address
Description

VUL-030 — Published Security Policy (security.txt)

Finding Metadata	Value
ID	VUL-030
Title	Published Security Policy (security.txt) — Responsible-Disclosure Reconnaissance & Best-Practice Observation (Security Policy)
Severity	**LOW**
CVSS v3.1 Score	N/A (0.0)
CVSS Vector	N/A — not a vulnerability
OWASP Category	N/A — Informational (Responsible Disclosure / RFC 9116)
Affected URL	http://localhost:3000/.well-known/security.txt
CWE	N/A — Informational

33.1 Description

Before active testing, an ethical (“white-hat”) researcher should locate the target's published security policy. The application exposes afile at the RFC 9116 standard location,

defining the security contact, encryption key, acknowledgements page, and related disclosure metadata. Locating and reading it is the intended, ethical first action, and is exactly the behaviour this exercise rewards.

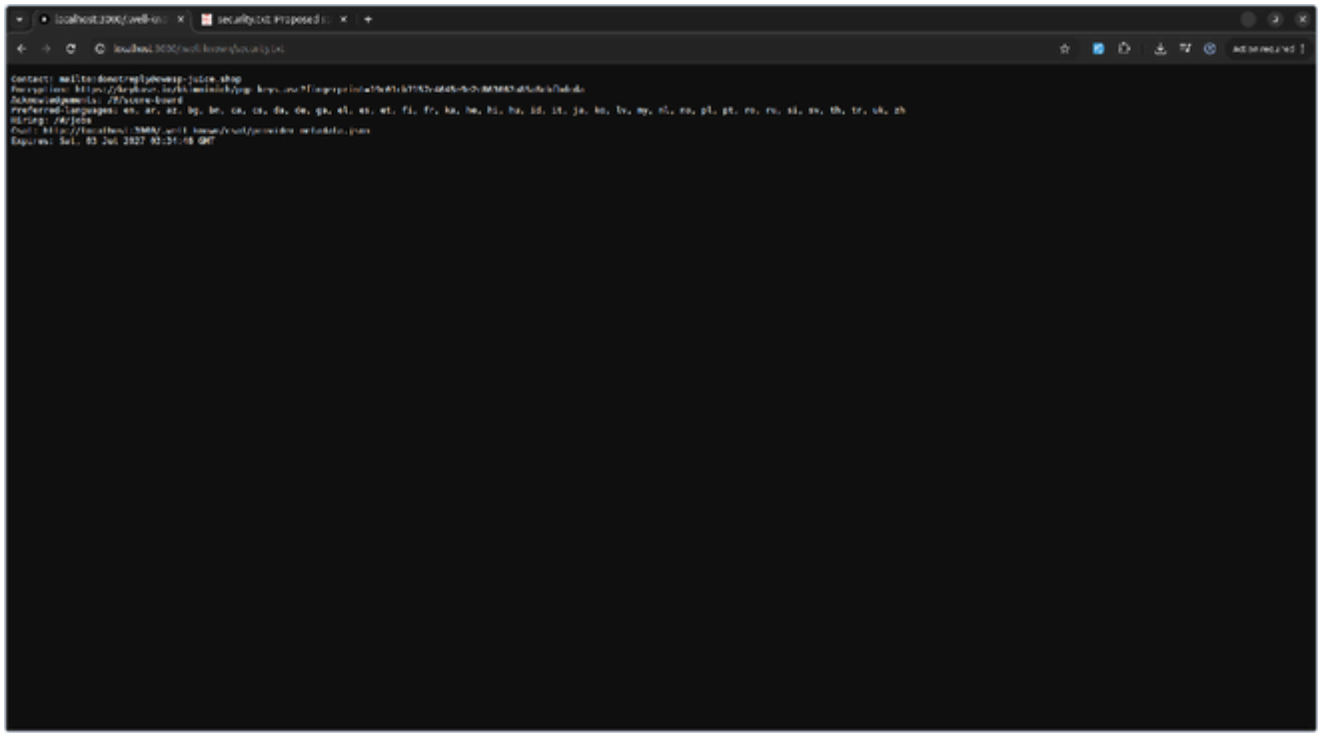
1. **Steps to Reproduce**
    1. Confirm the canonical location from the standard (securitytxt.org / RFC 9116):,

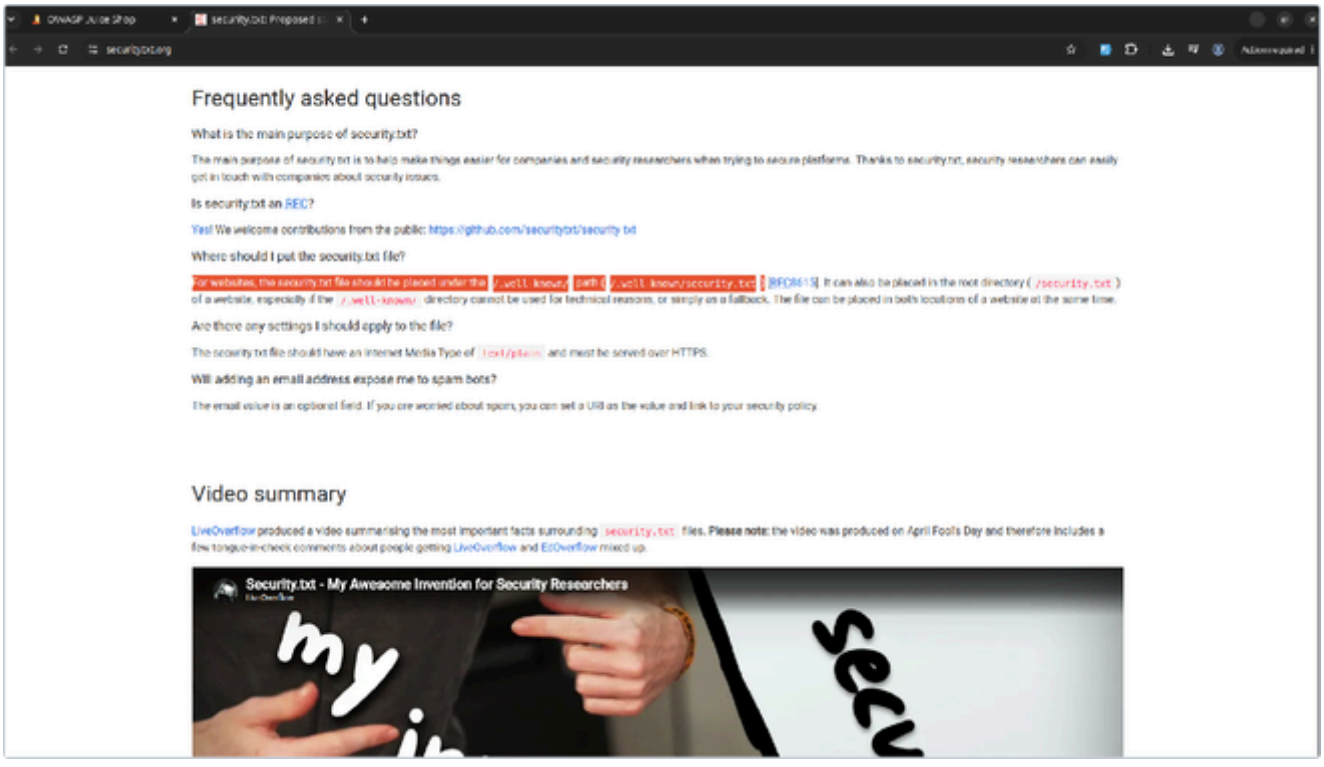
served as

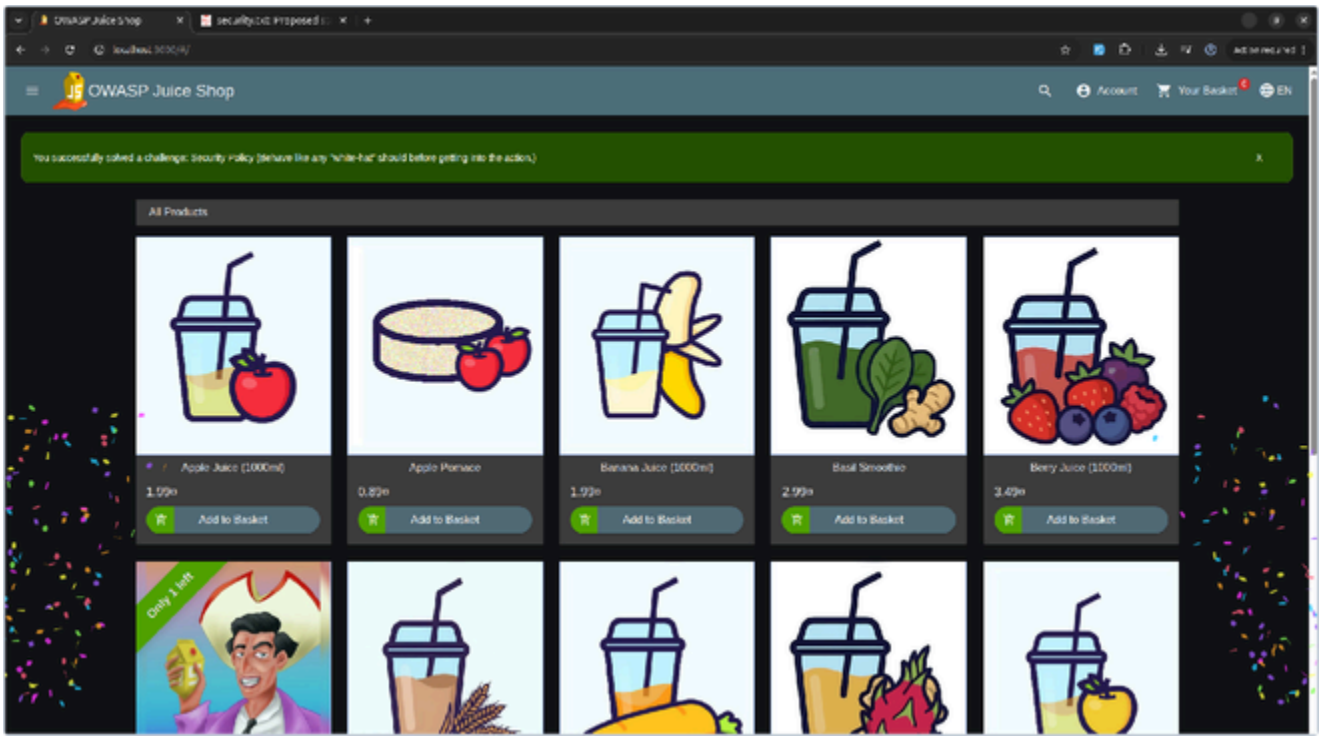
1. Request

over HTTP

1. The server returns the policy: Contact: mailto:donotreply@owasp-juice.shop, an Encryption PGP key, Acknowledgements: /#/score-board, Preferred-Languages, Hiring: /#/jobs, a Csaf URL, and Expires: Sat, 03 Jul 2027.
2. Reviewing the policy satisfies the white-hat expectation; the application confirms: “You successfully solved a challenge: Security Policy”.







The “Security Policy” challenge is confirmed solved, rewarding the white-hat step of reviewing the disclosure policy before testing.

Conclusion

This assessment identified 30 distinct findings across OWASP Juice Shop, ranging from Low to Critical severity. The most severe issues are the SQL Injection authentication-bypass flaws (VUL-006, VUL-017, VUL-018), which allow complete, unauthenticated takeover of any account including the administrator, and the Stored/DOM-based Cross-Site Scripting findings (VUL-002, VUL-003, VUL-011, VUL-019, VUL-025), which allow arbitrary script execution in victims' browsers. Remediation should prioritize parameterized database queries and consistent, context-aware output encoding first, followed by re-establishing server-side authorization checks on every endpoint identified in the Broken Access Control findings.